STUDENT ACADEMIC PERFORMANCE DECISION SUPPORT SYSTEM FOR FURTHER EDUCATION

OMETH DAHAMVIN ILAMPERUMA

Candidate Number: 0362

Centre Number: 12144

Contents

ANALYSIS CHAPTER

AN OUTLINE OF THE PROBLEM AND PROPOSED SOLUTION

In further education institutes in the UK, monitoring and analysing student academic performance is often a time-consuming and largely manual process. Teachers are required to extract student marks from the Virtual Learning Environment (VLE), calculate averages, determine risk levels, estimate predicted grades, and record suggested interventions. These tasks are frequently completed using spreadsheets or separate documents, which increases the likelihood of human error, duplication of work, and inconsistent decision-making. Additionally, generating clear and visual summaries of performance for parents and academic management can be inefficient and lacks standardisation.

Another key issue is the early identification of at-risk students. Without an automated system, teachers must manually interpret grade data to determine which students require additional support. This can delay intervention, potentially affecting student progression and achievement. Furthermore, parents often have limited, delayed, or fragmented access to meaningful information about their child's academic performance, making it difficult for them to provide timely support.

The proposed Student Performance Decision Support System (DSS) aims to address these challenges by providing a centralised, computational solution that improves efficiency, accuracy, transparency, and early intervention in student performance management. For this project, I am planning to develop a Student Performance Decision Support System (DSS) for further education institutes in the UK. The system will enable teachers to track student grades, identify at-risk students, generate structured reports, produce module-wise grade distribution graphs, and suggest appropriate interventions. It will also provide parents with clear and accessible information about their child's academic progress. Through automated analysis, the system will determine risk levels, highlight students who may require additional support, and calculate predicted grades for summative examinations, ensuring timely and informed decision-making.

For this system to function effectively, it must support the export of student formative exam data for a given module. The required data includes student ID, student name, module name, average mark, attendance and homework completion. This information can be extracted from the institution's Virtual Learning Environment (VLE) as a CSV file. The CSV file can then be uploaded into the Student Performance Decision Support System (DSS), allowing the system to store, organise, and retrieve student records efficiently.

STAKEHOLDERS

The primary stakeholders of the system are teachers and parents:

- Teachers: Will use the system to efficiently track student performance, identify issues, and make data-driven decisions.
- Parents: Will use the system to monitor their child's progress and engage with college interventions.

I have selected Ms. Smith, a Computer Science tutor at ABC College in the UK, to represent the primary target user of the system. She has daily responsibility for tracking student grades, calculating predicted grades, identifying risk levels, and recording intervention strategies. Currently, she performs these tasks manually using Google Documents before transferring the information into a separate computerised system where students can view their grades and feedback.

This process is time-consuming, repetitive, and increases the risk of human error. Ms. Smith has clearly expressed the need for a more automated solution that provides accurate calculations, clear visual summaries, and efficient report generation. Maintaining regular communication with her throughout the development process ensures that the system is designed around real classroom practices and directly addresses the practical challenges faced by teachers.

HOW THE PROBLEM CAN BE SOLVED BY COMPUTATIONAL METHODS

This problem is well-suited for computational methods because the DSS involves data processing, pattern recognition, and algorithmic decision-making. Using computational thinking:

THINKING ABSTRACTLY

Abstraction allows unnecessary complexity to be removed so that the system can focus on the most important elements required for decision-making. In this project, students, grades, modules, and intervention strategies are represented as structured data objects, enabling the system to process and manage information efficiently. Instead of storing excessive historical details, the system generates simplified performance indicators such as predicted grade and risk level to provide clear and meaningful summaries. Student performance is visualised through charts rather than raw numerical data, making trends and concerns easier to interpret. Additionally, role-based access is implemented to abstract the differences between teacher and parent views, ensuring that each user sees only the information relevant to their role while maintaining data privacy.

THINKING AHEAD

Thinking ahead is essential to ensure that the system is developed in a structured and organised manner. The primary inputs to the system include student names, module details, grades, and average scores imported from a CSV file. From these inputs, the system produces meaningful outputs such as individual PDF reports, performance graphs, risk alerts, and suggested interventions. The solution is implemented using Python as the programming language, with Tkinter for the graphical user interface, Matplotlib for data visualisation, ReportLab for PDF report generation, and SQLite for data storage and management. In terms of user interaction, teachers are able to import data, search for students, analyse performance, and generate reports, while parents can securely access their child's performance summary and downloadable PDF reports.

THINKING PROCEDURALLY

Thinking procedurally involves breaking the system down into clear, manageable components so that it can be developed and implemented in a logical, step-by-step manner. The system is organised into several functional modules. These include importing student performance data from a CSV file, searching and retrieving specific student records, calculating predicted grades, identifying at-risk students based on predefined criteria, and recommending appropriate intervention strategies. The reporting module generates module-wise visual summaries and produces structured PDF reports for teachers and parents. Finally, the access control component ensures that teachers have full access to all student records and analytical features, while parents are restricted to viewing only their own child's performance information, thereby maintaining data privacy and security.

THINKING LOGICALLY

Thinking logically ensures that the system operates correctly and produces accurate, rule-based outcomes. The system applies conditional logic to analyse student data and make decisions automatically. For example, if a student's average falls below a predefined threshold, the system flags the student as at-risk and enables appropriate intervention recommendations. When a parent logs into the system, access control logic ensures that only their child's performance data is displayed, protecting student privacy. Iterative processes are used to calculate class averages and identify performance trends across modules. After all necessary calculations and analysis have been completed, reports are generated automatically based on predefined rules, ensuring consistency, accuracy, and efficiency in decision-making.

THINKING CONCURRENTLY

Efficiency in the system is improved by running operations concurrently. For example, while one process calculated student grades and determines at-risk status, another process can simultaneously update data visualisations such as module grade distributions. Similarly, PDF reports can be generated in the background while the teacher continues to interact with the

system, allowing multiple tasks to be completed without unnecessary waiting and improving overall responsiveness and user experience.

RESEARCH

Research – Interview with Teacher: Initial Requirements

To design a system that truly fits classroom needs, an interview was conducted with Ms. Smith, a Computer Science tutor at ABC College in the UK. The goal was to understand her current workflow, challenges, and the features she would like to see in a student performance tracking system.

Plan

The interview aimed to explore the teacher's daily workflow, difficulties in managing student performance, and desired features for a performance tracking system.

Questions Asked

1. How do you currently track student performance?
2. What challenges do you face when calculating grades?
3. What reports or visualisations would be helpful?
4. Should the system provide intervention suggestions?

Transcript Summary

From the discussion, several key points emerged:

- Manual tracking is time-consuming and repetitive.
- Calculating predicted grades and averages by hand increases the risk of errors.
- Identifying at-risk students requires manual review, delaying interventions.
- Reports for parents need to be clear and easy to interpret.
- Automated grade predictions and risk identification would save significant time.
- Visual indicators, dashboards, and alerts are essential for spotting performance issues quickly.

Detailed Analysis of Current Problems

Excessive Administrative Workload

Teachers spend significant time exporting data, entering it into spreadsheets, calculating averages, predicting grades, and determining risk levels manually.

Increased Risk of Human Error

Manual data handling can lead to mistakes in calculations, grade predictions, or risk classification, which may affect academic decisions and parent communication.

Inconsistent Decision-Making

Without structured rules, teachers may apply slightly different criteria when assessing student performance, leading to inconsistencies across classes or modules.

Delayed Identification of At-Risk Students
Manually analysing grades slows the detection of performance drops, delaying timely support for struggling students.

Fragmented Data Storage
Student performance data is often scattered across multiple documents or systems, making it harder to retrieve and increasing the likelihood of outdated or inconsistent records.

Limited Visual Insight
Spreadsheets display raw numbers that do not easily highlight trends, gaps, or grade distributions. Teachers need clear visual summaries for quick, informed decision-making.

Time-Consuming Parent Reporting
Preparing reports manually takes time and may result in inconsistent communication to parents.

HOW THE PROPOSED SYSTEM ADDRESSES THESE NEEDS

Automated Calculations

Once a CSV file is uploaded, averages, predicted grades, and risk levels are calculated automatically, reducing workload and errors.

Consistent Rule-Based Analysis

Predefined logic ensures fair and consistent assessment of at-risk students across all modules.

Early Risk Alerts: Students below defined thresholds are automatically flagged, allowing proactive support.

Centralised Data Management

A structured SQLite database stores all student records efficiently for easy retrieval.

Visual Dashboards and Graphs

Grade distributions and performance charts provide instant visual insight.

Automated PDF Report Generation

Standardised, professional reports for teachers and parents reduce preparation time.

Secure Role-Based Access

Teachers have full access, while parents see only their child's data, ensuring privacy and compliance with data protection.

Research Outcome

The research highlights that teachers need a system that is accurate, automated, intuitive, and reduces administrative burden. They require consistent performance analysis, early

identification of at-risk students, and clear visual summaries to support decision-making. Parents need structured and accessible reports, with access securely restricted. The proposed Student Performance Decision Support System addresses these needs directly, offering a practical, user-focused solution that aligns with real classroom challenges.

ANALYSIS OF EXISTING SYSTEMS

TES Engage (Progress Tracking System)

TES Engage is a web-based progress tracking platform mainly designed for use on desktop systems within schools. It is a commercial system, meaning schools must purchase a licence to use it.

The platform focuses on monitoring student attainment and predicting future outcomes, such as GCSE grades. It analyses past assessment data, including mock exam results, to generate automatic target grades and highlight students who may be at risk of underperforming.

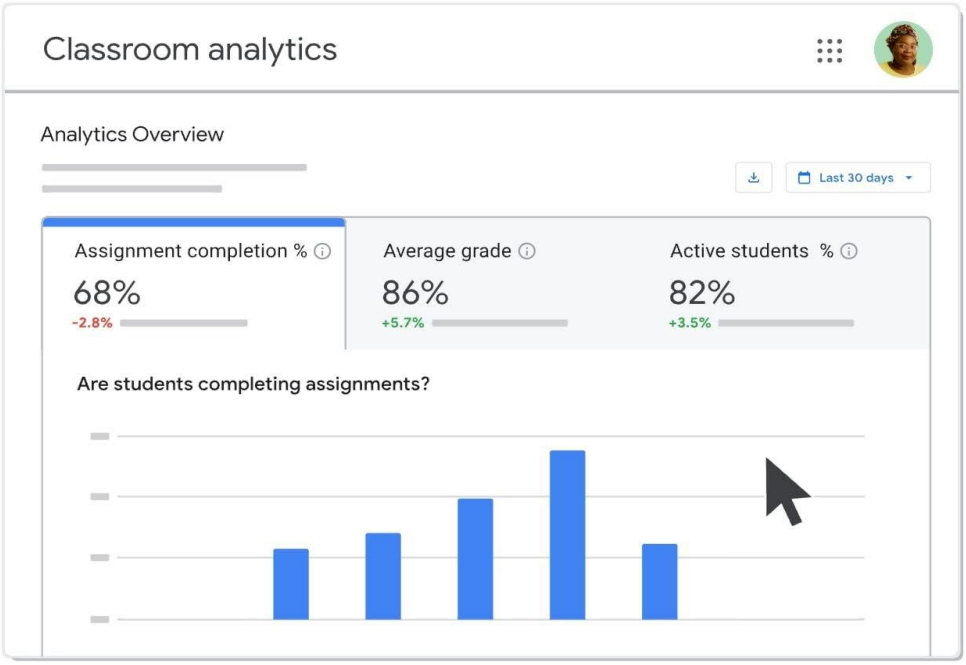
Some of its key features include progress-over-time graphs, real-time data dashboards, automatic target grade generation, and automatic import of assessment data from exams and internal tests. These tools help teachers quickly identify trends, gaps between expected and actual performance, and sudden drops in achievement, allowing earlier intervention.

A major benefit of TES Engage is its efficiency. Because assessment data can be imported automatically, teachers spend less time on manual data entry and administrative tasks. This allows them to focus more on lesson planning, marking, and supporting students.

However, the system can be complex, especially for new or less experienced users. Staff may require training to use it effectively. In addition, access is usually limited to teachers, meaning students and parents may not be able to independently view real-time progress data.

Overall, TES Engage is a powerful data-driven tool for monitoring and supporting student progress, although its complexity and restricted access may limit usability for some users (www.tes.com, n.d., 2026).

Google Classroom (Learning Management System)



(Google for Education, 2026)

Google Classroom is a web-based platform that can be accessed on both desktop and mobile devices. It is free for schools that use Google Workspace for Education. Although it is mainly designed as a learning management system, it also offers some basic features that help track student progress.

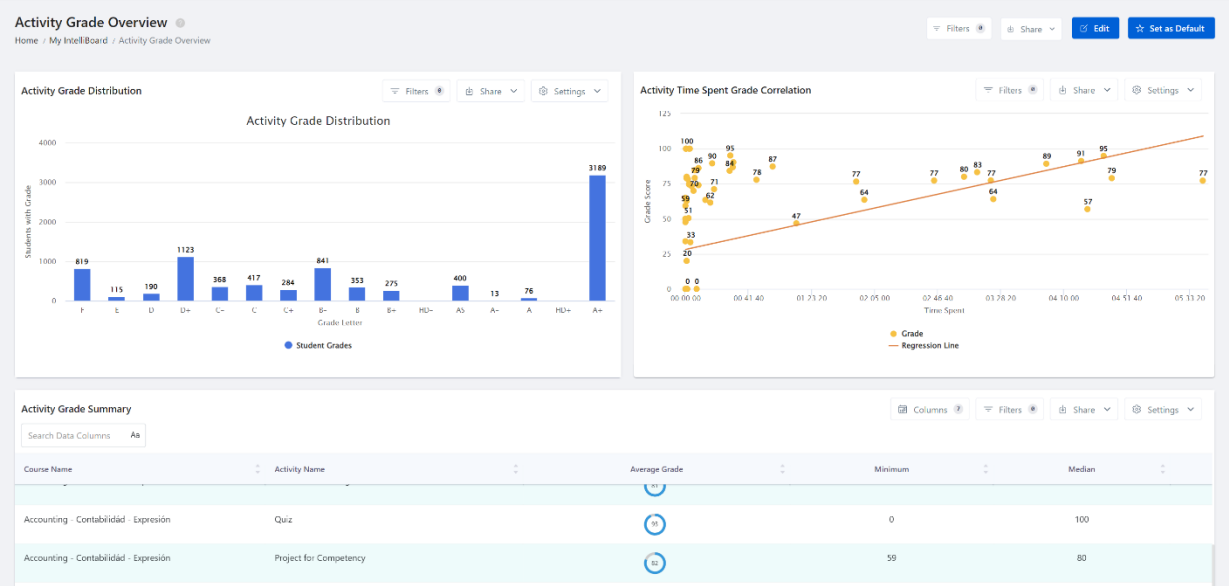
Teachers can create and distribute assignments, collect work online, provide grades and feedback, and communicate with students through announcements and comments. The platform includes assignment tracking, grade summaries, and real-time feedback, as well as integration with tools such as Google Docs, Sheets, and Drive.

One of the main strengths of Google Classroom is how simple and accessible it is. Students can easily check their assignments, deadlines, and grades at any time, which encourages independence and responsibility. Since it works on both computers and mobile devices, it also supports communication outside school hours.

However, Google Classroom has limited analytical features. It does not offer predictive grade calculations, detailed performance dashboards, cohort comparisons, or automatic identification of at-risk students. Progress tracking is mainly based on assignment grades and completion, which may not give a complete picture of long-term academic performance.

Overall, Google Classroom is effective for managing coursework and monitoring basic progress, but it lacks the deeper analytical tools needed for advanced decision-based tracking. (Google for Education, 2026).

Moodle (Learning Management System)



(Moodle, 2017)

Moodle is an open-source learning management system that supports online learning, course organisation, and student progress monitoring. It is web-based and can be accessed from different devices, making it suitable for both classroom and remote learning.

Teachers can upload learning materials, create assignments and quizzes, and manage grades using the built-in gradebook. Moodle also includes activity completion tracking, which allows teachers to see whether students have accessed resources and completed required tasks. This can help identify students who may be disengaged or falling behind.

One of Moodle's biggest strengths is its flexibility. Because it is open-source, schools can customise the system and add plugins to expand its features, including additional analytics tools. This makes it adaptable to different school requirements.

However, Moodle can be more technical to set up and manage compared to simpler platforms. Its wide range of features may require training and technical support to ensure that all tools, especially progress tracking features, are used effectively.

Overall, Moodle provides a flexible and customisable environment that supports both learning management and student progress monitoring, though it may require more technical expertise to implement successfully (Moodle, 2017).

FEATURES OF THE PROPOSED SOLUTION

- Student Management: Sort and search student records.
- Performance Analysis: Module grade distributions predicted grades and risk flags.
- Intervention Suggestions: Automatic alerts for at-risk students.

- Report Generation: PDFs with charts for teachers and parents.
- Role-Based Access: Teachers have full access; parents have limited access.
- Dashboard: Visual overview of class and individual performance.

LIMITATIONS

One limitation of the system is in its predictive accuracy. Currently, it uses simple thresholds to access student performance, which means it may not capture every factor that affects a learner's results. While this approach is not as precise as advances statistical machine learning models, it was chosen deliberately to keep the system simple, easy to use, and fast for teachers to make decisions.

Scalability is another consideration. The system relies on SQLite as its database, which works well for small to medium-sized colleges or training centres. However, for lager institutions or multi-campus settings, a more powerful database might be needed to handle bigger volumes of data efficiently and maintain smooth performance.

Parent access is also limited in the current design. Parents can view only one student at a time, which helps protect sensitive information and maintain privacy. In the future, this could be expanded if additional security measures are put in place.

Finally, report generation depends on the local computer’s resources, since the system operates offline. This ensures that the system is accessible without an internet connection and keeps it lightweight, but it may make it more difficult to share reports remotely or work collaboratively with other educators.

HARDWARE AND SOFTWARE REQUIREMENTS

HARDWARE:

A standard desktop computer or laptop capable of running a graphical desktop application.

The computer must have at least 4GB of RAM to ensure smooth operation of the user interface, database processing, visual charts, and PDF report generation.

A monitor with sufficient resolution is required so that dashboards, tables, graphs, and pie charts can be displayed clearly and interpreted accurately.

No specialist hardware is required, ensuring the system can be used easily in a typical school or home environment without additional cost.

SOFTWARE:

- Python 3.14, Tkinter, Matplotlib, ReportLab
- SQLite3 database
- A standard desktop computer or laptop capable of running a graphical desktop application.
- The computer must have at least 4GB of RAM to ensure smooth operation of the user interface, database processing, visual charts, and PDF report generation.
- A monitor with sufficient resolution is required so that dashboards, tables, graphs, and pie charts can be displayed clearly and interpreted accurately.
- No specialist hardware is required, ensuring the system can be used easily in a typical school or home environment without additional cost.

SUCCESS CRITERIA

The system will meet several key requirements to ensure it is both effective and practical for educators and parents.

ID	Success Criterion	How it will be Measured/Tested	Justification
SC1	The system must correctly authenticate users and assign the correct role (teacher/parent).	Attempt valid and invalid logins; verify correct access is granted or denied in 100% of cases.	Ensures secure access to sensitive student data and supports role-based functionality.
SC2	The system must enforce role-based access control.	Log in as teacher and parent and verify access restrictions (e.g., parents cannot access full dataset).	Protects student privacy and ensures the system meets stakeholder requirements.
SC3	The system must successfully import CSV data and store it without errors.	Import a CSV file and verify that all records are correctly stored and persist after restart.	Reduces manual data entry and demonstrates efficient computational data handling.

SC4	The system must allow accurate searching by student name or ID.	Perform searches using valid and invalid inputs; confirm correct results and validation messages.	Improves efficiency compared to manual searching and enhances usability.
SC5	The system must correctly identify at-risk students.	Test students with averages above and below 50; verify correct classification in all cases.	Supports early intervention, which is a key requirement identified in the problem.
SC6	The system must calculate predicted grades accurately and consistently.	Input identical data multiple times and confirm the same predicted grade is produced.	Ensures reliable, standardised decision-making and reduces human error.
SC7	The system must generate appropriate intervention suggestions.	Compare outputs against predefined rules for different performance levels.	Provides actionable insights, supporting teachers in decision-making.
SC8	The system must generate accurate visualisations (pie charts).	Generate charts and compare values against actual dataset distributions.	Enhances understanding of data through visual representation.
SC9	The system must generate a complete PDF report.	Export a report and verify all required fields and charts are included and correctly formatted.	Automates reporting, reducing workload and ensuring consistency.
SC10	The system must correctly sort student data.	Apply ascending and descending sorting and verify correct order of records.	Demonstrates use of algorithms and improves data analysis.
SC11	The system must handle errors without crashing.	Test invalid inputs, missing files, and incorrect CSV formats; confirm appropriate error messages appear.	Ensures system reliability and suitability for real-world use.

SC12	The system must provide a user-friendly interface.	Observe users completing tasks (search, report generation) without assistance.	Ensures accessibility for both teachers and parents.
SC13	The system must perform efficiently.	Measure response time for key actions (search, sort, report generation) — should be under 2–3 seconds.	Demonstrates the effectiveness of computational methods over manual processes.

DESIGN CHAPTER

SYSTEM ARCHITECTURE

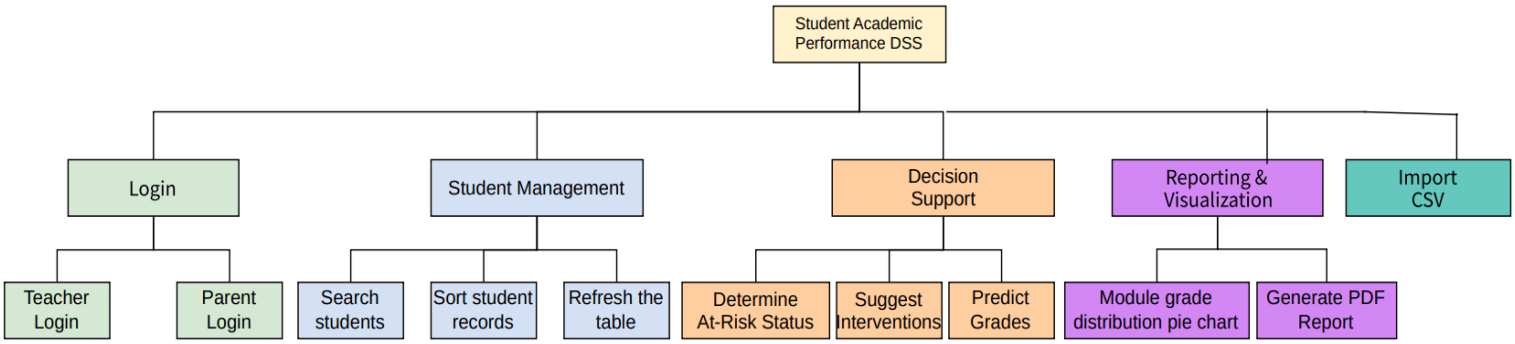
The Student Academic Performance Decision Support System will be designed as a modular desktop application and will be developed using the Python programming language. The system will follow a layered architecture to ensure maintainability, clarity, and separation of concerns.

The architecture consists of:

- Presentation layer (TKinter UI)
- Application Logic layer (Processing and Algorithms)
- Data storage layer (SQLite Database)

This layered architecture ensures that the user interface, processing logic and database operations are independent. This reduces complexity and allows each component to be tested individually.

SYSTEMS DIAGRAM



Using top-down modular design helps me plan and structure my solution before I start coding. By breaking the overall problem into smaller, manageable parts, I can focus on each component individually and understand its specific requirements. This approach allows me to apply computational thinking skills such as thinking ahead by identifying the required inputs and expected outputs for each module before implementation and thinking logically by determining where decisions are needed and evaluating how those decisions may affect other parts of the system. Overall, this method gives my solution a clear structure, making it easier to develop the program systematically and efficiently while reducing errors and improving maintainability.

DATABASE DESIGN

The backend of the Student Academic Performance DSS utilises a Relational Database Management System (RDBMS) via SQLite. The design follows a Single-Table Schema (Flat File approach), optimised for high-speed read/write operations during CSV imports from college Virtual Learning Environments (VLEs).

The system centralises all academic metrics into a single entity: students. This structure ensures that the DSS can perform real-time calculations (such as Predicted Grade, Suggested Interventions and Risk Analysis) without the overhead of complex table joins.

STUDENT TABLE

Attribute	Data Type	Constraint	Description
Student_id	TEXT	PRIMARY KEY	Unique identifier used to prevent duplicate records during VLE imports.
name	TEXT	NOT NULL	The full name of the student for reporting purposes.
module	TEXT	NOT NULL	The specific academic module or course code.
average	REAL		The numerical mean score; the primary driver for grade logic.
attendance	REAL	DEFAULT 100	Percentage of presence, used for grade prediction weighting.
Homework_complete	INTEGER	DEFAULT 1	A boolean flag (1/0) indicating task completion status.

Table Name: Student

ENTITIES

Since the current version of the DSS will focus on a specific snapshot of student performance, the system will contain only one strong entity: Student.

- Primary Key (PK): The student_id acts as the unique anchor.
- Other non-primary key attributes: name, module, average, Letter Grade, GPA, attendance and homework complete

MODULES OF THE STUDENT PERFORMANCE DECISION SUPPORT SYSTEM

CSV IMPORT

The purpose of the CSV file import module is to allow educators to efficiently upload and store multiple student records into the system at once instead of entering them manually. It reads structured data from a CSV file, validates and converts necessary fields (such as average and attendance), and saves the information into the SQLite database while preventing duplicate entries. After importing, it updates the system's internal data and refreshes the display so the newly added students can be viewed and managed within the Academic Performance Decision Support System.

PREDICTED GRADE CALCULATION

The Predicted Grade Calculation module estimates a student's likely final grade based on their current academic performance and encouragement factors. It starts with the student's average mark and then adjusts it according to attendance and homework completion. If attendance is below certain thresholds, marks are reduced, and an additional deduction is applied if homework is incomplete. The adjusted score is then constrained within valid limits (0–100) and converted into a letter grade (A–E). The purpose of this module is to provide an early performance forecast, helping teachers and parents identify potential risks and take corrective action before final assessments.

AT RISK LEVEL IDENTIFICATION

The At-Risk Level Identification module helps identify students who may be struggling academically. It looks at a student's current average score and flags them as "at risk" if it falls below a certain threshold, such as 50%. Students performing above this level are considered safe. The main goal of this module is to give educators and parents an early warning so they can provide support or take intervention measures before the student's performance declines further.

SUGGESTED INTERVENTIONS

The Suggested Interventions module provides recommendations to help improve a student's academic performance based on their current results. Depending on their average score, attendance, and homework completion, the system suggests appropriate actions, such as extra practice, tutoring, or immediate intervention for students who are falling behind. The purpose of this module is to guide educators and parents in offering timely support to help students succeed and prevent further academic difficulties.

SORTING STUDENT RECORDS

The Sorting Student Records module allows users to organise student data in a meaningful order based on different criteria such as student ID, name, module, average score, letter grade, GPA, attendance, or homework completion. This makes it easier to analyse performance, compare students, and quickly locate specific records. The module improves data accessibility and helps educators manage and interpret student information more efficiently.

SEARCH STUDENTS

The Search Students module enables users to quickly find specific students within the system by entering a student ID or name. It filters the records and displays only the matching results, making it easier to access individual student information without manually browsing through the entire list. This module helps educators and parents save time and efficiently monitor or review a student's performance.

EXPORT STUDENT REPORT

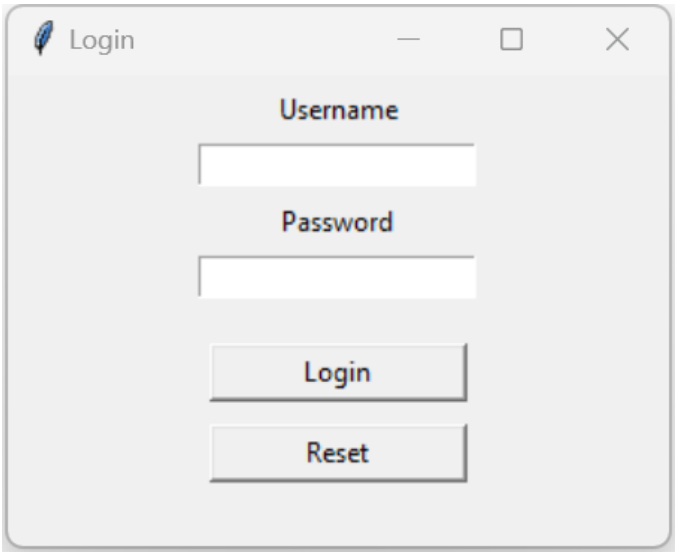
The Export Student Report module allows users to generate and save a detailed report of a student's academic performance. The report includes information such as average scores, letter grades, GPA, attendance, homework completion, at-risk status, suggested interventions, and predicted grades. It can also include visual representations like grade distribution charts. This module helps educators and parents easily review, share, and keep a record of student performance for evaluation and decision-making purposes.

LOGIN

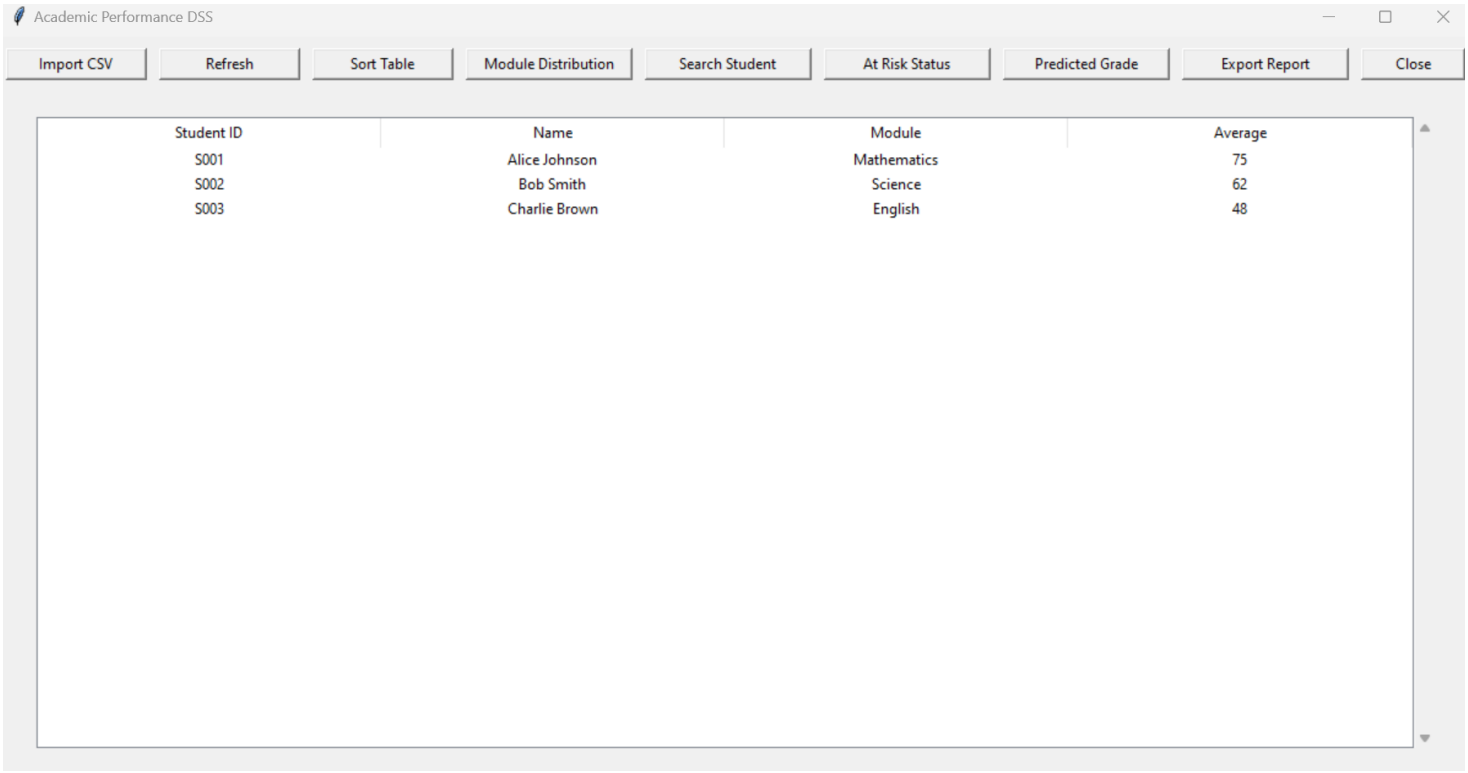
The Login module controls access to the system by verifying the identity of users. Each user, such as a teacher or parent, must enter a valid username and password to gain access. Based on their role, the system grants appropriate permissions, ensuring that sensitive student information is secure and only accessible to authorised users. This module helps maintain data privacy and provides a personalised experience for different types of users.

USER INTERFACE DESIGN

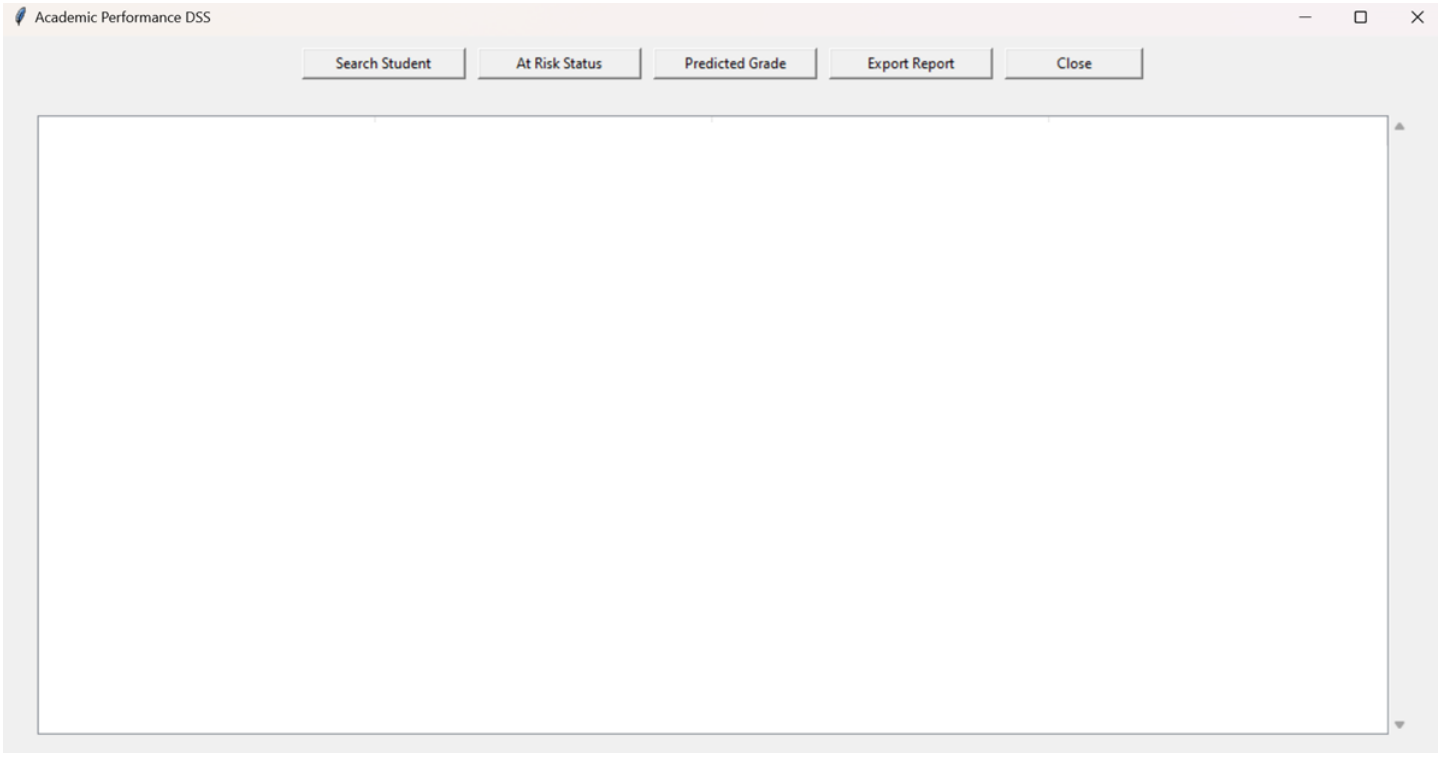
LOGIN SCREEN



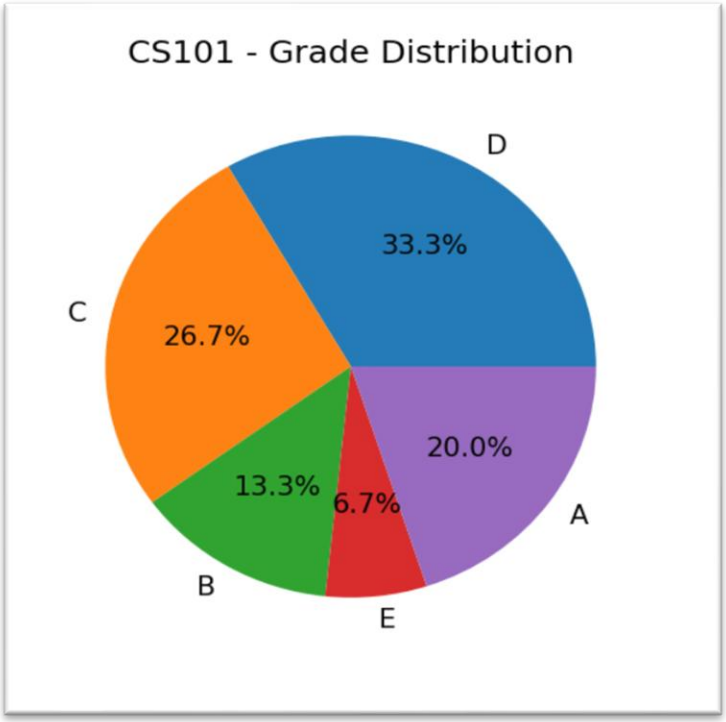
STUDENT ACADEMIC PERFORMANCE DECISION SUPPORT SYSTEM DASHBOARD – TEACHER VIEW



STUDENT ACADEMIC PERFORMANCE DECISION SUPPORT SYSTEM DASHBOARD – PARENT VIEW



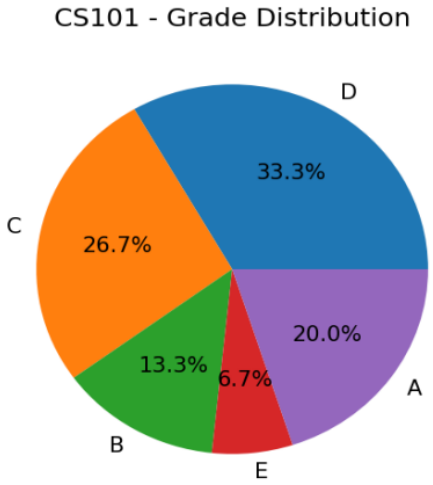
MODULE DISTRIBUTION GRAPH



STUDENT PROGRESS REPORT

Student Report - Isabella Jackson

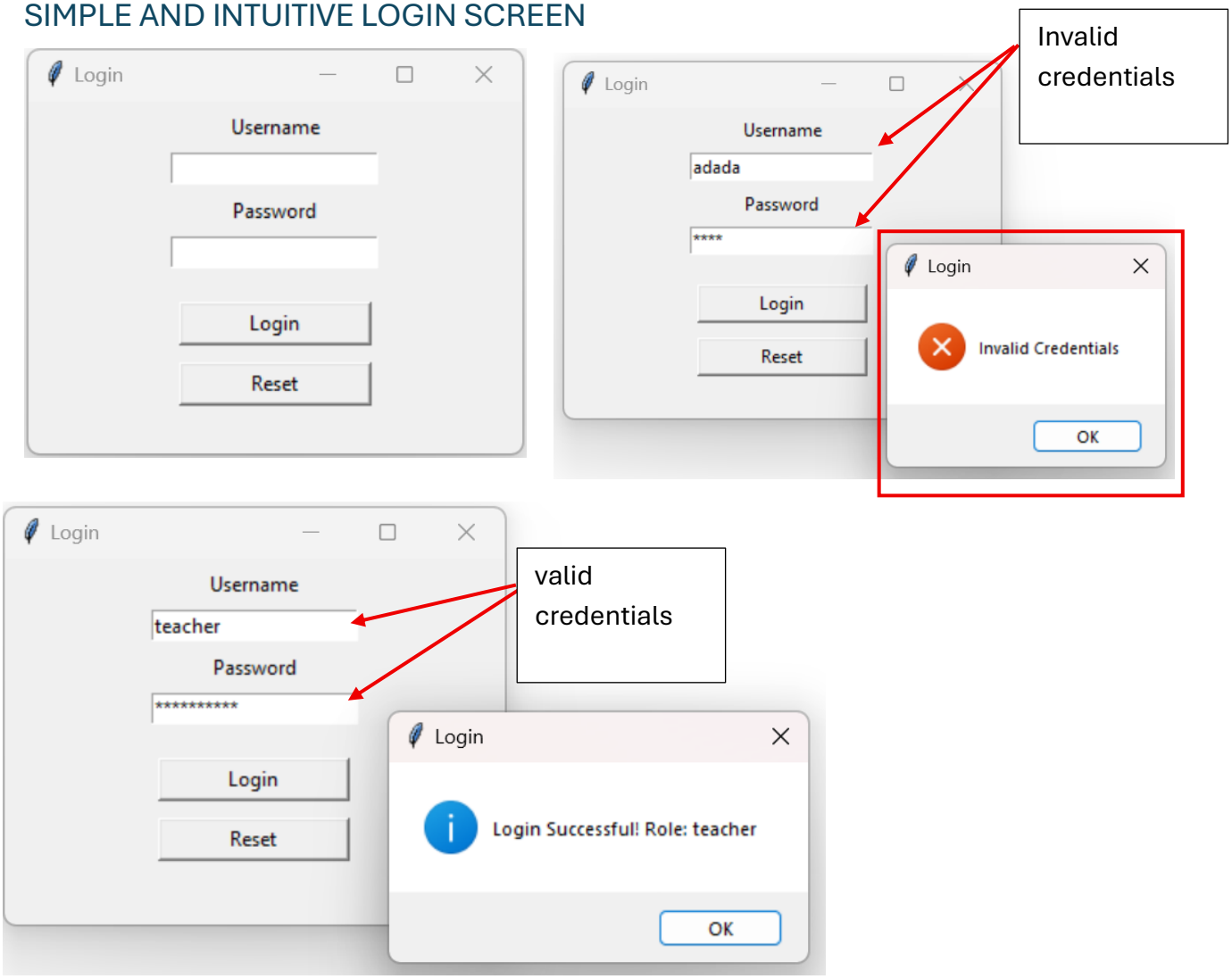
Student ID: S1010
Name: Isabella Jackson
Module: CS101
Average: 62.54
Letter Grade: B
GPA: 3
Attendance: 73.38
Homework Complete: Yes
At Risk: No
Suggested Intervention: Encourage practice
Predicted Grade: C



USABILITY FEATURES

The Student Academic Performance Decision Support System (DSS) is designed with usability in mind, ensuring that both educators and parents can navigate the system easily, interpret academic data accurately, and perform tasks without confusion or errors. The system incorporates features that enhance learnability, accessibility, feedback, and error prevention. The following highlights the key usability features in the Login Screen and Main Dashboard.

SIMPLE AND INTUITIVE LOGIN SCREEN



The login screen is designed to be simple and intuitive, displaying only the essential elements: Username, Password, and the two action buttons—Login and Reset. Each input field has a clear label positioned above it, making the interface easy to follow. The password field includes masking, ensuring input privacy. Validation is built into the login process: if incorrect credentials are entered, the system presents a clear “Invalid Credentials” error dialog, guiding the user to correct the mistake. When valid details are provided, a “Login Successful” confirmation dialog appears, giving users immediate feedback. The inclusion of the Reset button further enhances usability by allowing users to quickly clear both fields and re-enter

their information. Together, these features prevent user errors, maintain security, and provide helpful feedback throughout the login process.

CLEAN AND CONSISTENT DASHBOARD LAYOUT

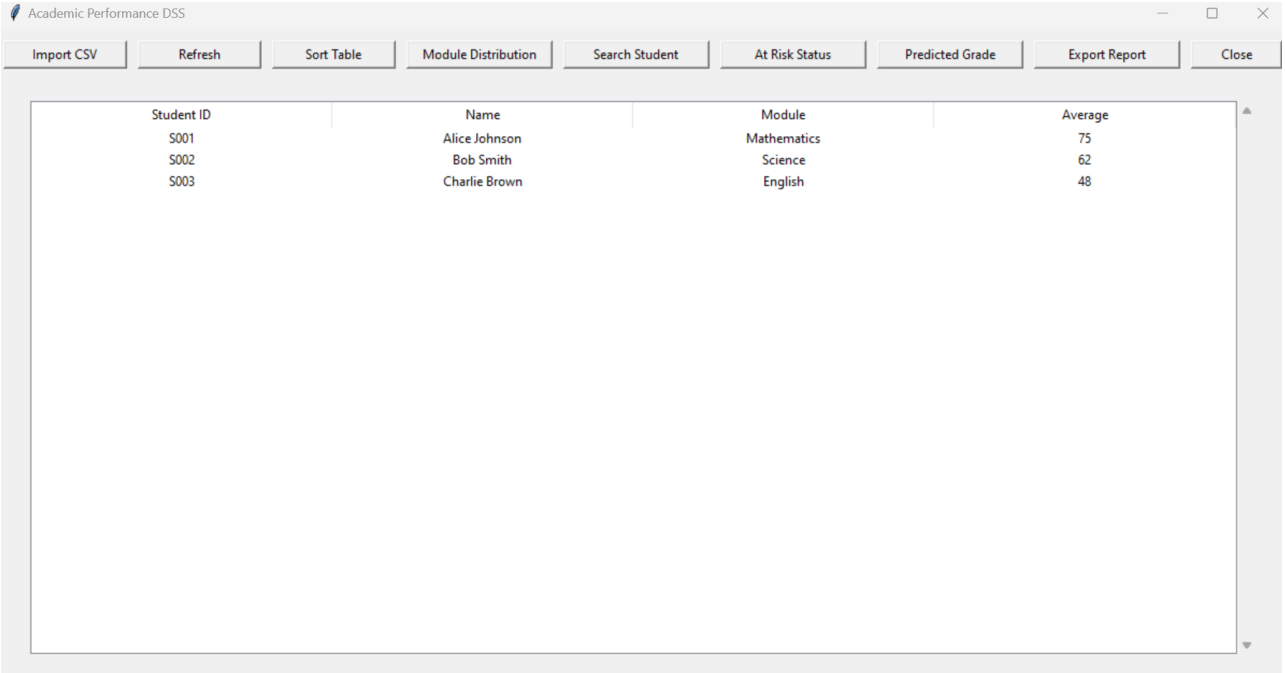


Figure: Student Performance Decision Support System main application dashboard

The main dashboard uses a clear, structured layout. A horizontal button bar at the top provides access to all key functions, while a central table displays student data. Buttons are arranged logically to follow a natural workflow, and the table uses uniform column widths with centre-aligned text. Clear column headings help users quickly identify each data field. This structure reduces visual clutter and makes the system easier for both new and experienced users to navigate.

ROLE-BASED INTERFACE SIMPLIFICATION

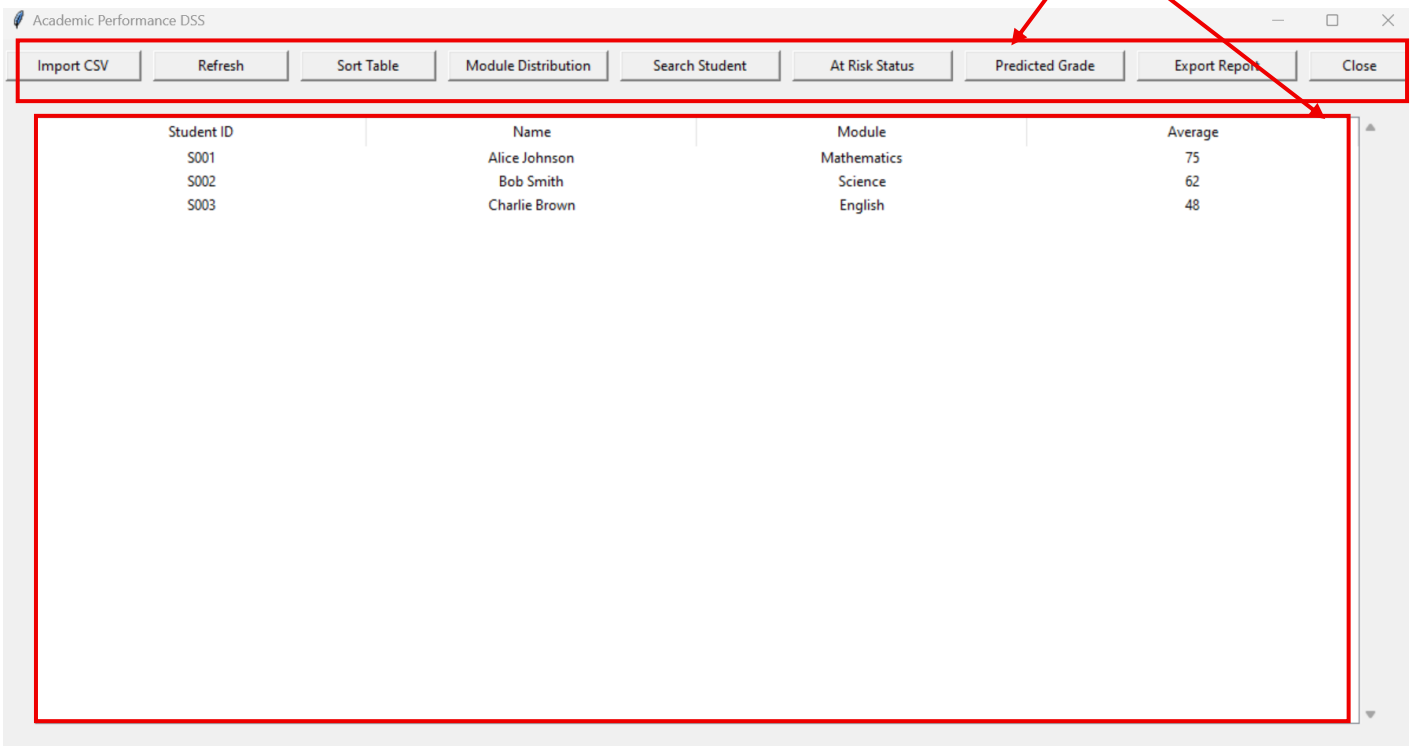


Figure: Student Performance Decision Support System- Teacher main application dashboard

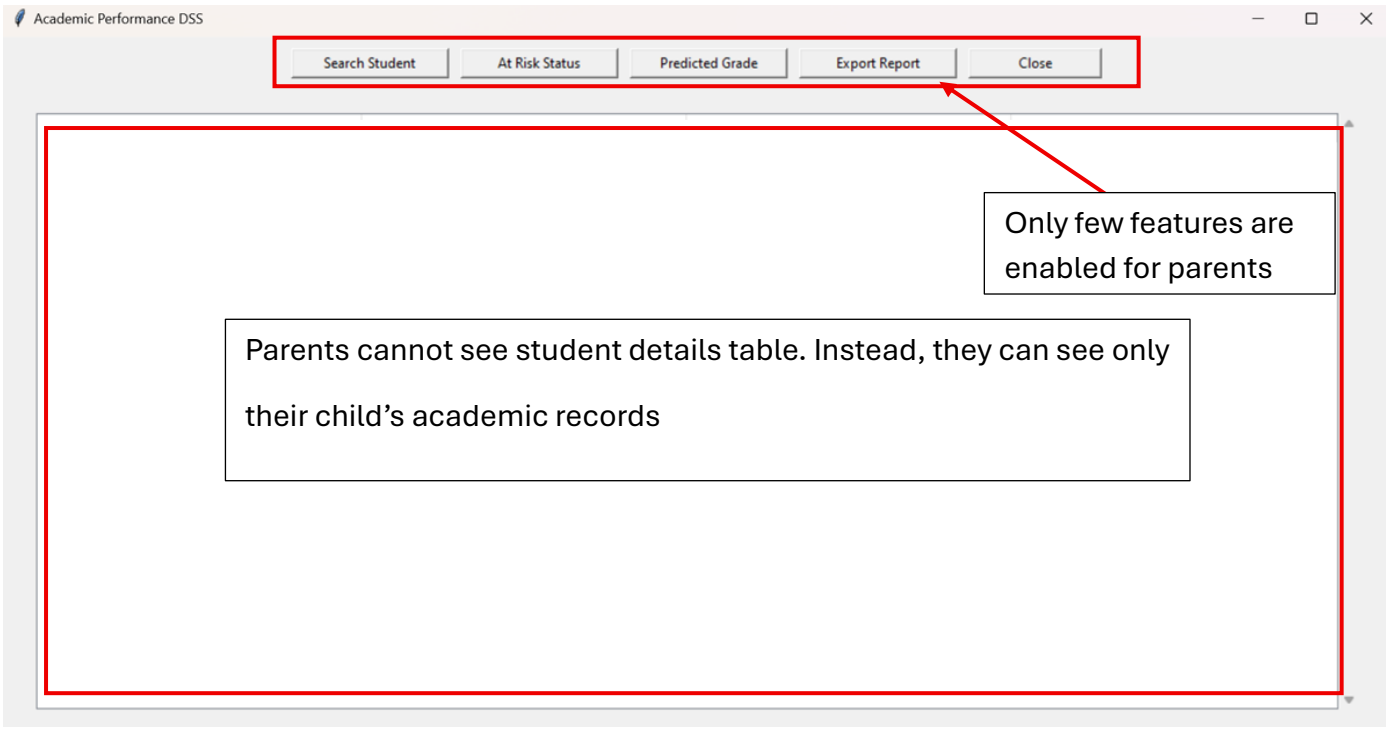


Figure: Student Performance Decision Support System- Parent main application dashboard

The interface adapts based on the user's role. Teachers see all functions, including CSV import, table sorting, and module distribution, while parents see only non-editing features like searching students and viewing interventions. This prevents confusion, avoids accidental misuse, and ensures users only see relevant features.

VALIDATION ON DASHBOARD FUNCTIONS

All dashboard actions include validations to maintain proper system operation:

- The Search Student feature provides a message if no matching student is found.
- Table Actions require a student to be selected before displaying information such as At-Risk Status, Predicted Grade, or Suggested Interventions.
- The CSV Import function only accepts .csv files, fills in missing values with default data, validates all data types, and ignores duplicate student IDs.
- The Report Export feature ensures a student is selected before generating a report and safely creates charts and PDF files.

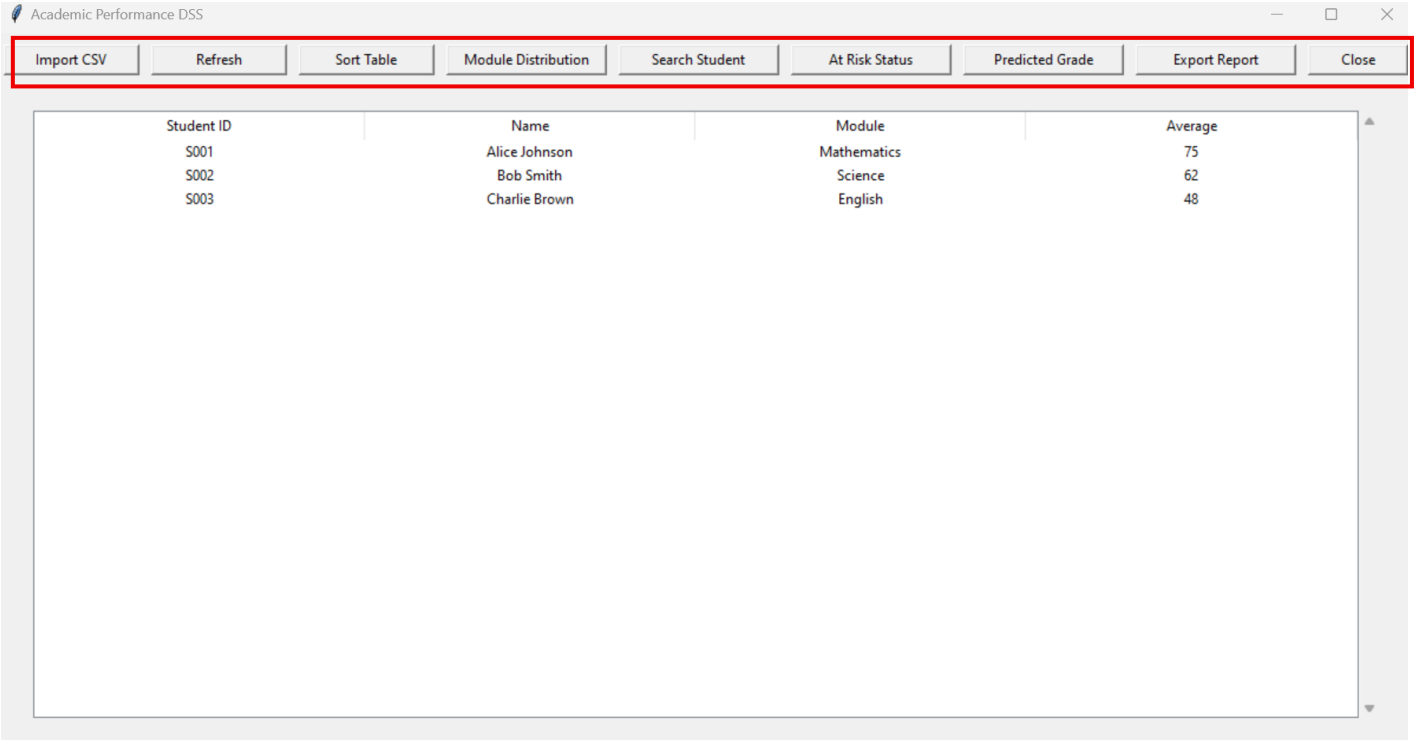
These validations prevent runtime errors, protect data integrity, and guide users in correct system usage.

CLEAR FEEDBACK AND SYSTEM STATUS INDICATORS

The system provides real-time feedback through dialogs: success messages (e.g., "Report Exported Successfully"), errors (e.g., "Invalid credentials"), warnings (e.g., "Select student first"), and information messages (e.g., "No student found"). This transparency keeps users informed about system actions and aligns with good usability practices.

CONSISTENCY IN CONTROLS AND TERMINOLOGY

Button labels consistently use verbs (e.g., Search Student, Export Report), and table formats, icons, and styling remain uniform throughout the system. Consistency reduces learning time, avoids confusion, and makes the interface predictable.



STRUCTURED AND READABLE DATA PRESENTATION

The Treeview table presents student data clearly, with consistent numeric formatting, textual values for status indicators (e.g., Yes/No for homework), and automatic updates after sorting or refreshing. This makes it easy for users to detect low grades, poor attendance, or missing homework and supports faster decision-making.

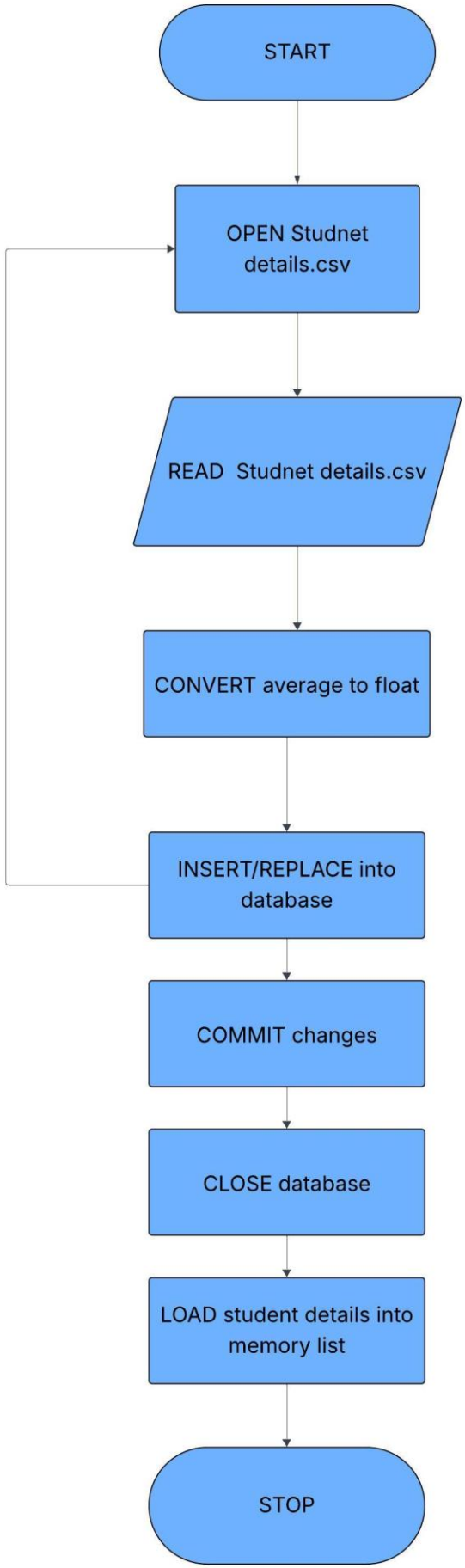
ALGORITHM DESIGN

1. **CSV Import module**

Pseudocode

```
FUNCTION import_csv(file_path)
    OPEN file
    FOR each row in csv:
        READ student_id, name, module, average, attendance, homework complete
        CONVERT average to float
        INSERT OR REPLACE into database
    COMMIT changes
    CLOSE database
    LOAD students into memory list
END FUNCTION
```

Flowchart

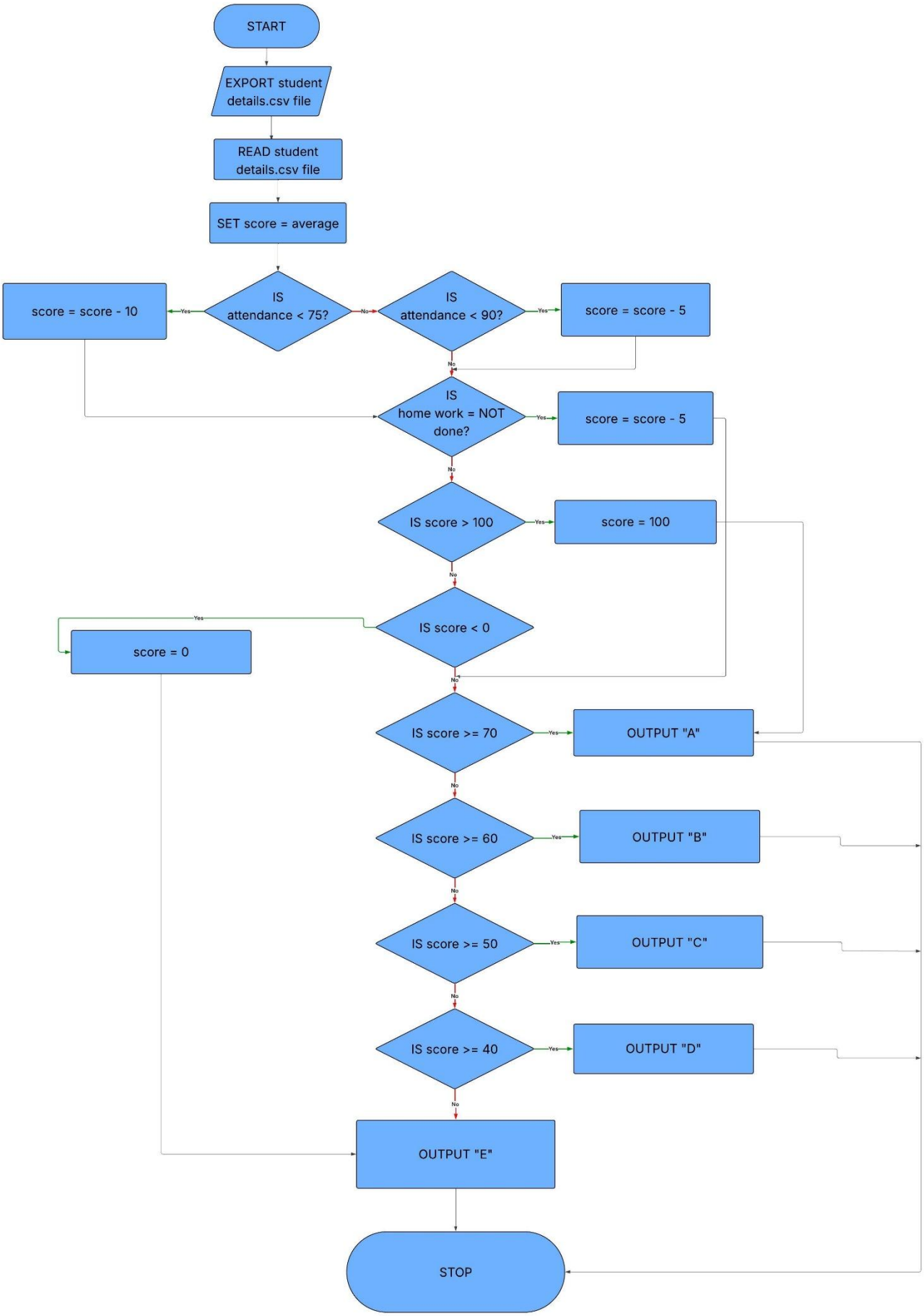

START
OPEN Studnet details.csv
READ Studnet details.csv
CONVERT average to float
INSERT/REPLACE into database
COMMIT changes
CLOSE database
LOAD student details into memory list
STOP

2. Predicted grade calculation

Pseudocode

```
FUNCTION predicted_grade:
  SET score TO average

  IF attendance < 75 THEN
    SUBTRACT 10 FROM score
  ELSE IF attendance < 90 THEN
    SUBTRACT 5 FROM score
  END IF

  IF homework is NOT complete THEN
    SUBTRACT 5 FROM score
  END IF

  IF score > 100 THEN
    SET score TO 100
  ELSE IF score < 0 THEN
    SET score TO 0
  END IF

  IF score >= 70 THEN
    RETURN "A"
  ELSE IF score >= 60 THEN
    RETURN "B"
  ELSE IF score >= 50 THEN
    RETURN "C"
  ELSE IF score >= 40 THEN
    RETURN "D"
  ELSE
    RETURN "E"
  END IF
END FUNCTION
```

Flowchart

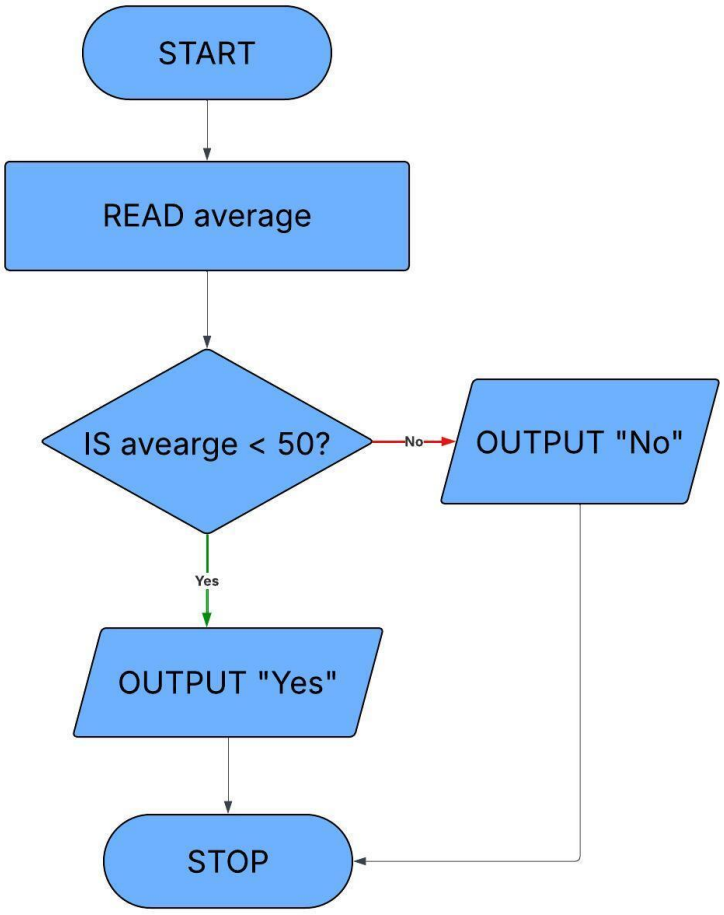

START
EXPORT student details.csv file
READ student details.csv file
SET score = average
IS attendance < 75?
Yes
score = score - 10
No
IS attendance < 90?
Yes
score = score - 5
No
IS home work = NOT done?
Yes
score = score - 5
No
IS score > 100
Yes
score = 100
No
IS score < 0
Yes
score = 0
No
IS score >= 70
Yes
OUTPUT "A"
No
IS score >= 60
Yes
OUTPUT "B"
No
IS score >= 50
Yes
OUTPUT "C"
No
IS score >= 40
Yes
OUTPUT "D"
No
OUTPUT "E"
STOP

3. At risk level identification

Pseudocode

```
FUNCTION at_risk(average)
    IF average < 50:
        RETURN "Yes"
    ELSE:
        RETURN "No"
END FUNCTION
```

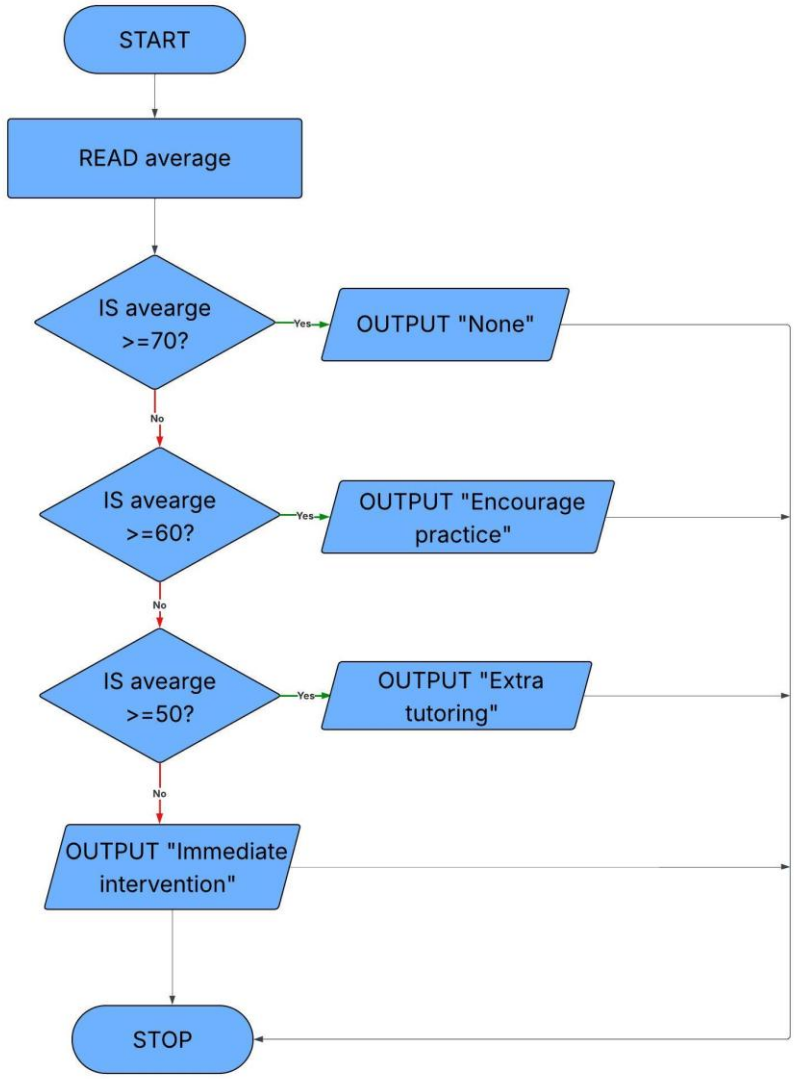
Flowchart



4. Suggested interventions

Pseudocode

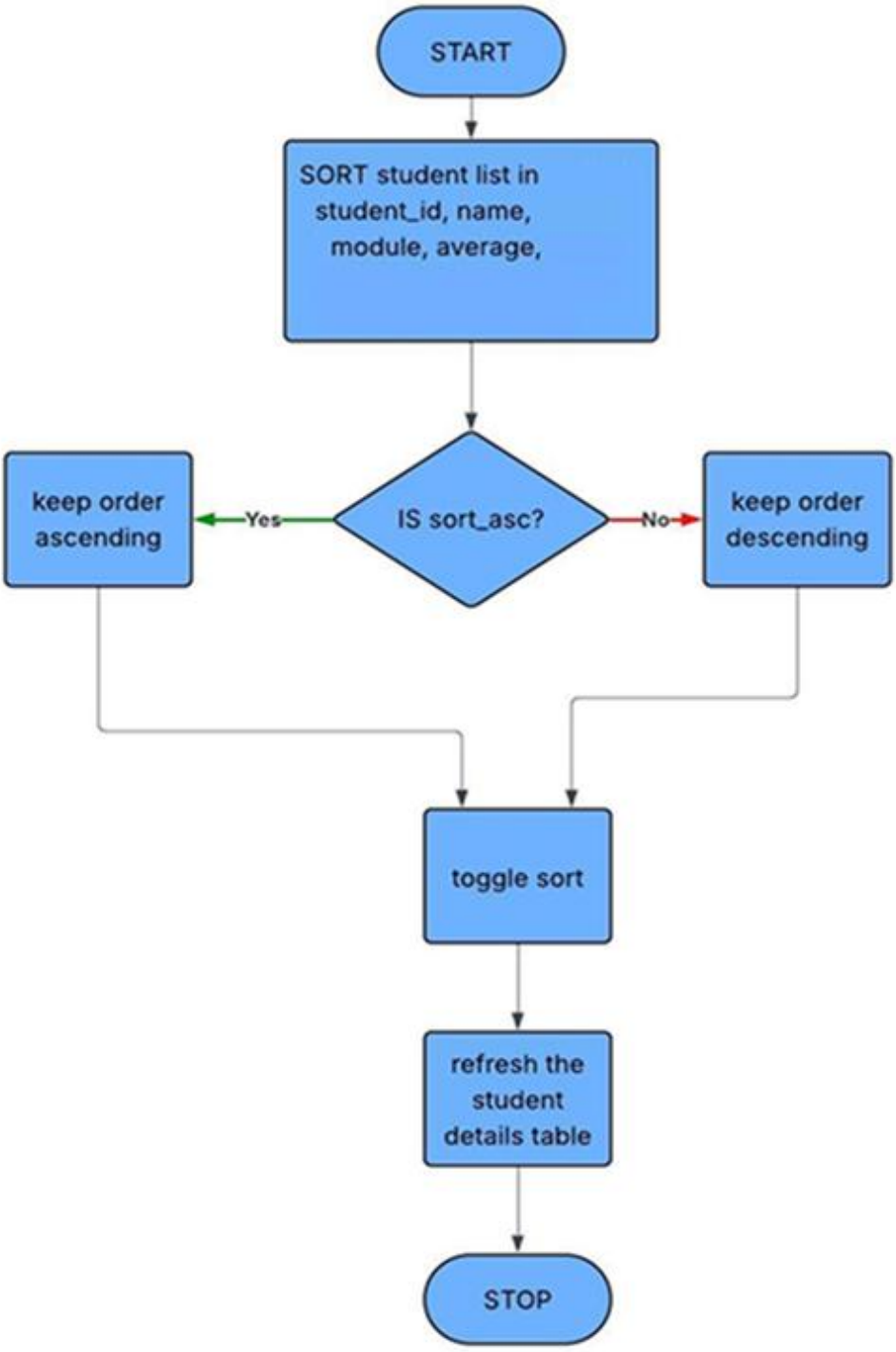
```
FUNCTION suggested_intervention(average)

  IF average ≥ 70:
    RETURN "None"

  ELSE IF average ≥ 60:
    RETURN "Encourage practice"

  ELSE IF average ≥ 50:
    RETURN "Extra tutoring"

  ELSE:
    RETURN "Immediate intervention"

END FUNCTION
```



5. Sorting student records
   Pseudocode

```
PROCEDURE sort_all_columns

  SORT students BY:
    lowercase(student_id),
    lowercase(name),
    lowercase(module),
    (average)

  IF sort_asc = FALSE THEN
    REVERSE students list
  ENDIF

  TOGGLE sort_asc

  CALL refresh_table(students)

END PROCEDURE
```

Flowchart

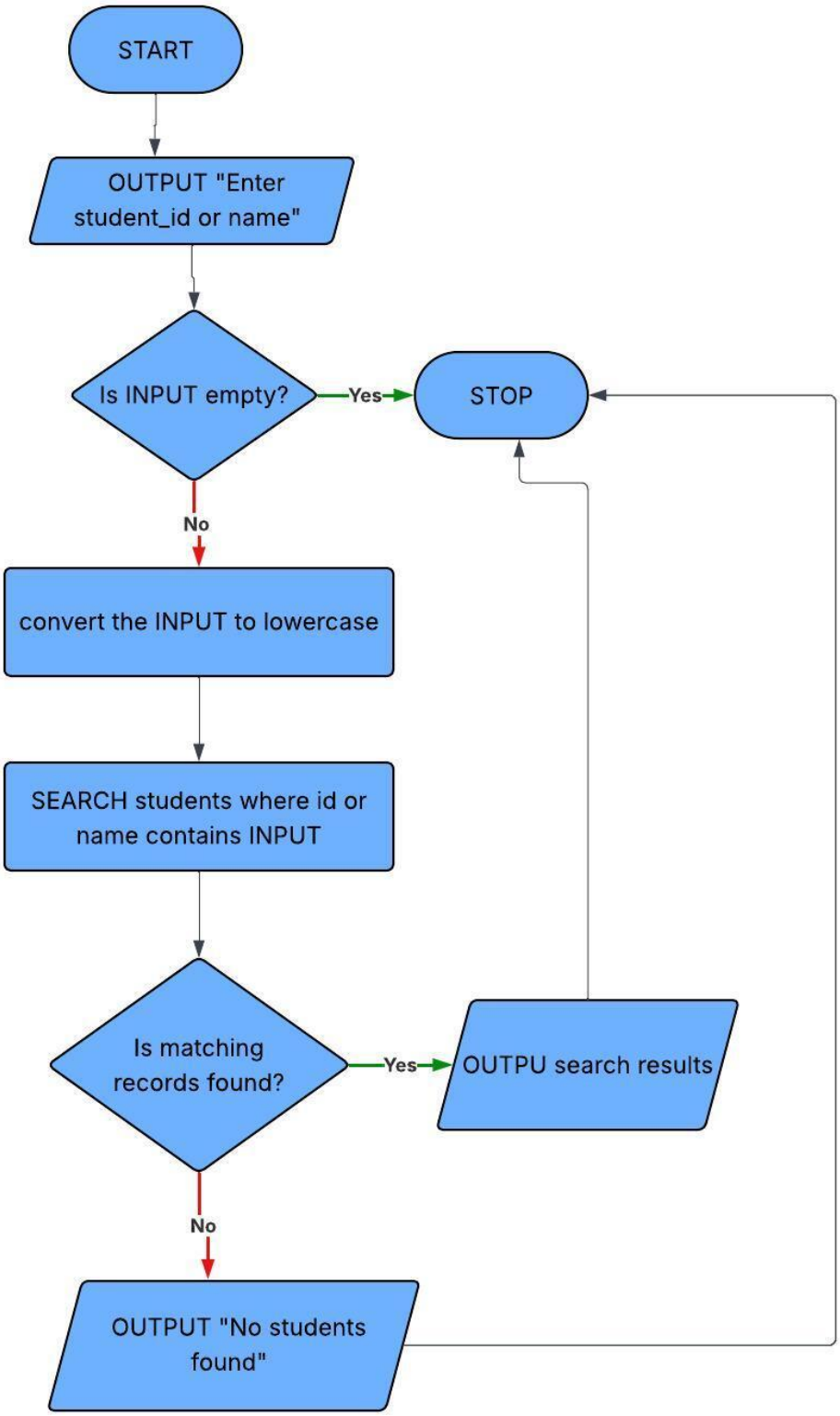

START
SORT student list in student_id, name, module, average,
IS sort_asc?
Yes
No
keep order ascending
keep order descending
toggle sort
refresh the student details table
STOP

6. Search students

Pseudocode

```
FUNCTION search_student
  ASK user for input "Enter Student ID or Name"
  IF input is empty
    RETURN
  CONVERT input to lowercase
  FILTER students WHERE student ID or name contains input
  IF no students found
    SHOW message "No student found"
    RETURN
  DISPLAY filtered students in table
END FUNCTION
```

Flowchart

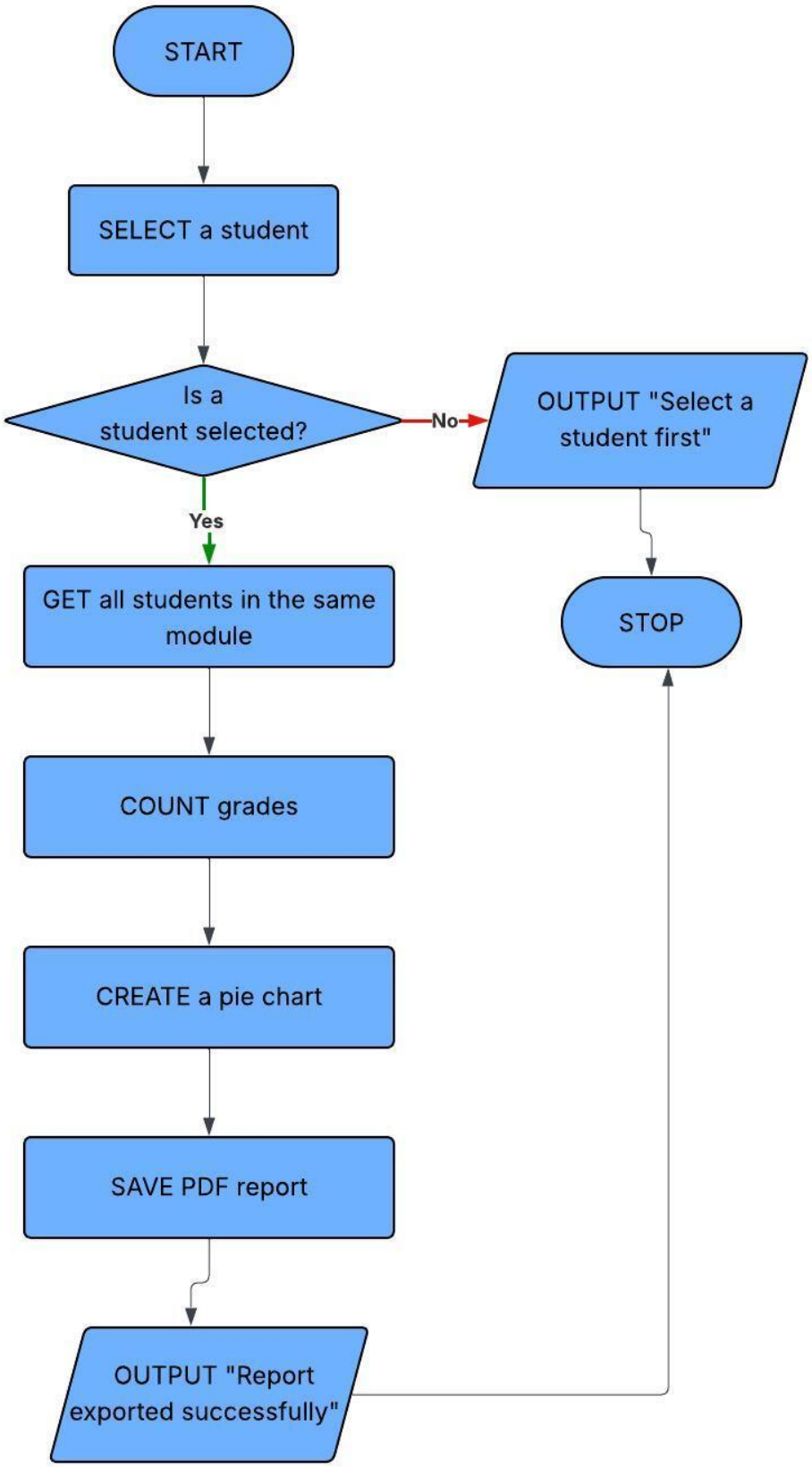

START
OUTPUT "Enter student_id or name"
Is INPUT empty?
Yes
STOP
No
convert the INPUT to lowercase
SEARCH students where id or name contains INPUT
Is matching records found?
Yes
OUTPU search results
No
OUTPUT "No students found"

7. Export student report

Pseudocode

```
FUNCTION export_student_report
    SELECT student s
    IF no student selected
        SHOW warning "Select student first"
        RETURN

    GET all students in s's module → module_students
    COUNT letter grades → grades
    CREATE pie chart → image

    ASK user where to save PDF
    CREATE PDF
    WRITE student details
    ADD pie chart
    SAVE PDF

    SHOW "Report Exported Successfully"
END FUNCTION
```

Flowchart

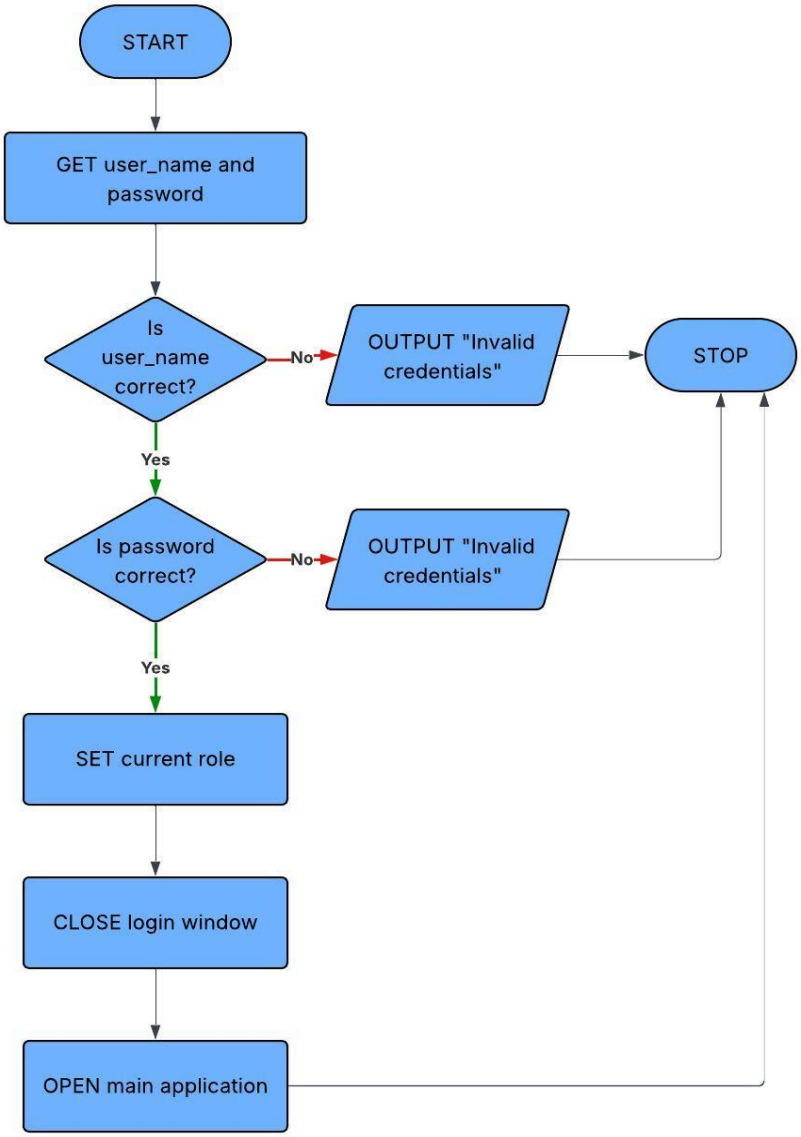

START
SELECT a student
Is a student selected?
No
OUTPUT "Select a student first"
Yes
STOP
GET all students in the same module
COUNT grades
CREATE a pie chart
SAVE PDF report
OUTPUT "Report exported successfully"

8. Login

Pseudocode

```
FUNCTION login
    GET username from username_entry
    GET password from password_entry

    IF username exists in USERS AND password matches
        SET current_role to user's role
        CLOSE login window
        OPEN main application window
    ELSE
        SHOW error "Invalid credentials"
END FUNCTION
```

Flowchart



KEY VARIABLES AND DATA STRUCTURES

Method / Structure / Variable	Name	Data Type	Purpose	Justification & Validation
Class	Student	Class	Encapsulates all student academic data and decision logic	Supports Object-Oriented design (modularity, maintainability). Input validation required for marks (0–100), attendance (0–100), and homework (Boolean).
Attribute	student_id	String	Unique identifier for each student.	Defined as PRIMARY KEY in database to prevent duplicates. Must be non-empty and unique.
Attribute	name	String	Stores student name.	Must not be empty. Prevents incomplete records.
Attribute	module	String	Stores subject/module name.	Should be validated to avoid inconsistent spellings.
Attribute	average	Float	Stores average mark.	Must be validated between 0–100 to ensure correct grade calculation.
Attribute	attendance	Float	Stores attendance percentage.	Must be validated between 0–100. Default = 100 to prevent null errors.
Attribute	homework_complete	Integer (0/1 Boolean)	Indicates homework completion status.	Restricted to 0 or 1. Prevents prediction miscalculations.

Method	at_risk()	Function	Identifies students below pass mark (<50).	Provides decision-support indicator based on validated average.
Method	suggested_intervention()	Function	Recommends support level based on performance.	Implements DSS logic using defined performance thresholds.
Method	predicted_grade()	Function	Predicts final grade using average, attendance, homework.	Combines validated inputs. Score is clamped (0–100) to prevent invalid
Data Structure	students	List	Stores student records in memory.	Allows sorting, searching, filtering. Integrity depends on validated
Data Structure	USERS	Dictionary	Stores login credentials and roles.	Enables role-based access control. Username must exist and
Variable	current_role	String	Tracks logged-in user role.	Controls system permissions (teacher vs parent). Must match predefined
Database Table	students	SQLite Table	Persistent storage of student records.	Enforces structure and PRIMARY KEY constraint. Numeric fields
Method	import_csv()	Procedure	Imports student data from CSV file.	Converts data types (float/int). Should validate missing fields and numeric
Method	module_distribution()	Procedure	Generates and displays grade distribution for a selected module	Ensures valid input and handles missing data before visualizing results

Method	sort_students()	Function	Sorts a list of student objects in ascending or descending order using multiple attributes	Validates input and ensures consistent sorting using defined comparison keys
Method	search_students()	Function	Searches for students by name or ID and returns matching results	Ensures search term is valid and performs case-insensitive matching for accuracy
Method	search_ui()	Function	Handles user input, validation, and displays search results based on user role	Validates input, manages different user roles, and provides appropriate feedback for results or errors

TEST PLANS

TEST DATA FOR ITERATIVE TESTING

CSV Import

ID	Testing Type	Data	Expected
CSV1	Valid	S1001, John Smith, CS101, 47.62, 72.66, No	CSV exports correctly: all fields present and formatted properly
CSV2	Valid	S1004, Olivia Davis, CS101, 53.38, 71.56, Yes	CSV exports correctly: numeric and string fields preserved
CSV3	Valid	S1010, Isabella Jackson, CS101, 62.54, 73.38, Yes	CSV exports correctly: Boolean field (Homework) exported as Yes/No
CSV4	Boundary	S1021, Matthew Lee, CS101, 71.53, 84.46, Yes	CSV includes boundary averages: numeric precision preserved
CSV5	Boundary	S1017, Alexander Robinson, CS101, 60.09, 91.82, No	Boundary grade included letter grade exported correctly

CSV6	Boundary	S1009, James Thomas, CS101, 39.20, 90.72, Yes	Lower boundary average: data exported accurately
CSV7	Invalid	S1026, Elizabeth King, CS101, 48.84, ? , 53.74, Yes	Invalid attendance: CSV exports with flagged error
CSV8	Invalid	S1027, David Wright, CS101, 93.49, 73.28, 0	Homework field invalid format: CSV exports with flagged error
CSV9	Invalid	S1016, Amelia Martinez, CS101, 39.83, 96.17, 1	Average very low, tests system handling of extreme low values
CSV10	Invalid	S1020, Evelyn Lewis, CS101, 57.46, -10, 0	Negative attendance: CSV exports value but flagged as invalid

Justification

The CSV export test data ensure the system handles valid, boundary, and invalid inputs correctly while maintaining data accuracy. Valid cases confirm proper formatting, boundary cases check precision, and invalid cases ensure errors are safely handled without crashing, making the feature reliable for real-world use.

Predictive Grade Calculation

ID	Testing Type	Data	Expected
PG1	Valid	S1021, Matthew Lee, CS101, 72, 92, Yes	Predicted grade correct; boundary for A grade
PG2	Valid	S1017, Alexander Robinson, CS101, 60.09, 91.82, No	Predicted grade correct; boundary for B grade
PG3	Boundary	S1004, Olivia Davis, CS101, 53.38, 71.56, Yes	Predicted grade calculation correct; boundary for C grade
PG4	Boundary	S1003, Liam Brown, CS101, 42.67, 83.92, No	Predicted grade deduction applied for incomplete homework
PG5	Invalid	S1027, David Wright, CS101, 93.49, 73.28, 0	Invalid homework field: predicted grade capped, system handles gracefully

Justification

The predictive grade calculation test data were designed to check that the system applies grading rules correctly in different situations, including normal, edge, and invalid cases. Valid tests confirm that grades are assigned accurately at key boundaries like A, B, and C. Boundary tests ensure that deductions, such as for incomplete homework, are applied consistently. Invalid input tests show that unusual or incorrect values are handled safely, with grades kept within valid limits instead of causing errors. Overall, this shows that the predictive grade feature is reliable, robust, and useful for supporting academic decision

Attendance & Homework Complete

ID	Testing Type	Data	Expected
LA1	Valid	S1001, John Smith, CS101, 47.62, 72.66, No	Predicted grade deduction applied for low attendance; homework complete handled
LA2	Boundary	S1002, Emma Johnson, CS101, 47.44, 51.55, No	Very low attendance + missing homework; cumulative penalties tested
LA3	Boundary	S1006, Ava Moore, CS101, 48.32, 78.62, No	Attendance just below threshold; homework incomplete applied
LA4	Boundary	S1008, Sophia Anderson, CS101, 53.35, 62.06, No	Attendance moderate; homework penalty applied; upper boundary tested
LA5	Invalid	S1020, Evelyn Lewis, CS101, 57.46, -10, 0	Negative attendance; predicted grade handled gracefully

Justification

The test data were designed to ensure that the predictive grade algorithm correctly applies penalties for attendance and homework in a range of situations, including normal, edge, and invalid cases. Valid and boundary tests confirm that deductions are applied accurately when attendance drops below set thresholds or when homework is incomplete, even near the limits. The invalid test checks that unrealistic attendance values are handled safely, with grades kept within valid ranges instead of causing errors. This shows that the system models real-world academic penalties effectively while remaining stable and reliable.

At risk status

ID	Testing Type	Data	Expected
AR1	Valid	Average = 75	At risk status = No
AR2	Valid	Average = 45	At risk status = Yes
AR3	Boundary	Average = 50	At risk status = No

Justification

The test data verifies that the system correctly determines “At Risk” status using a threshold of 50. A student with an average above 50 (75 in AR1) is correctly classified as not at risk, while a student below 50 (45 in AR2) is correctly identified as at risk. The boundary case (50 in AR3) confirms that the rule uses a strict comparison (average < 50), as a score of exactly 50 is classified as not at risk. Together, these cases validate both typical and edge conditions**.**

Suggested intervention

ID	Testing Type	Data	Expected
SI1	Valid	Average = 48	Suggested Intervention= Immediate intervention
SI2	Valid	Average = 75	Suggested Intervention = None
SI3	Boundary	Average = 50	Suggested Intervention = Extra tutoring

Justification

The test data validates that the system assigns the correct Suggested Intervention based on average score ranges. In SI1, a student with an average of 48 falls below 50 and is given “Immediate intervention,” confirming that low-performing students receive urgent support. In SI2, a student with an average of 75 is assigned “None,” verifying that high-performing students do not require intervention. In SI3, the boundary case of 50 is assigned “Extra tutoring,” ensuring that the system correctly handles threshold values and applies a different level of support at the cutoff point. Together, these cases confirm correct behaviour for low, high, and boundary conditions.

Module Distribution – Pie Chart

ID	Testing Type	Data	Expected
MD1	Valid	M205	Display pie chart
MD2	Valid	m205	Display pie chart
MD3	Invalid	M999	Do not display pie chart
MD4	Invalid	blank	Cancel the pop up

Justification

The test data ensures the system correctly handles module input for displaying a pie chart. Valid inputs ("M205" and "m205") display the chart, confirming normal operation and case insensitivity. An invalid code ("M999") prevents the chart from displaying, while a blank input cancels the pop-up. These cases cover valid, invalid, and edge scenarios.

Report Export – PDF

ID	Testing Type	Data	Expected
RE1	Valid	S1001, John Smith, CS101, 47.62, 72.66, No	PDF report generated with all fields correctly
RE2	Valid	S1004, Olivia Davis, CS101, 53.38, 71.56, Yes	PDF report includes at-risk status and suggested intervention
RE3	Boundary	S1006, Ava Moore, CS101, 48.32, 78.62, No	PDF report shows predicted grade deduction for homework
RE4	Boundary	S1021, Matthew Lee, CS101, 71.53, 84.46, Yes	PDF report includes pie chart and boundary grade formatting
RE5	Invalid	S1020, Evelyn Lewis, CS101, 57.46, -10, 0	Very low/negative attendance; ensures PDF handles extreme predicted grade without crashing

Justification

These test data were created to ensure that the PDF report export function produces complete, accurate, and reliable reports across normal, boundary, and extreme data scenarios. Valid tests check that all student details, at-risk status, and suggested interventions are included correctly. Boundary tests make sure that grade thresholds, deductions, and visual elements like the module distribution pie chart are displayed

properly. Extreme or illogical values, such as negative attendance, are handled safely without causing errors, showing that the report generation works reliably in all situations.

Login & Role Handling

ID	Testing Type	Data	Expected
L1	Valid	Username: teacher, Password: teacher123	Role: Teacher; full dashboard visible
L2	Valid	Username: parent, Password: parent123	Role: Parent; limited dashboard, teacher-only buttons hidden
L3	Invalid	Username: teacher, Password: wrongpass	Login fails; error message displayed
L4	Invalid	Username: wronguser, Password: teacher123	Login fails; error message displayed
L5	Boundary	Username: (empty), Password: (empty)	Login fails; prompts for credentials
L6	Boundary	Username: parent, Password: TEACHER123	Login fails; password is case-sensitive
L7	Valid	Username: teacher, Password: teacher123	Role verification; teacher-only functions accessible
L8	Valid	Username: parent, Password: parent123	Role verification; teacher-only functions NOT visible
L9	Boundary	Username: teacher, Password: teacher123	Multiple consecutive logins; session resets correctly
L10	Boundary	Username: parent, Password: parent123	Logout and re-login; dashboard refresh works

Justification

These test data were created to make sure the login system works securely and enforces role-based access correctly. Valid tests check that users are given the right role and see the appropriate dashboard, with teacher-only features hidden from parents. Invalid and boundary tests confirm that wrong credentials, empty fields, or case-sensitive password mismatches are properly rejected with clear messages. Session-handling tests show that repeated logins and dashboard refreshes reset the system state correctly. Overall, this demonstrates a reliable and secure login system that protects sensitive student data.

POST-DEVELOPMENT TEST SCENARIOS

Test Scenario 1: Teacher – Importing Student CSV Data

- Open the system
- Log in as Teacher
  - Username: teacher
  - Password: teacher123
    - Click Import CSV
  - Select file: students_post_dev.csv
    - Verify the following students are loaded correctly in the table:
  - Student ID: S1001, Name: John Smith, Module: CS101, Average: 47.62, Attendance: 72.66, Homework: No
  - Student ID: S1004, Name: Olivia Davis, Module: CS101, Average: 53.38, Attendance: 71.56, Homework: Yes
  - Student ID: S1007, Name: William Taylor, Module: CS101, Average: 68.18, Attendance: 97.78, Homework: Yes
  - Student ID: S1010, Name: Isabella Jackson, Module: CS101, Average: 62.54, Attendance: 73.38, Homework: Yes
  - Student ID: S1013, Name: Lucas Martin, Module: CS101, Average: 82.80, Attendance: 82.60, Homework: No
    - Click Refresh Table to ensure all imported data displays correctly
  - Exit the system

Test Scenario 2: Teacher – Predictive Grade and Intervention

- Open the system
- Log in as Teacher
  - Username: teacher
  - Password: teacher123
    - Select Student ID: S1010, Isabella Jackson
    - Click Predicted Grade
  - Verify predicted grade: Average 62.54, Attendance 73.38, Homework Yes → Predicted Grade: B (attendance deduction applied)
    - Click At Risk Status
  - Verify student is not at risk
    - Click Suggested Intervention
  - Verify intervention: “Extra tutoring”
    - Repeat for Student ID: S1002, Emma Johnson
  - Predicted Grade considers Attendance 51.55, Homework No → grade deduction applied
  - At Risk: Yes

- Suggested Intervention: “Immediate intervention”
- Exit the system

Test Scenario 3: Teacher – Export Student Report

- Open the system
- Log in as Teacher
  - Username: teacher
  - Password: teacher123
    - Select Student ID: S1001, John Smith
    - Click Export Report
  - Save file as JohnSmith_Report.pdf
    - Open PDF and verify:
  - Student ID, Name, Module, Average, Letter Grade, GPA, Attendance, Homework
  - At Risk Status: Yes
  - Suggested Intervention: Immediate intervention
  - Predicted Grade: E (after low attendance & homework deduction)
  - Module distribution pie chart included for CS101
    - Repeat export for Student ID: S1007, William Taylor
  - Verify predicted grade deduction applied correctly for attendance/homework
  - At Risk Status: No
  - Suggested Intervention: None
  - Exit the system

Test Scenario 4: Teacher – Module Distribution Visualization

- Open the system
- Log in as Teacher
  - Username: teacher
  - Password: teacher123
    - Click Module Distribution
  - Enter module: CS101
    - Verify pie chart displays grades for students:
  - A: Lucas Martin, Charlotte Thompson, William Taylor
  - B: Isabella Jackson, Noah Wilson
  - C: Olivia Davis, Sophia Anderson
  - D: John Smith, Liam Brown
  - E: James Thomas, Amelia Martinez
  - Close chart and exit the system

Test Scenario 5: Parent – Viewing Student Performance

- Open the system
- Log in as Parent
  - Username: parent
  - Password: parent123
    - Verify teacher-only buttons (Import CSV, Sort Table, Module Distribution) are not visible
    - Search Student: S1003, Liam Brown
    - Click Predicted Grade
  - Verify predicted grade matches system calculation
    - Click At Risk Status
  - Verify student flagged correctly as At Risk
    - Click Suggested Intervention
  - Verify recommendation matches system logic
    - Attempt to access teacher-only features
  - Confirm access denied
  - Exit the system

Test Scenario 6: Post-Deployment – Boundary & Invalid Data

- Open the system
- Log in as Teacher
  - Username: teacher
  - Password: teacher123
    - Import CSV with boundary and invalid data:
  - S1006, Ava Moore, CS101, 48.32, 78.62, No (boundary low average)
  - S1027, David Wright, CS101, 93.49, 73.28, 0 (invalid homework format)
  - S1020, Evelyn Lewis, CS101, 57.46, -10, 0 (invalid negative attendance)
  - Verify boundary case (S1006) loaded correctly; predicted grade calculated accurately
  - Invalid fields (S1027, S1020) flagged or handled gracefully
    - Attempt Export Report for S1006 and S1027
  - Ensure system generates PDF without crashing
  - Predicted grades capped appropriately for invalid values
  - Exit the system

DEVELOPMENT CHAPTER

INTRODUCTION

The Academic Performance Decision Support System (DSS) was developed using an iterative development approach. This method allowed the system to be built step by step through several development stages. In each stage, a specific feature was designed, a prototype was created, the functionality was tested, and the results were reviewed. Based on the findings from each stage, necessary improvements were made before moving on to the next step.

This approach helped to identify and resolve issues early in the development process and ensured that the final system closely aligned with the requirements and objectives identified during the analysis phase. The test data defined during the design stage was reused throughout development to create formal test cases, allowing each feature to be validated consistently during iterative testing and post-development testing.

The system was implemented using the Python programming language due to its extensive support for database management, graphical interfaces, and data processing. Several Python libraries were used to support development, including:

Library	Purpose
Tkinter	Graphical User Interface
SQLite3	Database storage
Matplotlib	Data visualisation
ReportLab	PDF report generation
CSV	Importing student data

The solution was designed to be modular and maintainable, with different sections of the program responsible for different tasks such as database management, data processing, decision support analysis, and user interface management.

ITERATION 1 -DATABASE MODEL – 03-MARCH-2026

Objective

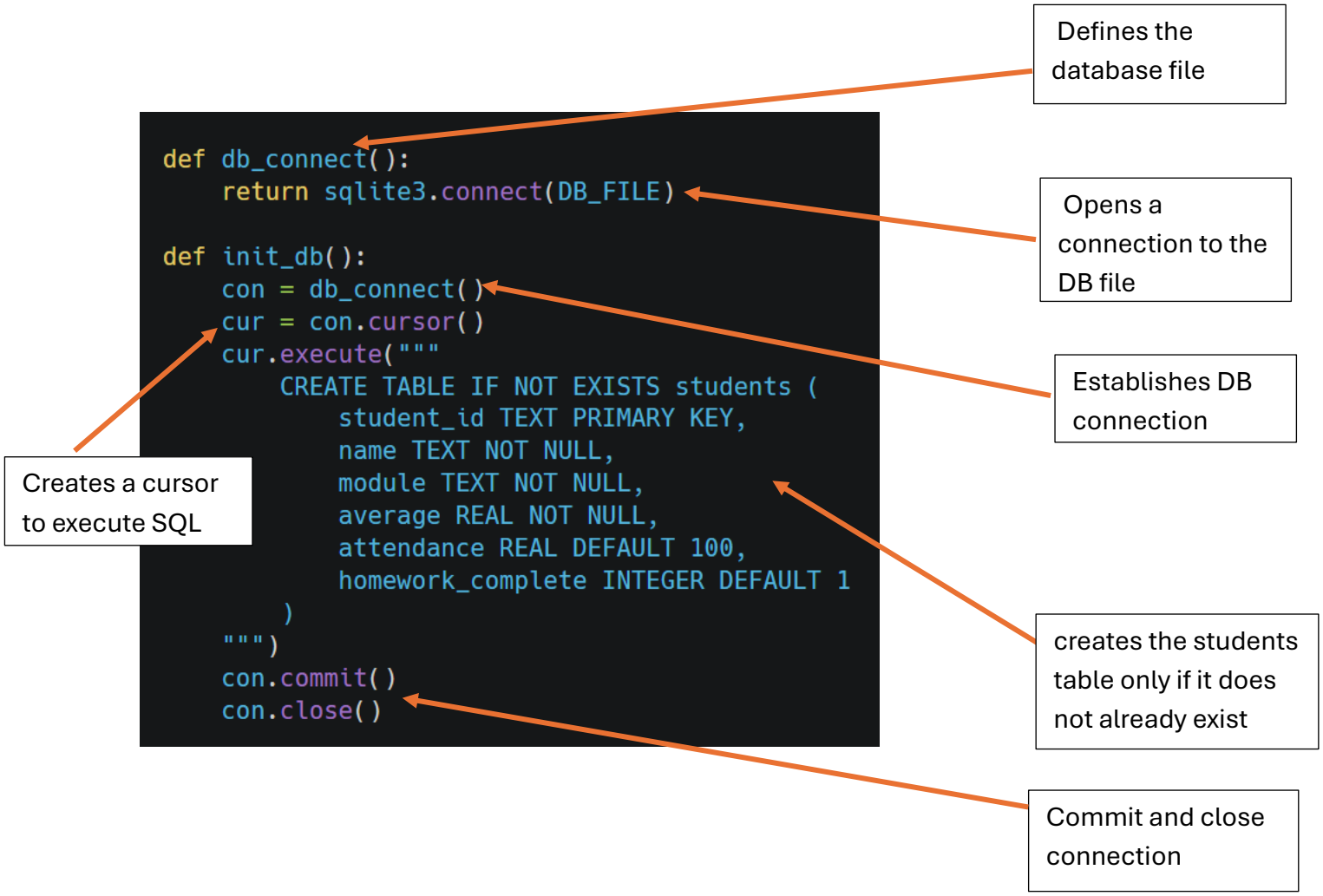
Prototype 1:
A backend-only prototype focusing on persistent data storage using SQLite, without a graphical user interface.

The first stage of development focused on creating a system that could permanently store student records that were extracted from the college Virtual Learning Environment (VLE) in a CSV file. The aim was to ensure that the data could be saved and easily accessed by the application when needed.

The database was designed to store key pieces of student information, including:

- Student ID
- Student Name
- Module
- Average
- Attendance
- Homework Completion

Code Snippet




```
def db_connect():
    return sqlite3.connect(DB_FILE)

def init_db():
    con = db_connect()
    cur = con.cursor()
    cur.execute("""
        CREATE TABLE IF NOT EXISTS students (
            student_id TEXT PRIMARY KEY,
            name TEXT NOT NULL,
            module TEXT NOT NULL,
            average REAL NOT NULL,
            attendance REAL DEFAULT 100,
            homework_complete INTEGER DEFAULT 1
        )
    """)
    con.commit()
    con.close()
```

Justification

SQLite was chosen as the database system for several reasons. First, it integrates easily with Python, making it straightforward to connect the database with the application. It also does not require a separate server installation, which simplifies the setup process and reduces system complexity.

In addition, SQLite is well suited for small-scale desktop applications like this project, as it is lightweight while still providing reliable data management. It also allows the system to store

student information permanently, ensuring that the data remains available even after the application is closed.

To maintain data integrity, the student_id field was set as the primary key. This ensures that each student record is unique and prevents duplicate entries from being added to the database.

Given the small dataset size expected in a school environment, the performance overhead of SQLite was minimal and entirely acceptable for this stage of development.

Testing

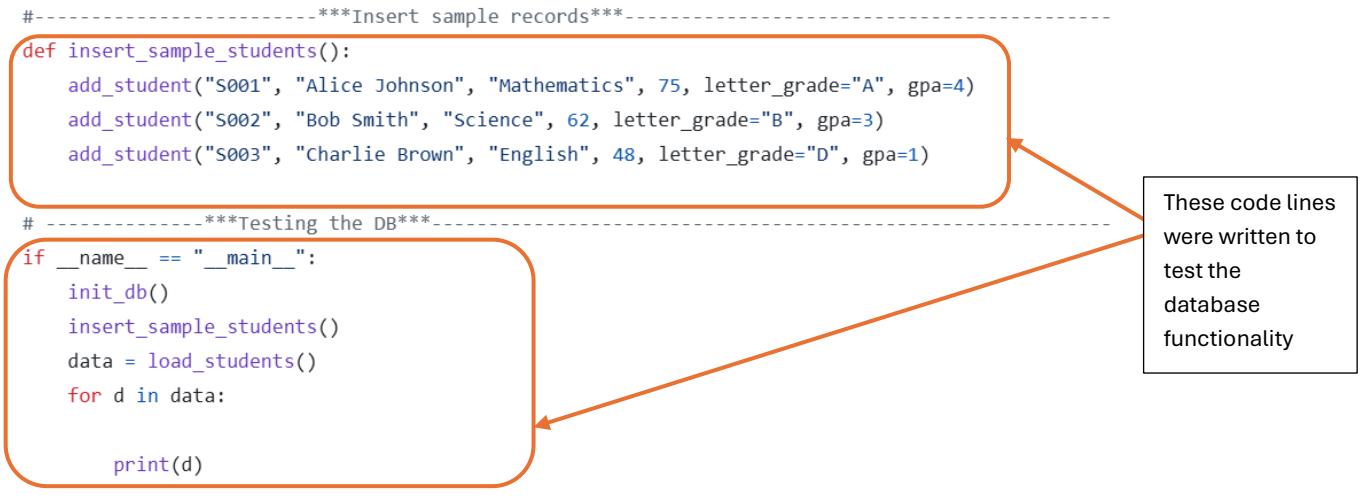
The database functionality was tested to ensure that student records could be stored, retrieved, and updated reliably. Testing focused on verifying that the database structure worked as intended and that student data was saved permanently and remained accessible across multiple runs of the application.

Test Case ID	Test Case	Test Description	Expected Outcome	Result
TC1	Database creation	Run application with no existing database file	Database file and student table created successfully	Pass
TC2	Insert valid record	Manually insert a valid student record into database	Record stored correctly in database	Pass
TC3	Retrieve records	Reload application after closing	Previously stored student records loaded correctly	Pass
TC4	Unique student ID	Attempt to insert a record using an existing student ID	Duplicate prevented or existing record updated	Fail Retested and **passed** in the prototype 3(Iteration 4)
TC5	Data storage	Close and reopen application	Student data retained in database	Pass

TC6	Invalid data type	Attempt to insert text into numeric database field	Error handled, record not stored	Fail Retested and **passed** in the prototype 3(Iteration 4)
TC7	Missing required field	Attempt to insert record with missing required values	Insert rejected, error reported	Fail Retested and **passed** in the prototype 3(Iteration 4)

Problems Encountered and Solutions

During testing of the database model, I added a set of sample student records by writing a function and printed them. However, a few test cases were initially failed. These failures were expected at this stage of development and helped identify areas requiring improvement in later iterations.



Problem 1: Duplicate student ID

When attempting to add a student with an ID that already existed, the database raised an error and stopped the insertion. Validations were not added to handle this error.

```
  File "e:\final Project\Codings\Database.py", line 35, in adding_student
    cur.execute("""
    ~~~~~~~~~~~~~~~^^^^
        INSERT INTO students
        ^^^^^^^^^^^^^^^^^^^^
        (student_id, name, module, average, attendance, homework_complete)
        ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
        VALUES (?, ?, ?, ?, ?, ?)
        ^^^^^^^^^^^^^^^^^^^^^^^^^
    """, (student_id, name, module, average, attendance, homework_complete))
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
sqlite3.IntegrityError: UNIQUE constraint failed: students.student_id
```

Solution:

This behaviour was accepted at this stage as it confirmed that the database correctly prevents duplicate records using a unique student ID. In later iterations, this was improved by handling the error more smoothly.

Problem 2: Invalid data type

When text was entered into numeric fields such as average or attendance, the database produced an error.

```
Traceback (most recent call last):
  File "e:\Final project\Codings\Database.py", line 67, in <module>
    insert_sample_students()
    ~~~~~~~~~~~~~~~~~~~~~~^^
  File "e:\Final project\Codings\Database.py", line 60, in insert_sample_students
    add_student("S005", "4334", 75)
    ~~~~~~~~~~~^^^^^^^^^^^^^^^^^^^
TypeError: add_student() missing 1 required positional argument: 'average'
```

Solution:

This highlighted the need for input validation. Error handling and validation were added in later stages to prevent incorrect data from being inserted.

Problem 3: Missing required fields

Records with missing mandatory information could not be inserted, causing an error.

```
Traceback (most recent call last):
  File "e:\Final project\Codings\Database.py", line 67, in <module>
    insert_sample_students()
    ~~~~~~~~~~~~~~~~~~~~~~^^
  File "e:\Final project\Codings\Database.py", line 61, in insert_sample_students
    add_student("S002", "Bob Smith",  62)
    ~~~~~~~~~~~^^^^^^^^^^^^^^^^^^^^^^^^^
TypeError: add_student() missing 1 required positional argument: 'average'
```

Solution:

The database constraints worked as intended by rejecting incomplete records. User-friendly validation and error messages were added in later iterations to improve system reliability.

ITERATION 2 -STUDENT CLASS – 07-MARCH-2026

Objective

Prototype 1 (extended):
Prototype 1 was extended to include an object-oriented Student model capable of generating decision-support outputs.

The second stage of development focused on creating an object-oriented data model to represent student records within the system. Using objects helps organise the program more effectively and makes the code easier to maintain and expand in the future.

Implementation
A Student class was created to represent each individual student record stored in the system. Each object of the class holds the data imported from the CSV file and saved in the database, including details such as student ID, name, module, average score, attendance, and homework completion.

The class provides functions that give useful insights from the data.

- At-Risk Detection: Checks if a student's average score is below 50 and flags them as "at risk," helping identify students who may need additional support.
- Predicted Grade: Estimates a student's final grade by considering average score, attendance, and homework completion. Attendance below 90% or incomplete homework reduces the score slightly, and the final letter grade (A–E) is assigned based on the adjusted score.
- Suggested Intervention: Recommends support measures depending on performance. High scorers require no intervention, mid-range students may be encouraged to practice or receive extra tutoring, while students below 50 are flagged for immediate intervention.

This design ensures that actionable insights are generated automatically for each student object without altering the original data imported from the CSV file.

Code Snippet

- The constructor initialises all student attributes required by the system.
- The at_risk() method checks whether the average score falls below the defined threshold.
- The predicted_grade() method adjusts the score based on attendance and homework before applying grade boundaries.

Justification

Using an object-oriented design has several benefits. It keeps all student-related data together in one organised structure, making the system easier to understand. This improves how the code is organised and makes it more readable and modular. It also makes the system easier to

maintain, update, and fix when needed. In addition, it allows the program to handle multiple student records efficiently and automatically generate useful insights.

Testing

Testing confirmed that student records stored correctly and accessed accurately. The system also successfully flagged at-risk students, predicted grades, and suggested interventions based on the imported data.

The following sample was created to test the student class

```
# ---------- ***CREATE SAMPLE STUDENT DATA***----------
s1 = Student("S001", "Alice Johnson", "Mathematics", 75, 70, 0)
s2 = Student("S002", "Bob Smith", "Science", 62,  88, 1)
s3 = Student("S003", "Charlie Brown", "English", 48, 70, 0)

students = [s1, s2, s3]

# ---------- ***TEST EACH FUNCTION WRITTEN UNDER STUDENT MODULE***----------
for s in students:
    print(f"Student: {s.name} ({s.student_id})")
    print(f"Module: {s.module}")
    print(f"Average: {s.average}")
    print(f"Attendance: {s.attendance}%")
    print(f"Homework Complete: {'Yes' if s.homework_complete else 'No'}")
    print(f"At Risk: {s.at_risk()}")
    print(f"Predicted Grade: {s.predicted_grade()}")
    print(f"Suggested Intervention: {s.suggested_intervention()}")
```

Test cases

Test Case ID	Test Case	Test Description	Input Data	Expected Outcome	Result
TC 8	Create student	Create student object with valid data	Student ("S001", "Alice Johnson", "Mathematics", 75, 70, 0)	Student object created successfully	Pass
TC 9	Store attributes	Assign values to all fields	ID: S001, Name: Alice Johnson, Module: Mathematics, Average: 75	All values stored correctly	Pass
TC 10	At-risk status	Check at-risk status for low average	Average = 45	At-risk returns "Yes"	Pass
TC 11	Not at risk	Check at-risk status for	Average = 75	At-risk returns "No"	Pass

		acceptable average			
TC 12	Predicted grade	Calculate grade with good performance	Student_id = S1021, Name = Mathew Lee, Module = CS101, Average = 72, Attendance = 92, Homework = 1	Correct grade returned (A)	Pass
TC 13	Predicted grade penalty	Calculate grade with penalties applied	Student_id = S1003, Name = Liam Brown, Module = CS101 Average = 42.67, Attendance = 83.92, Homework = 0	Grade reduced correctly	Pass
TC 14	Suggested intervention	Low performance intervention	Average = 48	"Immediate Intervention" returned	Pass
TC 15	No intervention needed	High performance intervention check	Average = 75	"None" returned	Pass

Problems Encountered and Solutions

Duplicate Student Records
student_id field accepted duplicate entries.

Solution: The student_id field was set as the primary key in the database, preventing duplicate records.

ITERATION 3 – USER INTERFACE - 07-MARCH-2026

Objective

Prototype 2:
A functional graphical user interface displaying student data using sample records, without full backend integration.

The main objective was to provide a clear, user-friendly interface that allows users to interact with student performance data efficiently while supporting different user roles within the system.

Implementation

The user interface was developed using the Tkinter and ttk libraries in Python, which provide standard GUI components suitable for desktop applications.

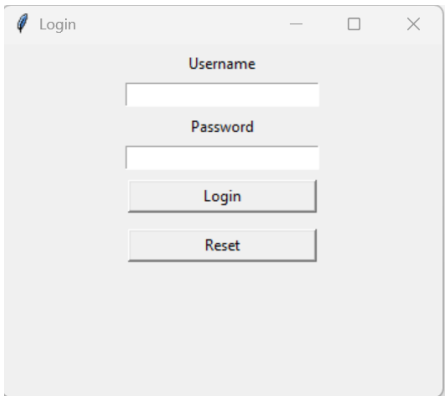
The user interface consists of two primary windows:

Login Window

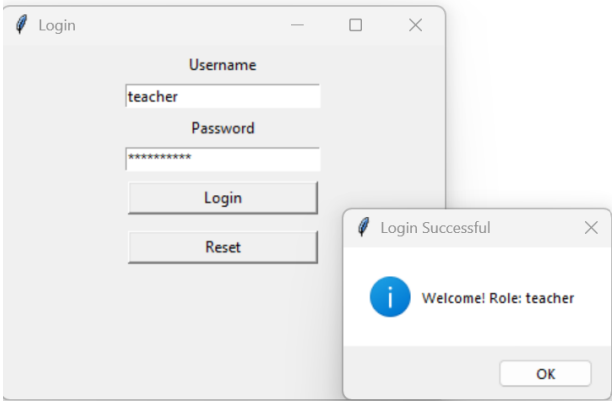
The login interface allows users to authenticate themselves before accessing the system. It includes:

- A username input field
- A password input field with masked characters
- A Login button to validate credentials
- A Reset button to clear all input fields

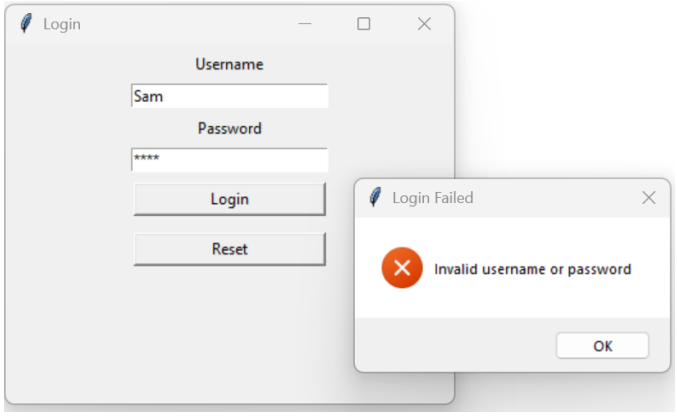
User credentials are validated against a predefined in memory data structure. Upon successful login, the system identifies the user's role and launches the main dashboard interface accordingly. If the credentials are invalid, an error message is displayed to notify the user.



Login Screen UI



Successful login message



Unsuccessful login message

Main Dashboard Window

After authentication, users are redirected to the Academic Performance DSS dashboard, which serves as the central hub of the system. The dashboard includes:

- A role-based toolbar containing action buttons
- A student data table for visualising academic performance data

Role Based Interface

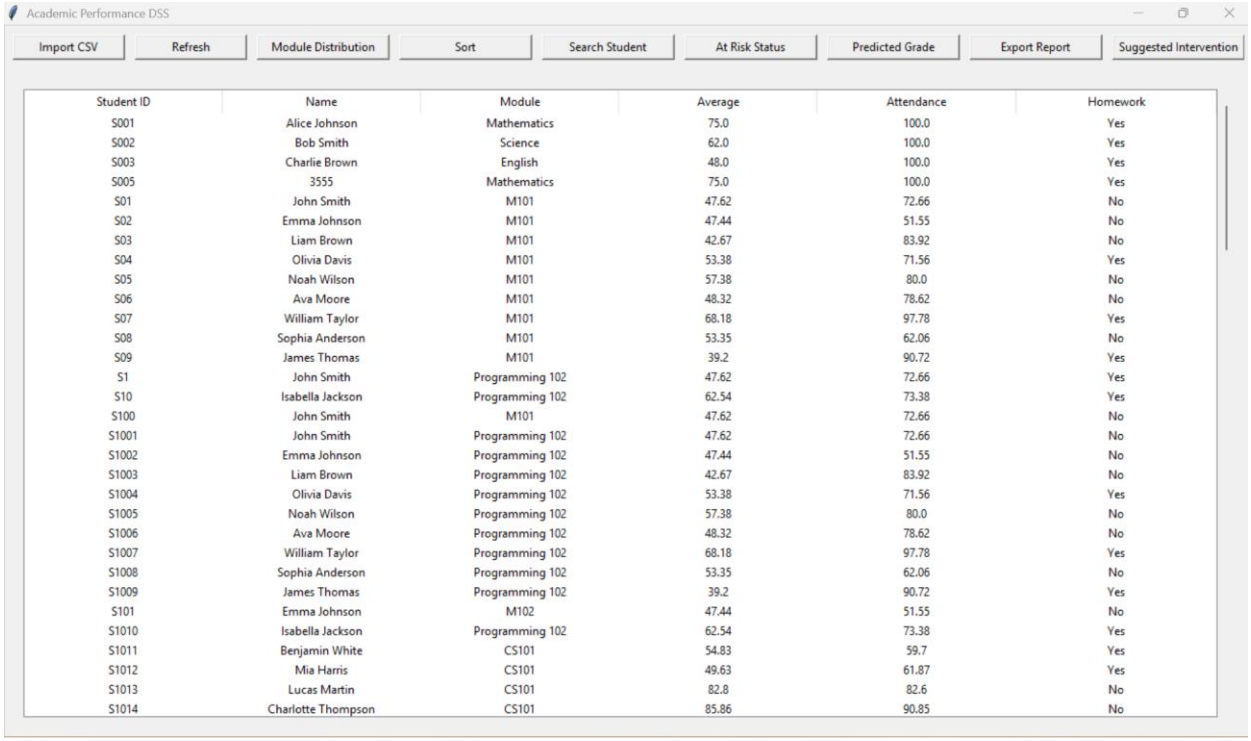
The system supports two user roles:

Teacher role

Teachers are granted access to additional administrative and analytical functions, including:

- Import CSV
- Refresh data
- Sort Table
- Search students
- Module distribution
- At risk status
- Predicted grade
- Suggested interventions
- Export report

A vertical scrollbar is included to support larger datasets. Sample student data is used in this iteration to demonstrate the layout and behaviour of the interface.



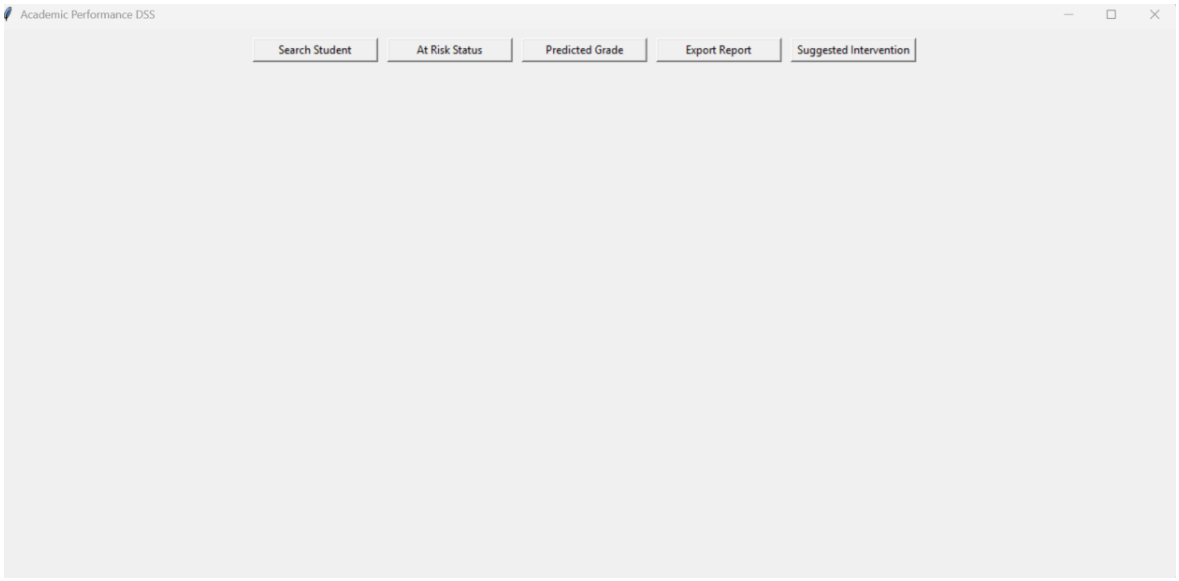
Student ID	Name	Module	Average	Attendance	Homework
S001	Alice Johnson	Mathematics	75.0	100.0	Yes
S002	Bob Smith	Science	62.0	100.0	Yes
S003	Charlie Brown	English	48.0	100.0	Yes
S005	3555	Mathematics	75.0	100.0	Yes
S01	John Smith	M101	47.62	72.66	No
S02	Emma Johnson	M101	47.44	51.55	No
S03	Liam Brown	M101	42.67	83.92	No
S04	Olivia Davis	M101	53.38	71.56	Yes
S05	Noah Wilson	M101	57.38	80.0	No
S06	Ava Moore	M101	48.32	78.62	No
S07	William Taylor	M101	68.18	97.78	Yes
S08	Sophia Anderson	M101	53.35	62.06	No
S09	James Thomas	M101	39.2	90.72	Yes
S1	John Smith	Programming 102	47.62	72.66	Yes
S10	Isabella Jackson	Programming 102	62.54	73.38	Yes
S100	John Smith	M101	47.62	72.66	No
S1001	John Smith	Programming 102	47.62	72.66	No
S1002	Emma Johnson	Programming 102	47.44	51.55	No
S1003	Liam Brown	Programming 102	42.67	83.92	No
S1004	Olivia Davis	Programming 102	53.38	71.56	Yes
S1005	Noah Wilson	Programming 102	57.38	80.0	No
S1006	Ava Moore	Programming 102	48.32	78.62	No
S1007	William Taylor	Programming 102	68.18	97.78	Yes
S1008	Sophia Anderson	Programming 102	53.35	62.06	No
S1009	James Thomas	Programming 102	39.2	90.72	Yes
S101	Emma Johnson	M102	47.44	51.55	No
S1010	Isabella Jackson	Programming 102	62.54	73.38	Yes
S1011	Benjamin White	CS101	54.83	59.7	Yes
S1012	Mia Harris	CS101	49.63	61.87	Yes
S1013	Lucas Martin	CS101	82.8	82.6	No
S1014	Charlotte Thompson	CS101	85.86	90.85	No

The main dashboard for teachers

Parent role

Parents have access only to informational features and cannot modify data.

This role-based UI design ensures that users see only the functions relevant to their responsibilities, improving usability and security. Parents can only search for their child, view the at-risk level, suggested interventions, predicted grade, and export the report. A pop-up message is displayed when the at-risk level, suggested interventions, or predicted grade buttons are clicked, showing the student's performance details.



The main dashboard for parents

Code Snippet for the main window UI

```
def build_main_window(role):
    global tree
    tree = None

    root = tk.Tk()
    root.title("Academic Performance DSS")
    root.geometry("1100x600")

    btn_frame = tk.Frame(root)
    btn_frame.pack(pady=10)

    if role == "teacher":
        tk.Button(btn_frame, text="Import CSV", width=15, command=import_csv).pack(side=tk.LEFT,
padx=5)
        tk.Button(btn_frame, text="Refresh", width=15, command=refresh_table_button).pack(side=tk.LEFT,
padx=5)
        tk.Button(btn_frame, text="Module Distribution", width=18,
command=module_distribution).pack(side=tk.LEFT, padx=5)


    tk.Button(btn_frame, text="Search Student", width=18, command=search_ui).pack(side=tk.LEFT, padx=5)
    tk.Button(btn_frame, text="At Risk Status", width=18, command=show_at_risk).pack(side=tk.LEFT,
padx=5)
    tk.Button(btn_frame, text="Predicted Grade", width=18,
command=show_predicted_grade).pack(side=tk.LEFT, padx=5)
    tk.Button(btn_frame, text="Export Report", width=18,
command=export_student_report).pack(side=tk.LEFT, padx=5)
    tk.Button(btn_frame, text="Suggested Intervention", width=18,
command=show_intervention).pack(side=tk.LEFT, padx=5)
    tk.Button(btn_frame, text="Close", width=15, command=root.destroy).pack(side=tk.LEFT, padx=5)

    #-------------***ROLE BASED (UI)***-----------------------------------
    if role == "teacher":
        table_frame = tk.Frame(root)
        table_frame.pack(fill=tk.BOTH, expand=True, padx=20, pady=20)

        columns = ("Student ID", "Name", "Module", "Average", "Attendance", "Homework")
        tree = ttk.Treeview(table_frame, columns=columns, show="headings")
        for col in columns:
            tree.heading(col, text=col)
            tree.column(col, width=160, anchor="center")

        scrollbar = ttk.Scrollbar(table_frame, orient="vertical", command=tree.yview)
        tree.configure(yscrollcommand=scrollbar.set)
        tree.pack(side=tk.LEFT, fill=tk.BOTH, expand=True)
        scrollbar.pack(side=tk.RIGHT, fill=tk.Y)

        refresh_table()

    root.mainloop()
```

Function to build the main application window based on user role

Create the main window

These buttons appear only if the user role is teacher

Common buttons for all users

If the user is a teacher, display the student data table

Code snippet for the login UI

```
init_db()
load_students()

login_window = tk.Tk()
login_window.title("Login")
login_window.geometry("350x280")

tk.Label(login_window, text="Username").pack(pady=5)
username_entry = tk.Entry(login_window, width=25)
username_entry.pack()

tk.Label(login_window, text="Password").pack(pady=5)
password_entry = tk.Entry(login_window, show="*", width=25)
password_entry.pack()

tk.Button(login_window, text="Login", width=20, command=login).pack(pady=8)
tk.Button(login_window, text="Reset", width=20, command=reset_fields).pack(pady=5)

login_window.mainloop()
```

Create the login window

Username input

Password input

login button

Reset button

Justification

The user interface design was developed to provide a clear, structured, and easy-to-use experience for all users. Separating the login interface from the main dashboard improves clarity and security by ensuring that authentication is completed before access to system features is granted. This reduces the risk of unauthorised access and supports controlled system navigation.

The interface also implements role-based access control at the user-interface level. Different features are displayed depending on whether the user is a teacher or a parent, ensuring that users only see functionality relevant to their role. This improves usability while also helping to protect sensitive student data.

Reusable interface components such as buttons and tables were used consistently throughout the application. This improves visual consistency and reduces code duplication, making the system easier to maintain and modify. The modular structure of the interface also supports future expansion, as additional features can be added without requiring major changes to the existing layout.

Tkinter was chosen as the interface toolkit because it is platform-independent and integrates directly with Python. This allows the application to run on multiple operating systems without modification and supports rapid development and testing of the interface. Overall, the design choices create a user interface that is efficient, maintainable, and suitable for the requirements of the system.

Interface responsiveness was prioritised, and with only a small number of students, the system performed smoothly during testing.

Test cases

Test case ID	Test Case	Test Description	Input Data	Expected Outcome	Result
TC16	Successful login	Log in with valid teacher credentials	Username: teacher Password: teacher123	Teacher dashboard loads correctly	Pass
TC17	Successful login (parent)	Log in with valid parent credentials	Username: parent Password: parent123	Parent dashboard loads correctly	Pass

TC18	Invalid login	Log in with incorrect credentials	Username: teacher Password: wrongpass	Error message displayed	Pass
TC19	Role-based UI	Compare teacher and parent interfaces	Teacher vs Parent login	Teacher sees admin controls; Parent sees restricted view	Pass
TC20	Reset fields	Click reset on login screen	Username and password entered	Fields cleared correctly	Fail Retested after fixing the issue and passed.

Problems Encountered and Solutions

1. UI Clutter with Multiple Buttons

The toolbar initially appeared cluttered with all controls displayed together.

Solution:

Removed the close button to avoid this issue.

2. No Backend Integration in Early Stage

The UI initially lacked real data sources and analytics.

Solution:

Sample student data was used to design and test the UI layout, enabling smooth integration with the data model and database in later iterations.

3. Cursor Reset Issue
After clicking the reset button, the input fields cleared but the cursor did not return to the username field, reducing usability.

Solution:
Added username_entry.focus() to the reset function to automatically place the cursor back in the username field.

Once the cursor focus issue was resolved, the reset functionality was re-tested and passed, confirming improved usability and correct behaviour.

ITERATION 4 – CSV IMPORT -07-3-2026

Objective

Prototype 3:
A partially integrated prototype combining the graphical interface with live database and CSV data import functionality.

The aim of this iteration was to allow student data to be imported from a CSV file into the system automatically. This reduces the need for manual data entry and allows student records exported from the college Virtual Learning Environment (VLE) to be added quickly and accurately to the application's database.

Implementation

The CSV import feature is implemented using Python's built-in csv module and an SQLite database. When triggered, a file dialog allows the user to select a CSV file, and the function exits safely if no file is chosen. The program ensures the student table exists, then reads the file using csv.DictReader and inserts each row into the database using an INSERT OR REPLACE statement to avoid duplicates.

Numeric values are converted appropriately, with default values used where needed, and data is standardised for consistency. After processing, changes are saved, the database is closed, and the user interface is refreshed. Basic error handling is included to manage issues such as missing files, invalid data, or missing columns.

Code snippet

```
def import_csv():

    #----------------------***OPEN FILE DIALOG TO SELECT CSV FILE***----------------
    try:
        path = filedialog.askopenfilename(filetypes=[("CSV","*.csv")])
        if not path:
            return

        #----------------------***ENSURE STUDENT TABLE EXISTS***---------------------
        init_db()

        con = db_connect()
        cur = con.cursor()

        #----------------------***READ CSV FILE AND INSERT DATA INTO DATABASE***---------
        with open(path, "r", newline='', encoding='utf-8') as f:
            reader = csv.DictReader(f)
            for row in reader:
                cur.execute("""
                    INSERT OR REPLACE INTO students
                    (student_id, name, module, average, attendance, homework_complete)
                    VALUES (?, ?, ?, ?, ?, ?)
                """, (
                    row["Student ID"],
                    row["Name"],
                    row["Module"],
                    float(row["Average"]),
                    float(row.get("Attendance", 100)),
                    1 if row.get("Homework", "Yes").lower() in ["yes","1","true"] else 0
                ))

        #----------------------***SAVE CHANGES AND CLOSE DATABASE***-------------------------
        con.commit()
        con.close()

        load_students()
        refresh_table()

        messagebox.showinfo("Success", "CSV Data Imported Successfully")

    #----------------------***HANDLE FILE OR DATA ERRORS***---------------------------------
    except FileNotFoundError:
        messagebox.showerror("Error", "CSV file not found.")

    except ValueError:
        messagebox.showerror("Error", "Invalid data format in CSV file.")

    except KeyError as e:
            messagebox.showerror("CSV Error", f"Missing column: {e}")
            return
```

Allows the user to select an external CSV file using a file dialog.

Prevents execution if no file is selected by the user.

Ensures the required database table exists before processing data.

Establishes a database connection and cursor to execute SQL commands.

Reads structured CSV data using column headers as dictionary keys and process each student record sequentially.

Maintains data integrity by preventing duplicate student records

Saves all database changes and releases system resources safely.

Error handling for missing csv file, invalid data formats & missing required columns

Justification

CSV was chosen as the import format because it is widely supported and easily exported from spreadsheet software and learning platforms. Using Python's built-in csv module provides a reliable and simple way to read structured data while reducing the risk of file formatting issues.

The use of INSERT OR REPLACE improves data integrity by ensuring that each student record remains unique based on student ID. This avoids duplicate entries and allows updated data to overwrite older records when needed.

Including exception handling improves the robustness of the system by preventing crashes and providing clear feedback when errors occur. Overall, the design is suitable for the size and purpose of the application and supports efficient data management.

Alternative data import formats such as JSON were considered however, CSV was chosen due to its widespread support and compatibility with existing school systems.

Testing

The CSV import feature was tested using several different scenarios to ensure it works correctly in different situations.

Test Case ID	Test Case	Input Data	Expected Outcome	Result
TC21	Valid CSV import	Correct CSV file selected	Student records imported successfully	Pass
TC22	Duplicate student ID	CSV containing existing ID	Record updated, no duplicate created	Fail Retested after fixing and passed
TC23	Cancel file selection	Cancel file dialog	Import safely cancelled	Pass
TC24	Invalid data format	Text in numeric field	Error message displayed	Fail Retested after fixing and passed
TC25	Missing CSV column	Column removed from CSV	Error message displayed, import stopped	Fail Retested after fixing and passed

Problems Encountered and Solutions

During the development of the CSV import feature, several issues were identified and resolved to improve reliability and usability.

Issue 1: Duplicate student records

When importing a CSV file containing an existing student ID, the database initially raised an error due to the primary key constraint.

Solution:

The SQL statement was changed to use INSERT OR REPLACE. This ensures that if a student ID already exists, the existing record is updated rather than creating a duplicate. This maintains data integrity and prevents repeated records.

```
with open(path, "r", newline='', encoding='utf-8') as f:
    reader = csv.DictReader(f)
    for row in reader:
        cur.execute("""
            INSERT OR REPLACE INTO students
            (student_id, name, module, average, attendance, homework_complete)
            VALUES (?, ?, ?, ?, ?, ?)
        """, (
```

Issue 2: Invalid data types in CSV files

Early testing showed that importing text values into numeric fields such as average score or attendance caused runtime errors.

```
File "e:\Final Project\Code\Final\Student Academic Performance DSS.py", line 407, in import_csv
    float(row["Average"]),
    ~~~~~^^^^^^^^^^^^^^^^
ValueError: could not convert string to float: 'adadad'
```

Solution:

Data type conversion was added using float() and exception handling (ValueError) was implemented. This prevents the program from crashing and displays a clear error message to the user when incorrect data is detected.

```
except ValueError:
    messagebox.showerror("Error", "Invalid data format in CSV file.")
```

Issue 3: Missing columns in CSV file

If a CSV file did not contain all required column headers, the program raised a KeyError, stopping the import process.

```
File "e:\Final Project\Code\Final\Student Academic Performance DSS.py", line 404, in import_csv
    row["Student ID"],
    ~~~^^^^^^^^^^^^^^
KeyError: 'Student ID'
```

Solution:

A KeyError exception handler was added to detect missing columns and display a helpful

error message. This informs the user of the problem and prevents invalid data from being imported.

```
except KeyError as e:
        messagebox.showerror("CSV Error", f"Missing column: {e}")
        return
```

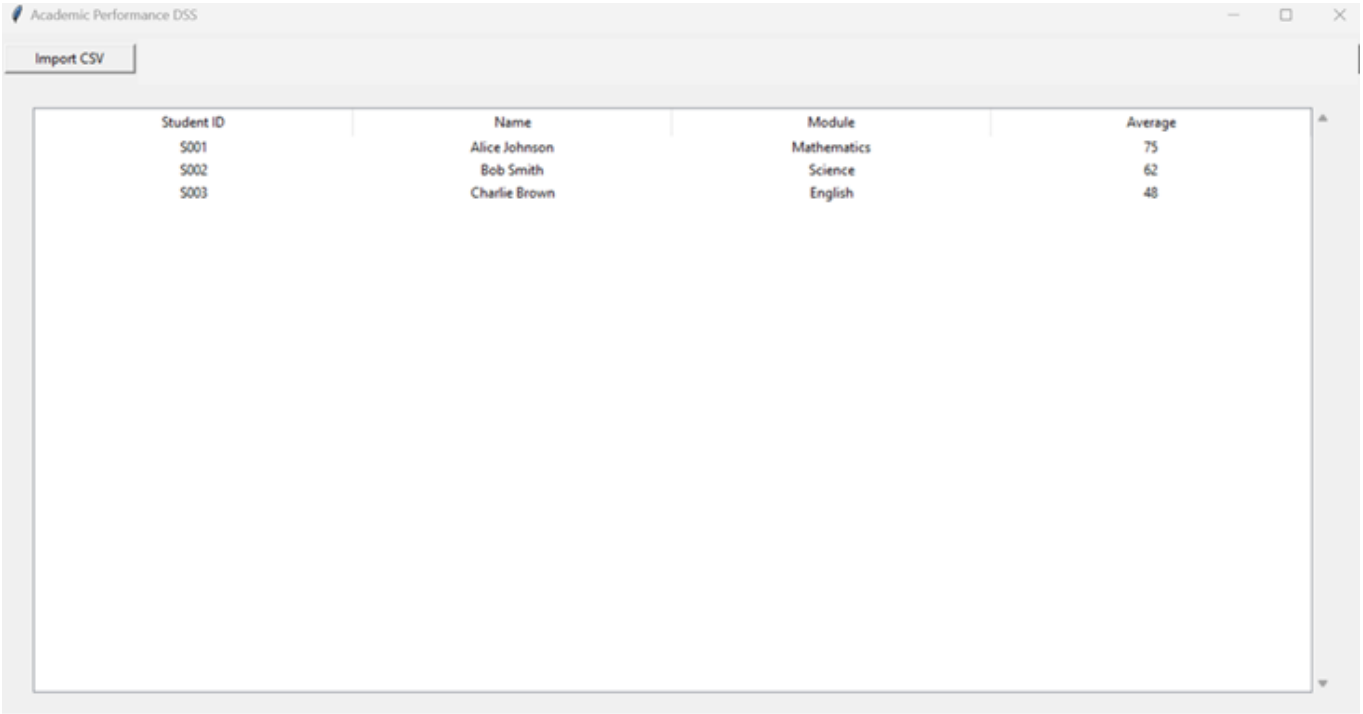
After adding exception handling for invalid data and missing columns, the CSV import feature was re-tested using all previously failing scenarios and passed in every case.

Evaluation

The CSV import feature successfully meets its objective by allowing student records to be imported in a widely compatible format. It integrates well with the existing database and user interface, supports administrative tasks, and makes the system more practical for real-world use.

The modular design of the feature also allows for future improvements, such as importing filtered records or supporting additional file formats. Overall, the functionality enhances the usefulness and flexibility of the system.

User Interface for the system after developing iteration 4



ITERATION 5 – SORTING ALGORITHM - 08-3-2026

Objective

Prototype 3 (refined):
Prototype 3 was refined to include dynamic sorting of student records within the teacher dashboard.

The aim of this iteration was to allow teachers to sort student records so that academic performance could be analysed more easily and patterns within the data could be identified.

Implementation

A Bubble Sort algorithm is used to sort a list of student objects by repeatedly comparing and swapping neighbouring records when they are in the wrong order. Each student is assigned a tuple-based comparison key containing attributes such as student ID, name, module, and average score, allowing multi-criteria sorting using lexicographical comparison. The algorithm also supports both ascending and descending order, controlled by a Boolean flag, making the sorting flexible and adaptable.

Code snippet

```
def sort_students(student_list, ascending=True):

    #--------------------***INPUT VALIDATION***--------------------------------------
    if student_list is None or not isinstance(student_list, list):
        return []
    n = len(student_list)

    #-------------***BUBBLE SORT TO SORT STUDENT TABLE IN ASCENDING & DESCENDING ORDER***-------
    for i in range(n):
        for j in range(0, n - i - 1):
            a = student_list[j]
            b = student_list[j + 1]
            #-----------------------****CREATE COMPARISON KEYS***-------------------------------
            key_a = (a.student_id, a.name, a.module, a.average)
            key_b = (b.student_id, b.name, b.module, b.average)
            #-------------------------***SORTING IN ASCENDING & DESCENDING ORDERS***-------------
            if ascending:
                if key_a > key_b:
                    student_list[j], student_list[j+1] = student_list[j+1], student_list[j]
            else:
                if key_a < key_b:
                    student_list[j], student_list[j+1] = student_list[j+1], student_list[j]
    return student_list
```

Justification

A Bubble Sort algorithm was used to sort the list of student objects as the dataset is small, making a simple algorithm sufficient without adding unnecessary complexity. Bubble Sort is easy to understand and implement, which is suitable for an educational application. The

algorithm uses a tuple-based comparison key (student ID, name, module, and average score), allowing Python to perform multi-field sorting. First, it compares the student ID, which acts as the primary sorting field. If two students have the same ID, the system then compares their names. If the names are also identical, it moves on to compare the module. Finally, if all previous fields are the same, the average score is used as the last sorting criterion. This ensures a consistent and structured ordering of student data.

It also supports both ascending and descending order using a Boolean parameter, improving flexibility. Input validation ensures only valid data is processed, increasing reliability. Overall, the design prioritises simplicity, readability, and functionality.

More efficient algorithms like Quick Sort were considered, but Bubble Sort was chosen as it is simple and sufficient for the small dataset.

Testing

Test Case ID	Test Case	Test Description	Input Data	Expected Outcome	Result
TC26	Valid sort (ascending)	Sort list in ascending order	List of student objects	Students sorted correctly	Pass
TC27	Valid sort (descending)	Sort list in descending order	List of student objects	Students sorted correctly	Fail Retested after fixing and passed.

Problems Encountered and Solutions

Sorting comparisons not working correctly

Initially, clicking the Sort button in the main window always produced the same ordering of student records. Users could not switch between ascending and descending order, which limited the usefulness of the sorting feature and caused confusion during testing.

Solution:

The issue was resolved by introducing a Boolean control variable called sort_ascending. This variable tracks the current sort direction and allows the sorting order to toggle each time the Sort button is pressed.

The sort_students_ui() function was updated to:

- Pass the current sort direction to the sorting algorithm

- Reverse the sort direction after each button press
- Refresh the table to display the newly sorted data

Code snippet

Toggles between ascending and descending sorting of the student list and updates the table display.

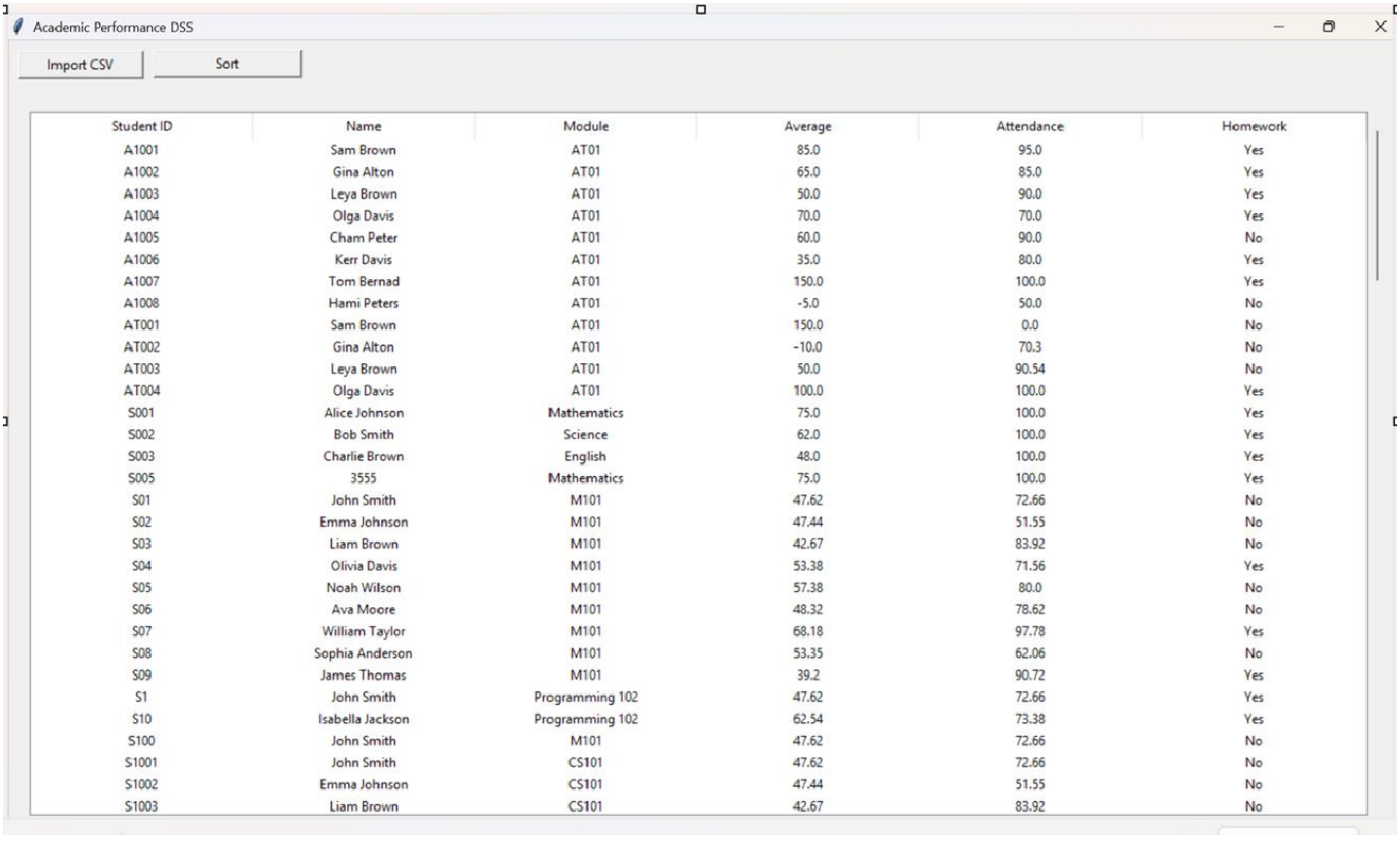
```
# ---------------- ***SORT STUDENTS UI FUNCTION*** ----------------------------
sort_ascending = True

def sort_students_ui():
    global students, sort_ascending
    students = sort_students(students, ascending=sort_ascending)
    sort_ascending = not sort_ascending
    refresh_table(students)
```

The algorithm works by comparing neighbouring student records using a combined set of key values. A simple Boolean flag controls whether the list is sorted in ascending or descending order. Once sorting is complete, the updated list is applied directly, and the table refreshes to display the new order.

Evaluation

The sorting module now works reliably and meets its intended purpose. It allows student records to be organised clearly, supports different viewing needs, and integrates well with the user interface. The simple design makes the function easy to understand and maintain, while still providing effective sorting functionality for the system.

User Interface for the system after developing iteration 5



Student ID	Name	Module	Average	Attendance	Homework
A1001	Sam Brown	AT01	85.0	95.0	Yes
A1002	Gina Alton	AT01	65.0	85.0	Yes
A1003	Leya Brown	AT01	50.0	90.0	Yes
A1004	Olga Davis	AT01	70.0	70.0	Yes
A1005	Cham Peter	AT01	60.0	90.0	No
A1006	Kerr Davis	AT01	35.0	80.0	Yes
A1007	Tom Bernad	AT01	150.0	100.0	Yes
A1008	Hami Peters	AT01	-5.0	50.0	No
AT001	Sam Brown	AT01	150.0	0.0	No
AT002	Gina Alton	AT01	-10.0	70.3	No
AT003	Leya Brown	AT01	50.0	90.54	No
AT004	Olga Davis	AT01	100.0	100.0	Yes
S001	Alice Johnson	Mathematics	75.0	100.0	Yes
S002	Bob Smith	Science	62.0	100.0	Yes
S003	Charlie Brown	English	48.0	100.0	Yes
S005	3555	Mathematics	75.0	100.0	Yes
S01	John Smith	M101	47.62	72.66	No
S02	Emma Johnson	M101	47.44	51.55	No
S03	Liam Brown	M101	42.67	83.92	No
S04	Olivia Davis	M101	53.38	71.56	Yes
S05	Noah Wilson	M101	57.38	80.0	No
S06	Ava Moore	M101	48.32	78.62	No
S07	William Taylor	M101	68.18	97.78	Yes
S08	Sophia Anderson	M101	53.35	62.06	No
S09	James Thomas	M101	39.2	90.72	Yes
S1	John Smith	Programming 102	47.62	72.66	Yes
S10	Isabella Jackson	Programming 102	62.54	73.38	Yes
S100	John Smith	M101	47.62	72.66	No
S1001	John Smith	CS101	47.62	72.66	No
S1002	Emma Johnson	CS101	47.44	51.55	No
S1003	Liam Brown	CS101	42.67	83.92	No

ITERATION 6 – SEARCHING ALGORITHM- 08-3-2026

Objective

Prototype 3 (extended)
Prototype 3 was extended to support validated searching with role-based output control.

The aim of this iteration was to enable users to locate individual student records by searching using either a student ID or a student name. At the same time, the system enforces role-based access control, ensuring that teachers can view multiple student records, while parents are restricted to viewing information related only to their own child.

Implementation

The search functionality was implemented using a linear search algorithm, as the dataset size is small and does not require a more complex approach. Each student record is checked sequentially to determine whether the search term appears in the student's name or student ID. This provides a clear and efficient solution appropriate for the scale of the system.

To improve robustness, the search is performed in a case-insensitive manner, ensuring users can find records without exact formatting. Input validation is applied before the search using a separate function. This ensures the search term is not empty and follows valid formats for student IDs and names. If the user cancels the search or enters invalid input, the system exits safely and displays clear feedback, preventing unnecessary processing.

Once results are found, role-based behaviour is applied. Teachers can view all matching records within the table, supporting administrative tasks. Parents are restricted to viewing only the first matching student's details, including performance indicators such as predicted grade and intervention. This enforces data privacy and aligns with real-world data protection requirements.

Code Snippets

```
#------GeeksforGeeks (2016). Linear Search Python. [online] GeeksforGeeks.
# Available at: https://www.geeksforgeeks.org/python/python-program-for-linear-search/

#-------------------***SEARCH STUDENTS FUNCTION***-----------------------------------------------
def search_students(students, search_term):
    if not search_term:
        return []
    search_term = search_term.lower()
    matches = []
    for s in students:
        if search_term in s.name.lower() or search_term in s.student_id.lower():
            matches.append(s)
    return matches
```

Performs a linear search through the student list, returning all records where the name or student ID matches the search term (case-insensitive).

```
def search_ui():
    term = simpledialog.askstring("Search", "Enter Student ID or Name")

    if term is None:
        return  #------------***FOR CANCEL SEARCH***---------------

    valid, error_message = is_valid_search_term(term)

    if not valid:
        messagebox.showerror("Invalid Input", error_message)
        return

    results = search_students(students, term)

    if current_role == "teacher":
        if results:
            refresh_table(results)
        else:
            messagebox.showinfo("No Result", "No student found with that ID or name.")

    elif current_role == "parent":
        if results:
            s = results[0]
            messagebox.showinfo(
                "Student Found",
                f"Name: {s.name}\n"
                f"Module: {s.module}\n"
                f"Average: {s.average}\n"
                f"At Risk: {s.at_risk()}\n"
                f"Predicted Grade: {s.predicted_grade()}\n"
                f"Suggested Intervention: {s.suggested_intervention()}"
            )
        else:
            messagebox.showinfo("No Result", "No student found with that ID or name.")
```

Prompts the user to search for a student by ID or name and displays results differently based on their role (teacher or parent).

Test cases

The following test cases were carried out to verify the search functionality of the system

Test Case ID	Test Case	Input Data	Expected Outcome	Result
TC28	Valid student ID	S12	Matching student returned	Pass
TC29	Valid student name	Daniel	Matching student(s) returned	Pass
TC30	Case-insensitive search	dAnIeL	Match found regardless of case	Fail Retested and passed.
TC31	Partial match	Dan	Student(s) with matching name returned	Pass
TC32	Empty input	""	Validation error message displayed	Fail

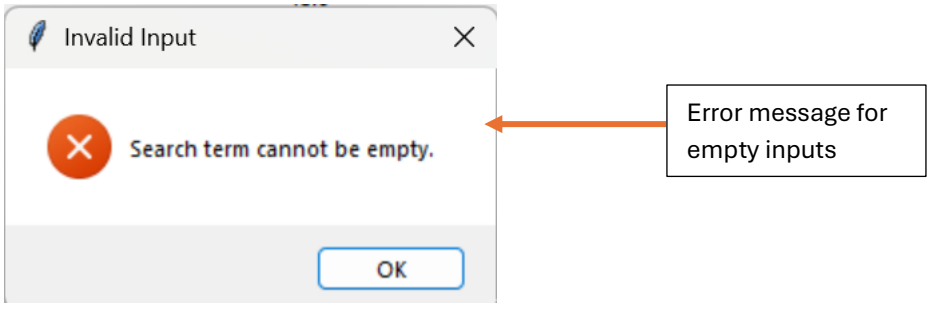
				Retested and passed.
TC33	Cancel search	Cancel dialog	Search exits safely with no action	Pass
TC34	Invalid characters	@@@	Validation error message displayed	Fail Retested and passed.
TC35	Non-existent ID	S99	"No student found" message displayed	Fail Retested and passed.
TC36	Teacher role search	Valid name	Table refreshed with matching results	Pass
TC37	Parent role search	Valid ID	Student details shown in popup only	Fail Retested and passed.

Problems Encountered and Solutions

Issue 1: Invalid search input caused unexpected behaviour

Problem:
Early testing showed that empty input or invalid characters could be entered, which led to unnecessary searching or unclear system behaviour.

Solution:
A separate validation function (is_valid_search_term) was implemented to check input before performing the search. Invalid input now triggers a clear error message, and the search does not proceed. This improves robustness and user feedback.



Issue 2: Case-sensitive searches returned no results

Problem:
Initially, searches failed if users entered names or IDs with different capitalisation.

Solution:
Both the search term and student attributes were converted to lower case before comparison. This made the search case-insensitive and improved usability.

Issue 3: Parents could potentially view multiple records

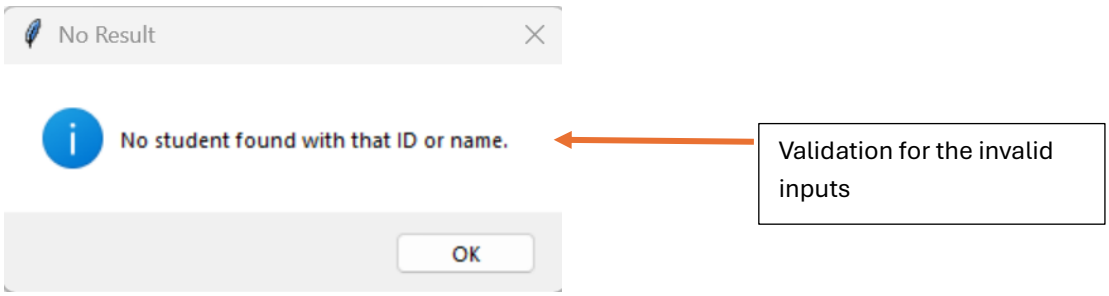
Problem:
Without role-based logic, parents could view search results in the same way as teachers.

Solution:
Role-based behaviour was added. Teachers can view all matching results in the table, while parents are restricted to viewing only the first matching student's details. This protects sensitive data and enforces access control.

Issue 4: No feedback when no results were found

Problem:
When a search matched no records, the system initially gave no response.

Solution:
An informative message box was added to notify users when no matching student is found. This improves clarity and user experience.

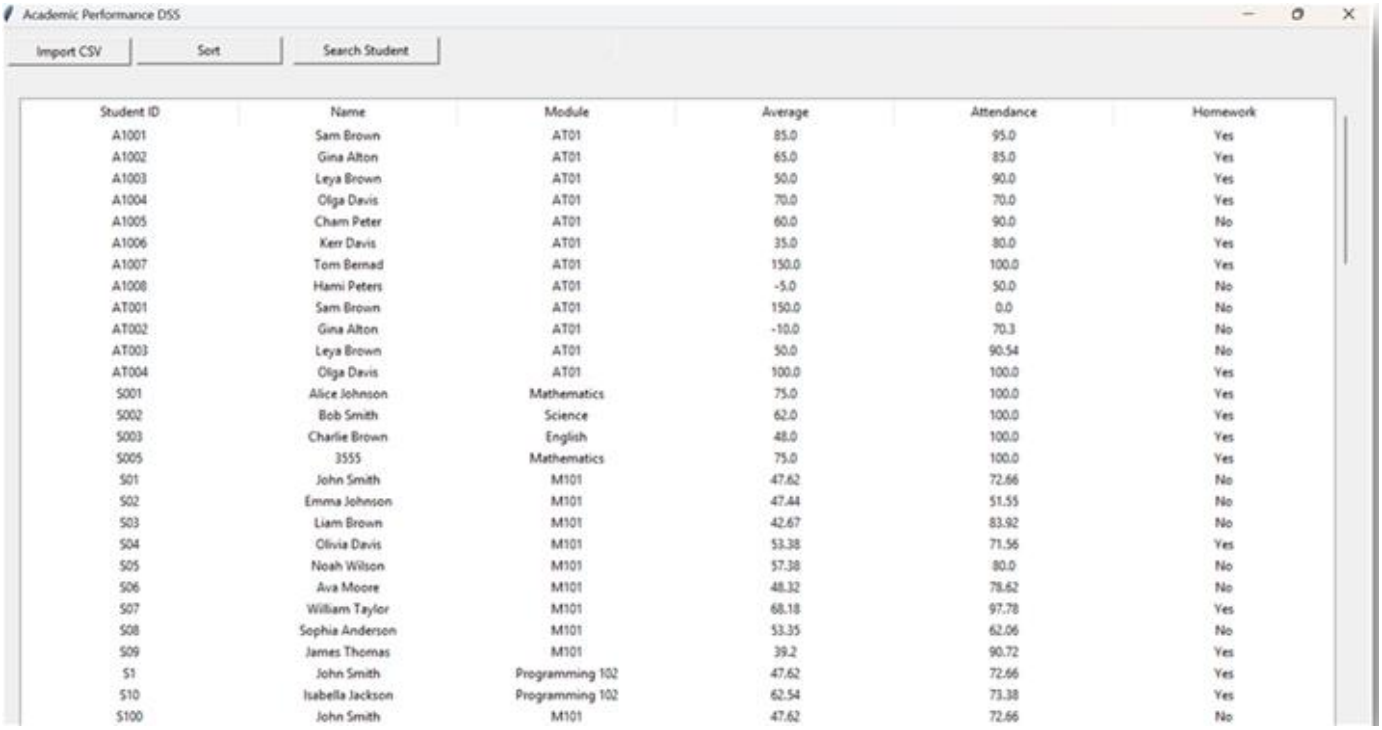


After implementing input validation and role-based output handling, all search test cases were re-executed and successfully produced the expected results.

Evaluation

The search algorithm now works reliably and securely. Input validation prevents incorrect data from being processed, while role-based access control ensures data privacy. Testing and user feedback guided improvements, resulting in a user-friendly and robust feature that integrates effectively with other decision-support components of the system.

User Interface for the system after developing iteration 6



Student ID	Name	Module	Average	Attendance	Homework
A1001	Sam Brown	AT01	85.0	95.0	Yes
A1002	Gina Alton	AT01	65.0	85.0	Yes
A1003	Leya Brown	AT01	50.0	90.0	Yes
A1004	Olga Davis	AT01	70.0	70.0	Yes
A1005	Cham Peter	AT01	60.0	90.0	No
A1006	Kerr Davis	AT01	35.0	80.0	Yes
A1007	Tom Bernad	AT01	150.0	100.0	Yes
A1008	Hami Peters	AT01	-5.0	50.0	No
AT001	Sam Brown	AT01	150.0	0.0	No
AT002	Gina Alton	AT01	-10.0	70.3	No
AT003	Leya Brown	AT01	50.0	90.54	No
AT004	Olga Davis	AT01	100.0	100.0	Yes
S001	Alice Johnson	Mathematics	75.0	100.0	Yes
S002	Bob Smith	Science	62.0	100.0	Yes
S003	Charlie Brown	English	48.0	100.0	Yes
S005	3555	Mathematics	75.0	100.0	Yes
S01	John Smith	M101	47.62	72.66	No
S02	Emma Johnson	M101	47.44	51.55	No
S03	Liam Brown	M101	42.67	83.92	No
S04	Olivia Davis	M101	53.38	71.56	Yes
S05	Noah Wilson	M101	57.38	80.0	No
S06	Ava Moore	M101	48.32	78.62	No
S07	William Taylor	M101	68.18	97.78	Yes
S08	Sophia Anderson	M101	53.35	62.06	No
S09	James Thomas	M101	39.2	90.72	Yes
S1	John Smith	Programming 102	47.62	72.66	Yes
S10	Isabella Jackson	Programming 102	62.54	73.38	Yes
S100	John Smith	M101	47.62	72.66	No

ITERATION 7 – MODULE GRADE DISTRIBUTION AND VISUAL ANALYTICS -11-03-2026

Objective

Prototype 4:
A fully integrated prototype combining decision-support logic with graphical data visualisation

The aim of this iteration was to allow teachers to visually analyse student performance within a specific module. By displaying the distribution of predicted grades in graphical form, the system supports quicker interpretation of trends and patterns, enabling more informed academic decision-making.

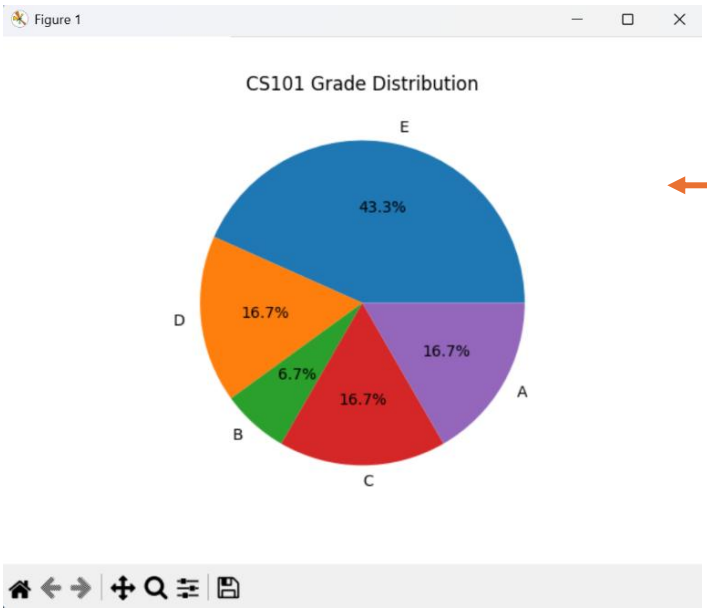
Implementation

The module grade distribution feature uses Matplotlib, a Python data visualisation library, to create a pie chart showing the predicted grades for a selected module. Matplotlib allows data to be presented graphically, making it easier to interpret patterns and trends compared to raw data. The user is prompted to enter a module name, and the system filters the relevant students in a case-insensitive way. If no students are found, a message is shown and the process stops safely.

For the matching students, each predicted grade is calculated and stored in a list. The system then counts the frequency of each grade and passes this data to Matplotlib to generate a pie chart. The chart displays the proportion and percentage of each grade, along with a clear title, providing an easy-to-understand visual summary of overall module performance. In the parent view, this graph is included in the generated PDF report to give a clear overview of the module's grade distribution.

Code snippet

```
# ----------------***MODULE DISTRIBUTION FUNCTION***-------- ----------------
def module_distribution():
    module = simpledialog.askstring("Module", "Enter Module Name")
    if not module:
        return
    module_students = [s for s in students if s.module.lower() == module.lower()]
    if not module_students:
        messagebox.showinfo("No Data", f"No students found for module: {module}")
        return
    grades = [s.predicted_grade() for s in module_students]
    grade_count = Counter(grades)
    plt.pie(grade_count.values(), labels=grade_count.keys(), autopct="%1.1f%%")
    plt.title(module + " Grade Distribution")
    plt.show()
```



Module grade distribution graph

Justification

The module grade distribution feature uses Matplotlib to generate a pie chart showing predicted grades for a selected module. Graphical visualisation is used to help users interpret data quickly, with pie charts effectively displaying grade proportions. The system prompts the user for a module name and filters student records in a case-insensitive manner, ensuring relevant results and handling cases where no data is found.

For matching students, predicted grades are calculated dynamically and their frequencies are counted using a Counter. This data is then used to create a pie chart with percentage labels and a clear title. The feature provides a quick visual summary of overall module performance, and in the parent view, the chart is included in the generated PDF report.

Testing

The following test cases were executed to verify the module distribution functionality of the system.

Test Case ID	Test Case	Input	Expected Result	Result
TC38	Valid module	M205	Pie chart displayed	Pass
TC39	Valid module (lower case)	m205	Pie chart displayed	Fail Retested and passed.
TC40	Invalid module	M999	"No students found" message	Fail Retested and passed.
TC41	Empty input	Cancel / blank	Function exits safely	Pass

Problems Encountered and Solutions

Issue 1: Case-Sensitive Module Matching

Initial testing showed that entering a module name with incorrect capitalisation resulted in no data being found.

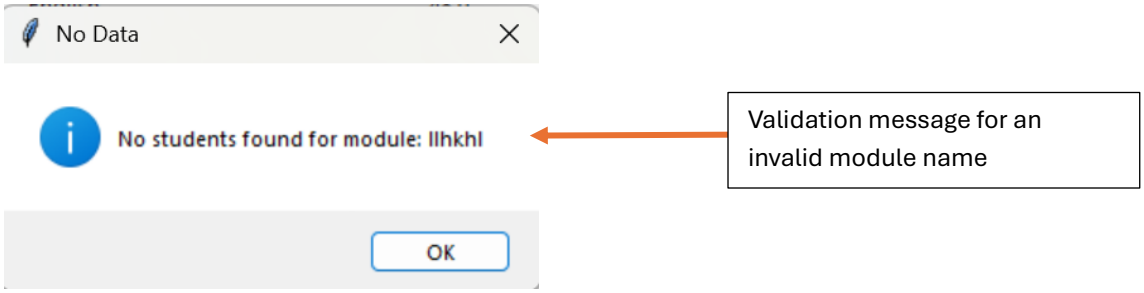
Solution:
Both the user input and student module values were converted to lower case before comparison, making the search case-insensitive and improving usability

Issue 2: No Feedback When Module Was Not Found

Early versions of the feature produced no visible response when no students were enrolled in the specified module.

Solution:

A user-friendly message was added to inform the user when no data is available for the selected module.

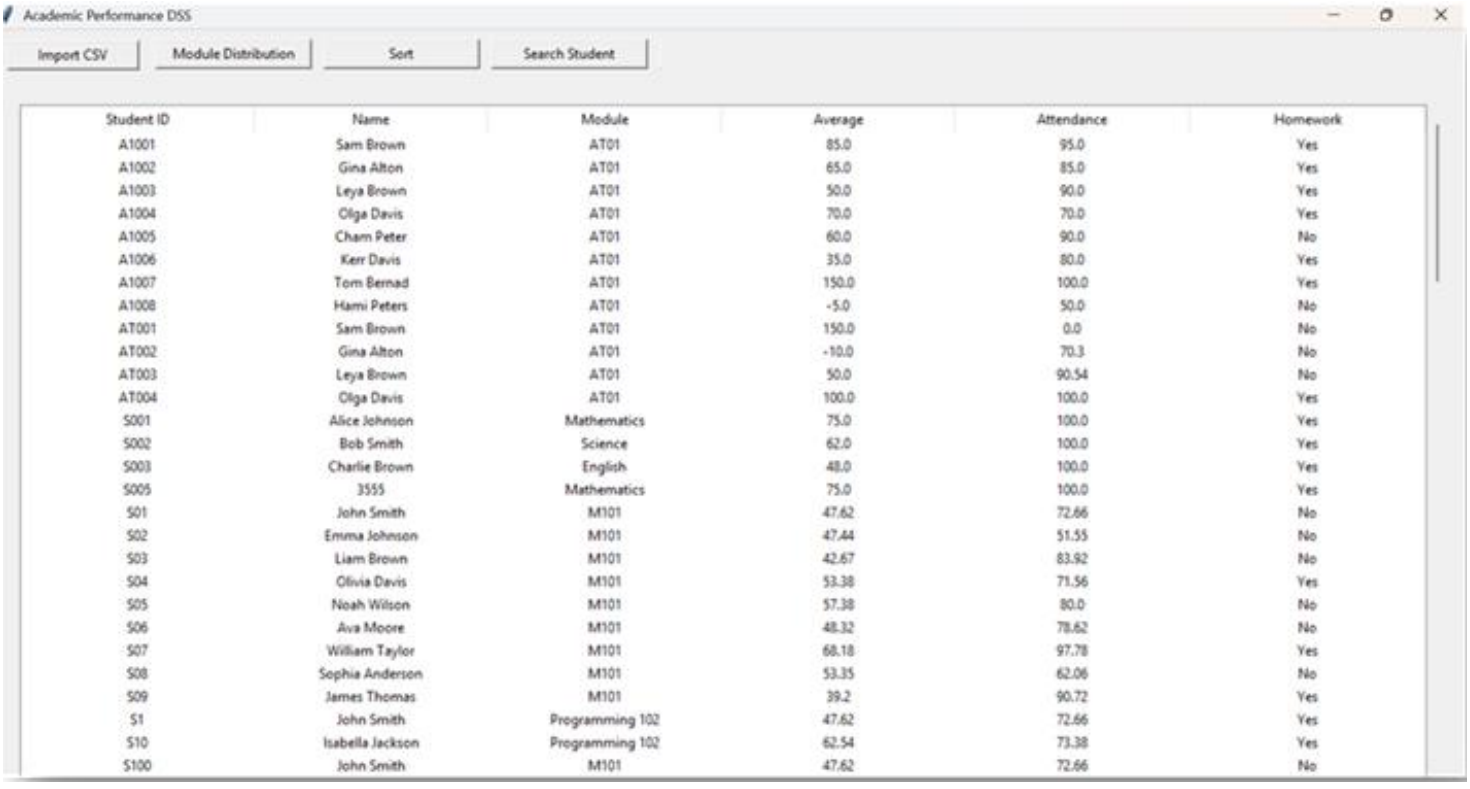


Evaluation

The module grade distribution feature successfully meets its objective by providing a clear visual representation of student performance within a module. It integrates effectively with existing analytical functions such as predicted grade calculation and supports informed decision-making. The feature enhances the system's analytical capabilities and provides a strong foundation for future developments, such as exporting charts or combining multiple modules in a single visualisation.

The feature was re-tested after introducing case-insensitive matching and user feedback messages, and all tests passed successfully.

User Interface for the system after developing iteration 7



Student ID	Name	Module	Average	Attendance	Homework
A1001	Sam Brown	AT01	85.0	95.0	Yes
A1002	Gina Alton	AT01	65.0	85.0	Yes
A1003	Leya Brown	AT01	50.0	90.0	Yes
A1004	Olga Davis	AT01	70.0	70.0	Yes
A1005	Cham Peter	AT01	60.0	90.0	No
A1006	Kerr Davis	AT01	35.0	80.0	Yes
A1007	Tom Bernad	AT01	150.0	100.0	Yes
A1008	Hami Peters	AT01	-5.0	50.0	No
AT001	Sam Brown	AT01	150.0	0.0	No
AT002	Gina Alton	AT01	-10.0	70.3	No
AT003	Leya Brown	AT01	50.0	90.54	No
AT004	Olga Davis	AT01	100.0	100.0	Yes
S001	Alice Johnson	Mathematics	75.0	100.0	Yes
S002	Bob Smith	Science	62.0	100.0	Yes
S003	Charlie Brown	English	48.0	100.0	Yes
S005	3555	Mathematics	75.0	100.0	Yes
S01	John Smith	M101	47.62	72.66	No
S02	Emma Johnson	M101	47.44	51.55	No
S03	Liam Brown	M101	42.67	83.92	No
S04	Olivia Davis	M101	53.38	71.56	Yes
S05	Noah Wilson	M101	57.38	80.0	No
S06	Ava Moore	M101	48.32	78.62	No
S07	William Taylor	M101	68.18	97.78	Yes
S08	Sophia Anderson	M101	53.35	62.06	No
S09	James Thomas	M101	39.2	90.72	Yes
S1	John Smith	Programming 102	47.62	72.66	Yes
S10	Isabella Jackson	Programming 102	62.54	73.38	Yes
S100	John Smith	M101	47.62	72.66	No

ITERATION 8 – PDF REPORT EXPORT- 11-03-2026

Objective

Prototype 4 (refined)

Prototype 4 was refined to include professional PDF report generation with embedded visual analytics.

The objective of this iteration was to allow teachers and parents to generate a detailed academic performance report for an individual student. The report combines key performance data with visual analysis in a professional PDF format that can be saved, shared, or reviewed outside the system.

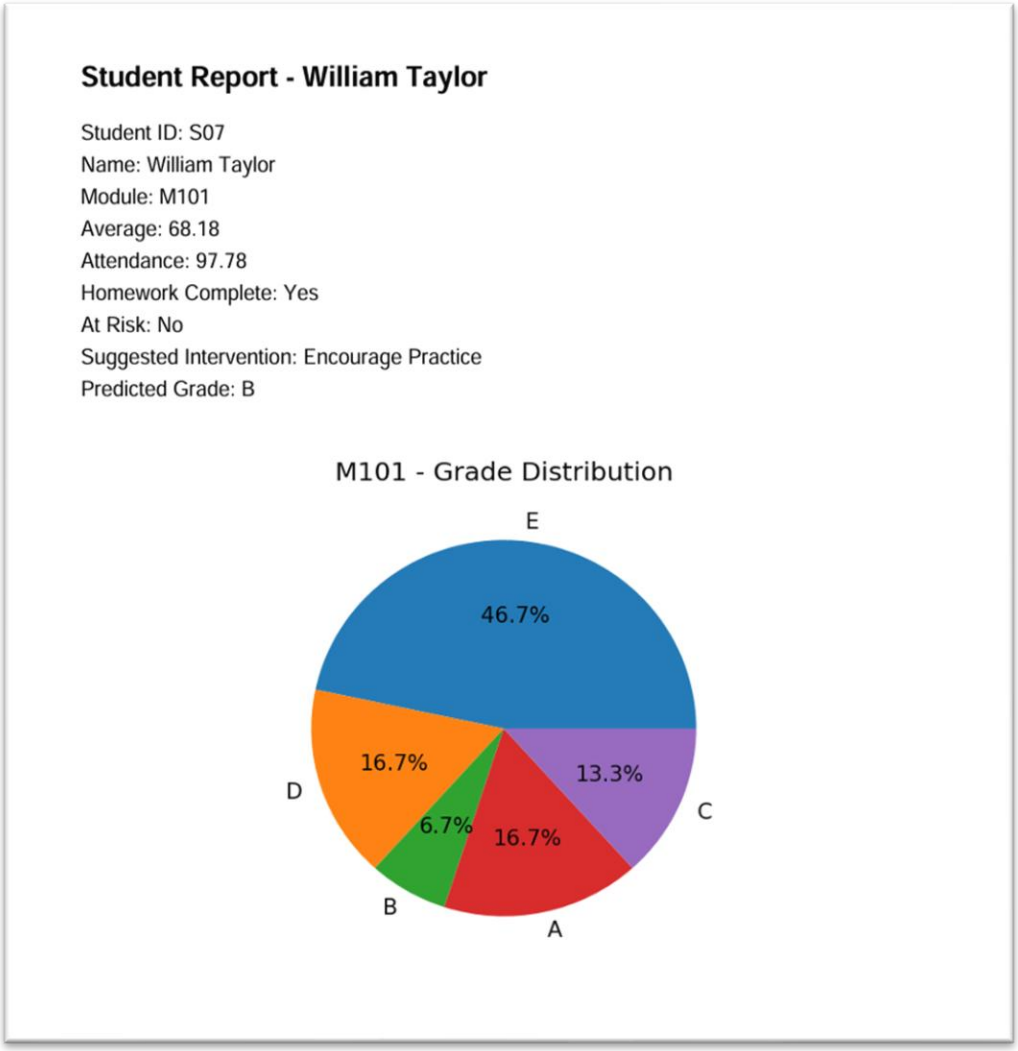
Implementation

This feature uses the ReportLab library, which allows Python to create professional PDF documents, along with Matplotlib for generating charts. When the export function is used, the system checks the user's role: parents search for their child by ID or name, while teachers select a student from the table, ensuring that sensitive data is accessed appropriately. The system collects all relevant information for the student, such as average score, attendance, homework completion, at-risk status, predicted grade, and suggested intervention. It also looks at all students in the same module to calculate the predicted grade distribution. A pie chart is then created using Matplotlib and added to the PDF, giving a clear, visual summary of student performance.

Code snippet

```
#--------------***ReportLab PDF Library User Guide. (n.d.). Available at:
https://www.reportlab.com/docs/reportlab-userguide.pdf
# ---------------- ---------***EXPORT REPORT***----------------------------- ----------------
def export_student_report():
    #----------------------***PARENT SEARCH FOR THEIR CHILD***------------------------------
    if current_role == "parent":
        term = simpledialog.askstring("Search", "Enter Student ID or Name")
        if not term:
            return
        results = search_students(students, term)
        if not results:
            messagebox.showinfo("No Result", "No student found with that ID or name.")
            return
        s = results[0]
    #----------------------***TEACHER SELECT STUDENT FROM TABLE***--------------------------
    else:
        s = get_selected_student()
        if not s:
            return

    module_students = [st for st in students if st.module == s.module]
    grades = Counter(st.predicted_grade() for st in module_students)

    #------***matplotlib.org. (n.d.). Basic pie chart — Matplotlib 3.3.4 documentation. [online]
Available at: https://matplotlib.org/stable/gallery/pie_and_polar_charts/pie_features.html.
    #----------------------***CREATE MODULE GRADE DISTRIBUTION PIE CHART***-----------------
    fig = plt.Figure(figsize=(4,4))
    ax = fig.add_subplot(111)
    ax.pie(list(grades.values()), labels=list(grades.keys()), autopct="%1.1f%%")
    ax.set_title(f"{s.module} - Grade Distribution")
    #----------------------***SAVE TEMPORARY CHART IMAGE***---------------------------------
    pie_path = "temp_pie.png"
    fig.savefig(pie_path, bbox_inches="tight", dpi=150)
    plt.close(fig)
    #----------------------***SELECT LOCATION TO SAVE PDF REPORT***-------------------------
    pdf_filename = filedialog.asksaveasfilename(defaultextension=".pdf")
    if not pdf_filename:
        os.remove(pie_path)
        return
    #--------------***ReportLab PDF Library User Guide. (n.d.). Available at:
https://www.reportlab.com/docs/reportlab-userguide.pdf
    #----------------------***CREATE PDF REPORT***-----------------------------------------
    c = canvas.Canvas(pdf_filename, pagesize=A4)
    width, height = A4
    c.setFont("Helvetica-Bold", 16)
    c.drawString(50, height - 50, f"Student Report - {s.name}")
    c.setFont("Helvetica", 12)
    y = height - 80

    #----------------------***ADD STUDENT DETAILS TO REPORT***-----------------------------
    fields = [
        ("Student ID", s.student_id),
        ("Name", s.name),
        ("Module", s.module),
        ("Average", s.average),
        ("Attendance", s.attendance),
        ("Homework Complete", "Yes" if s.homework_complete else "No"),
        ("At Risk", s.at_risk()),
        ("Suggested Intervention", s.suggested_intervention()),
        ("Predicted Grade", s.predicted_grade())
    ]
    for label, value in fields:
        c.drawString(50, y, f"{label}: {value}")
        y -= 18


    #----------------------***INSERT PIE CHART INTO PDF***---------------------------------
    y -= 10
    image_height = min(y - 40, 300)
    image_width = image_height
    x_pos = (width - image_width) / 2
    y_pos = y - image_height
    c.drawImage(pie_path, x_pos, y_pos, width=image_width, height=image_height)
    c.save()
    os.remove(pie_path)

    messagebox.showinfo("Success", "Report Exported Successfully")
```

Sample PDF Report



Test cases

The following test cases were executed to verify the PDF report generation functionality of the system.

Test Case ID	Test Case	Input	Expected Outcome	Result
TC42	Teacher report export	Teacher selects a student and clicks *Export Report*	PDF report generated successfully	Pass
TC43	Parent report export	Parent searches valid student ID/name	Correct PDF report generated	Pass

TC44	Cancel student search (parent)	Parent cancels search dialog	Cannot export PDF without a valid student id/name	Pass
TC45	Cancel save dialog	Cancel file save dialog	Export cancelled, no PDF created	Pass
TC46	Pie chart generation	Student in a module with others	Chart included in PDF	Fail Retested and passed.
TC47	Single-student module	Only one student in module	Pie chart generated correctly	Pass

Problems Encountered and Solutions

The PDF report was generated without including the module distribution pie chart.

Solution:
ReportLab cannot directly embed a Matplotlib figure object. To solve this, the pie chart was first saved as a temporary image file, which was then inserted into the PDF. After embedding, the temporary file is deleted to keep the system clean and prevent unnecessary storage use.

```
#----------------------***SAVE TEMPORARY CHART IMAGE***----------
pie_path = "temp_pie.png"
fig.savefig(pie_path, bbox_inches="tight", dpi=150)
plt.close(fig)
```

Code to save as a temporary image file

After embedding the pie chart as a temporary image file, the report generation feature was re-tested and successfully produced complete PDF reports in all scenarios.

Justification

The report generation feature uses ReportLab to create professional PDFs and Matplotlib to include a visual grade distribution. The system identifies the student based on the user's role. Parents search for their child, while teachers select from the table, ensuring sensitive data is protected. It collects key academic information, calculates predicted grades for the module, and generates a pie chart, which is embedded in the PDF.

Users choose where to save the report, and temporary files are removed after export. The PDF includes a clear title, student details, and the grade distribution chart. Using PDFs ensures formatting is preserved across devices, while combining text and visuals makes the report

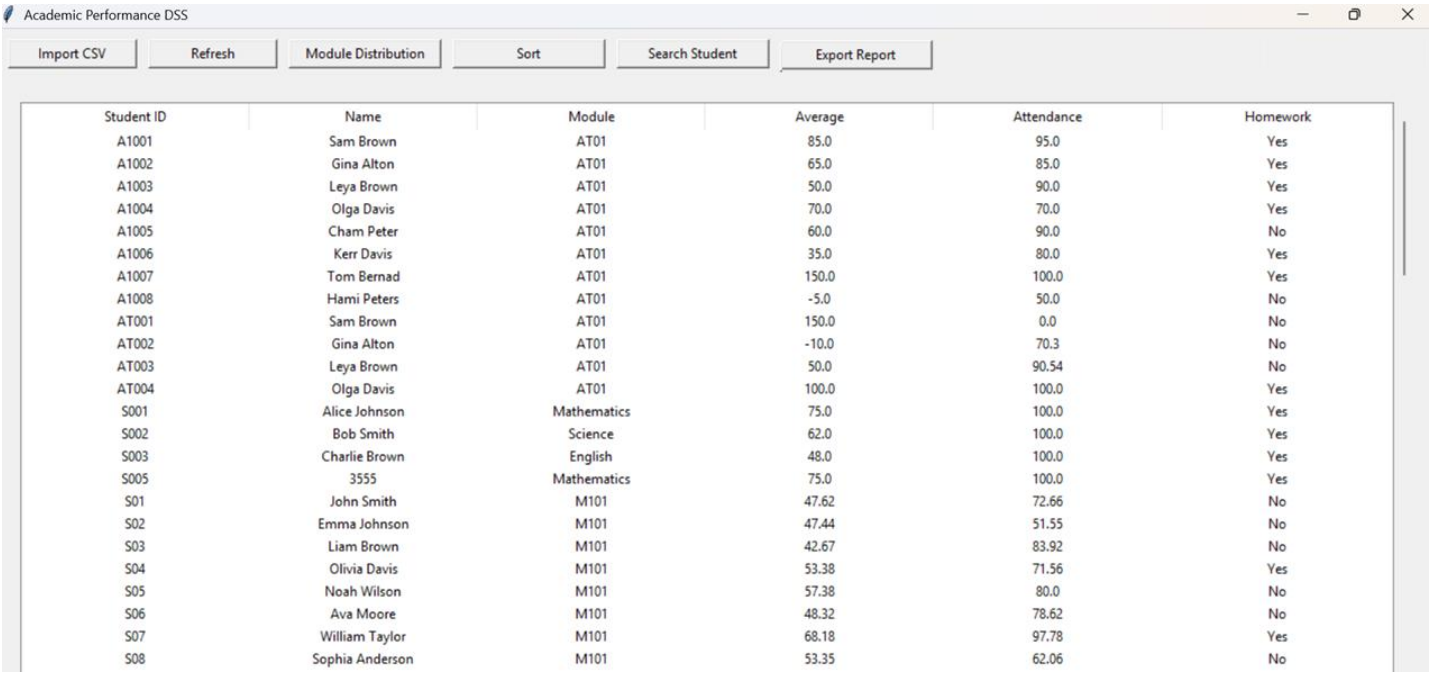
informative. Role-based access keeps data secure, and the modular use of Matplotlib and ReportLab ensures maintainability. Overall, this feature provides a clear, professional summary of academic performance, supporting the system as a decision support tool.

Alternative reporting formats such as HTML were considered, but PDF was chosen for portability and consistent formatting across devices.

Evaluation

This iteration successfully met its objective by providing a clear, well-structured student performance report. The combination of textual data and visual analysis improves understanding and supports informed decision-making. The feature integrates effectively with existing system components and enhances the overall usefulness of the application.

User Interface for the system after developing iteration 8



Student ID	Name	Module	Average	Attendance	Homework
A1001	Sam Brown	AT01	85.0	95.0	Yes
A1002	Gina Alton	AT01	65.0	85.0	Yes
A1003	Leya Brown	AT01	50.0	90.0	Yes
A1004	Olga Davis	AT01	70.0	70.0	Yes
A1005	Cham Peter	AT01	60.0	90.0	No
A1006	Kerr Davis	AT01	35.0	80.0	Yes
A1007	Tom Bernad	AT01	150.0	100.0	Yes
A1008	Hami Peters	AT01	-5.0	50.0	No
AT001	Sam Brown	AT01	150.0	0.0	No
AT002	Gina Alton	AT01	-10.0	70.3	No
AT003	Leya Brown	AT01	50.0	90.54	No
AT004	Olga Davis	AT01	100.0	100.0	Yes
S001	Alice Johnson	Mathematics	75.0	100.0	Yes
S002	Bob Smith	Science	62.0	100.0	Yes
S003	Charlie Brown	English	48.0	100.0	Yes
S005	3555	Mathematics	75.0	100.0	Yes
S01	John Smith	M101	47.62	72.66	No
S02	Emma Johnson	M101	47.44	51.55	No
S03	Liam Brown	M101	42.67	83.92	No
S04	Olivia Davis	M101	53.38	71.56	Yes
S05	Noah Wilson	M101	57.38	80.0	No
S06	Ava Moore	M101	48.32	78.62	No
S07	William Taylor	M101	68.18	97.78	Yes
S08	Sophia Anderson	M101	53.35	62.06	No

ITERATION 9 – LOGIN SYSTEM AND ROLE-BASED USER INTERFACE-15-03-2026

Objective

Final Prototype (Prototype 4)
A complete, role-secured academic performance decision support system ready for deployment.

The objective of this iteration was to control access to the system by introducing a login mechanism and to adapt the user interface according to the role of the logged-in user. This ensures that teachers and parents are presented with appropriate functionality and that sensitive student data is protected through role-based access control.

Implementation

The login system was implemented using a simple authentication mechanism based on predefined user credentials. When a user attempts to log in, the system retrieves the username and password entered in the login fields and checks them against stored user records.

If the entered credentials are valid, the user's role is determined and stored in a global variable. A confirmation message is displayed, the login window is closed, and the main application window is launched. The main interface is then built dynamically based on the user's role.

If the credentials are invalid, an error message is shown, and access to the system is denied. This prevents unauthorised users from accessing the application.

A reset function was also implemented to allow users to quickly clear the login fields. This improves usability and reduces frustration if incorrect details are entered.

Role-Based Interface Construction

Once a user logs in successfully, the build_main_window function creates the main interface and displays features based on their role. Teachers have access to administrative tools such as importing CSV files, refreshing data, sorting records, viewing module grade distributions, and accessing full student tables. Parents have restricted access, allowing them to search for a student and view performance-related information while protecting data privacy.

All users can search for students, view at-risk status, predicted grades, suggested interventions, and export reports. The main student table, displayed only for teachers, uses a scrollable TreeView to ensure easy navigation of large datasets.

Code snippets

```
# ----------------***LOGIN FUNCTION***--------------------- ----------------
def login():
    global current_role
    username = username_entry.get()
    password = password_entry.get()
    if username in USERS and USERS[username]["password"] == password:
        current_role = USERS[username]["role"]
        messagebox.showinfo("Login Successful", f"Welcome! Role: {current_role}")
        login_window.destroy()
        build_main_window(current_role)
    else:
        messagebox.showerror("Login Failed", "Invalid username or password")

def reset_fields():
    username_entry.delete(0, tk.END)
    password_entry.delete(0, tk.END)
```

Verifies user credentials, assigns the appropriate role, and opens the main interface upon successful login.

Clears the username and password input fields to allow the user to re-enter credentials.

Justification:

The login feature ensures that only authorised users can access the system. When a user enters their username and password, the login() function checks these against stored records and assigns a role, which determines what features they can see and use. This keeps sensitive student data secure and ensures teachers and parents only access information relevant to them.

The reset_fields() function lets users quickly clear the input fields, making the login process easier and more user-friendly. Using Tkinter provides a simple, intuitive interface, and keeping the login logic separate from the main interface makes the system easier to maintain and extend in the future.

Testing

The login and role-based interface functionality were tested using a range of scenarios.

Test Case ID	Test Case	Input	Expected Result	Result
TC48	Valid teacher login	Correct credentials Username: teacher Password: teacher123	Access granted, full interface shown	Pass
TC49	Valid parent login	Correct credentials Username: parent Password: parent123	Access granted, restricted interface shown	Pass

TC50	Incorrect password	Wrong password for parent/teacher	Error message displayed	Pass
TC51	Unknown username	Invalid username for parent/teacher	Error message displayed	Pass
TC52	Empty fields	No input	Login denied	Pass
TC53	Reset fields	Click reset	Fields cleared	Pass

Problems Encountered and Solutions

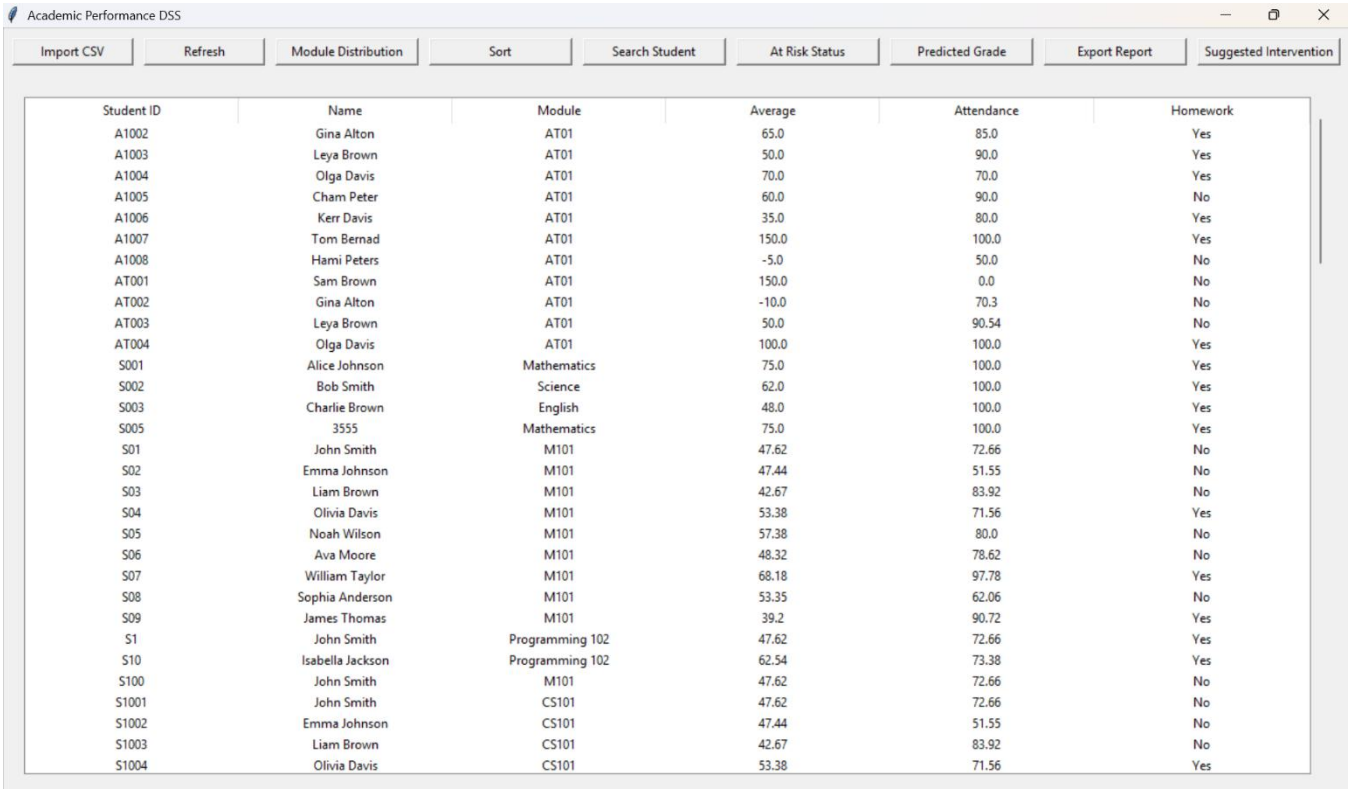
Issue:
Initially, all users were shown the same interface regardless of their role.

Solution:
The interface was modified to dynamically display features based on the logged-in user's role. Administrative tools and full data views were restricted to teachers only.

Evaluation

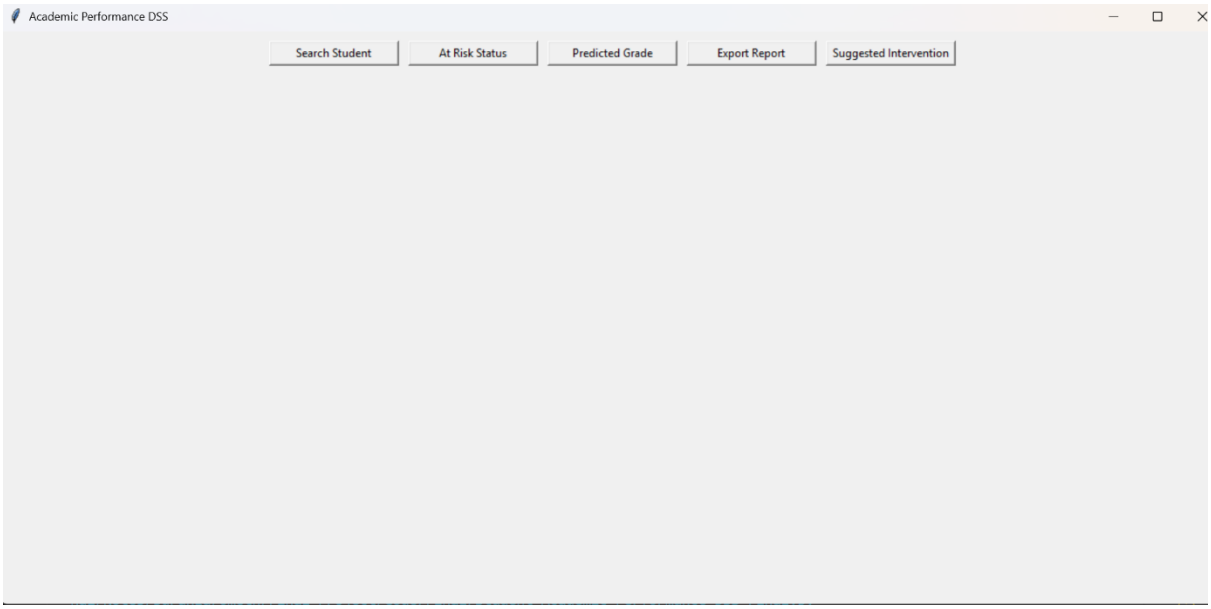
This iteration successfully introduced secure access and role-based user interaction. By combining authentication with a dynamically constructed interface, the system ensures usability, security, and ethical handling of student data. The solution integrates smoothly with existing components and provides a strong foundation for future improvements, such as enhanced authentication or additional user roles.

User Interface for the system (Teacher view) after developing iteration 9



Student ID	Name	Module	Average	Attendance	Homework
A1002	Gina Alton	AT01	65.0	85.0	Yes
A1003	Leya Brown	AT01	50.0	90.0	Yes
A1004	Olga Davis	AT01	70.0	70.0	Yes
A1005	Cham Peter	AT01	60.0	90.0	No
A1006	Kerr Davis	AT01	35.0	80.0	Yes
A1007	Tom Bernad	AT01	150.0	100.0	Yes
A1008	Hami Peters	AT01	-5.0	50.0	No
AT001	Sam Brown	AT01	150.0	0.0	No
AT002	Gina Alton	AT01	-10.0	70.3	No
AT003	Leya Brown	AT01	50.0	90.54	No
AT004	Olga Davis	AT01	100.0	100.0	Yes
S001	Alice Johnson	Mathematics	75.0	100.0	Yes
S002	Bob Smith	Science	62.0	100.0	Yes
S003	Charlie Brown	English	48.0	100.0	Yes
S005	3555	Mathematics	75.0	100.0	Yes
S01	John Smith	M101	47.62	72.66	No
S02	Emma Johnson	M101	47.44	51.55	No
S03	Liam Brown	M101	42.67	83.92	No
S04	Olivia Davis	M101	53.38	71.56	Yes
S05	Noah Wilson	M101	57.38	80.0	No
S06	Ava Moore	M101	48.32	78.62	No
S07	William Taylor	M101	68.18	97.78	Yes
S08	Sophia Anderson	M101	53.35	62.06	No
S09	James Thomas	M101	39.2	90.72	Yes
S1	John Smith	Programming 102	47.62	72.66	Yes
S10	Isabella Jackson	Programming 102	62.54	73.38	Yes
S100	John Smith	M101	47.62	72.66	No
S1001	John Smith	CS101	47.62	72.66	No
S1002	Emma Johnson	CS101	47.44	51.55	No
S1003	Liam Brown	CS101	42.67	83.92	No
S1004	Olivia Davis	CS101	53.38	71.56	Yes

User Interface for the system (parent view) after developing iteration 9



EVALUATION CHAPTER

INTRODUCTION

The purpose of this evaluation is to assess the effectiveness of the Student Performance Decision Support System (DSS) in meeting its intended objectives and success criteria defined during the analysis stage. The system was designed to improve the efficiency, accuracy, and usability of student performance monitoring for teachers and parents.

This evaluation uses test evidence, user feedback, and success criteria to determine whether the system has been successful. It also considers usability, limitations, maintenance, and potential future improvements.

POST-DEVELOPMENT TESTING

Extensive post-development testing was carried out to ensure that the system functions correctly and handles errors effectively.

AUTHENTICATION AND ROLE-BASED ACCESS

Post-development testing confirmed that user authentication and role allocation behave correctly under normal and invalid conditions.

SUCCESSFUL TEACHER LOGIN SHOWING FULL DASHBOARD

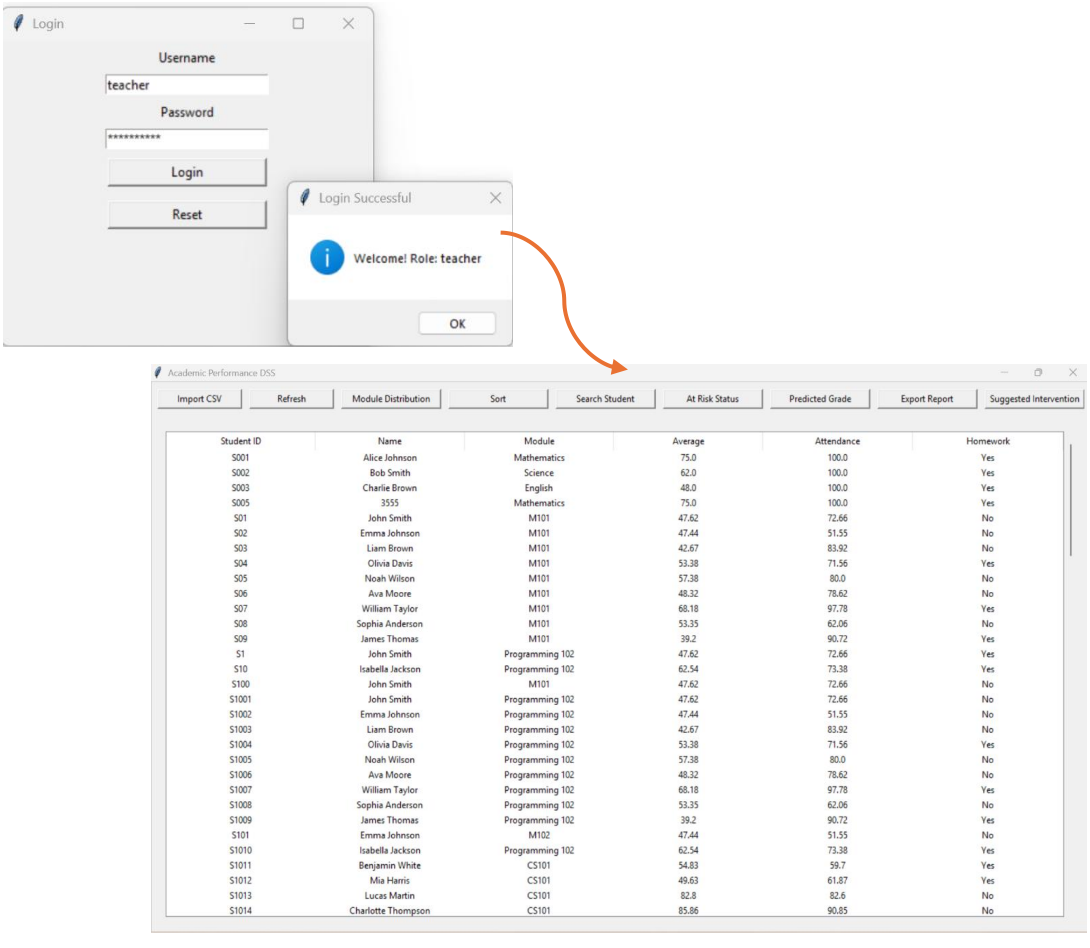


Figure 1

The Figure 1 demonstrates successful teacher authentication. The full dashboard is visible, confirming that valid credentials correctly assign the teacher role, satisfying success criteria SC1 identified in the analysis stage.

PARENT DASHBOARD (RESTRICTED VIEW)

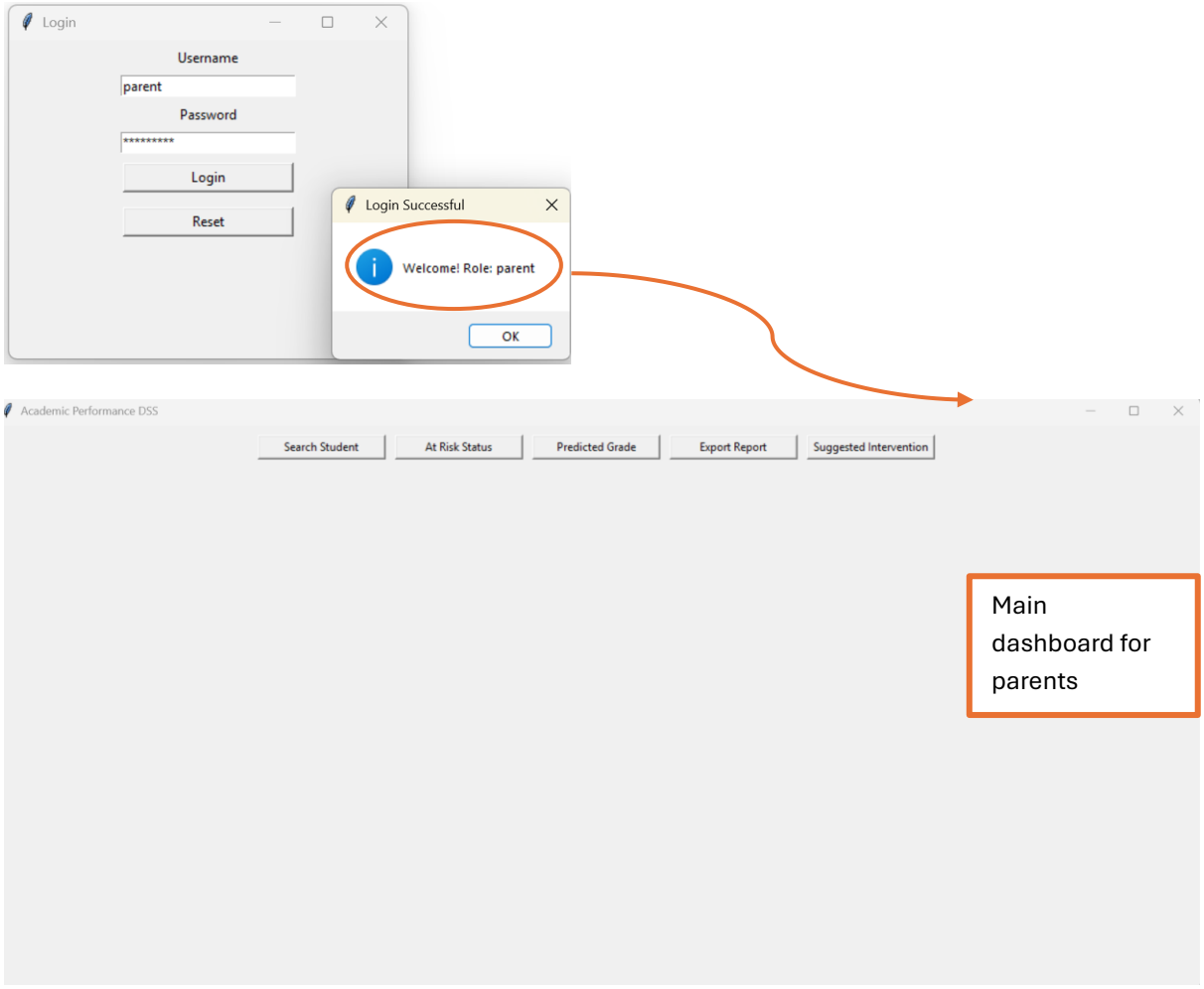


Figure2

Figure2 shows the restricted parent interface, where administrative features are hidden. This confirms that role-based access control is enforced immediately after successful login, satisfying success criteria SC2 identified in the analysis stage.

INVALID LOGIN ERROR MESSAGE

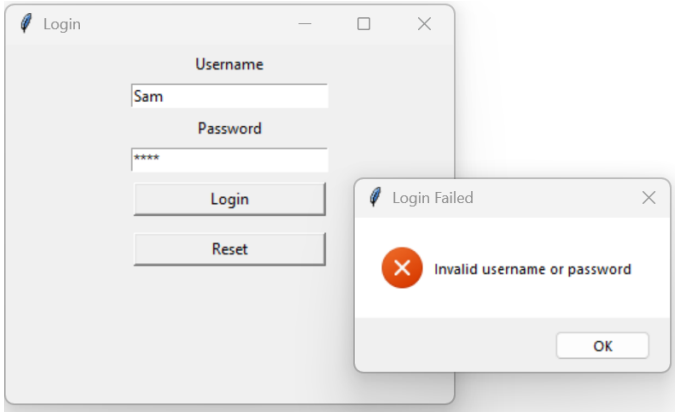


Figure3

Figure3 shows an error message displayed for invalid credentials. This confirms that unauthorised access is prevented, demonstrating secure authentication.

All authentication and access control tests behaved as expected in every case; therefore, success criteria identified in the analysis stage, SC1 and SC2 are fully met.

CSV IMPORT, VALIDATION, AND ROBUSTNESS

Post-development testing verified the system's ability to import valid data and handle invalid or malformed CSV files safely.

SUCCESSFUL CSV IMPORT POPULATING THE TABLE

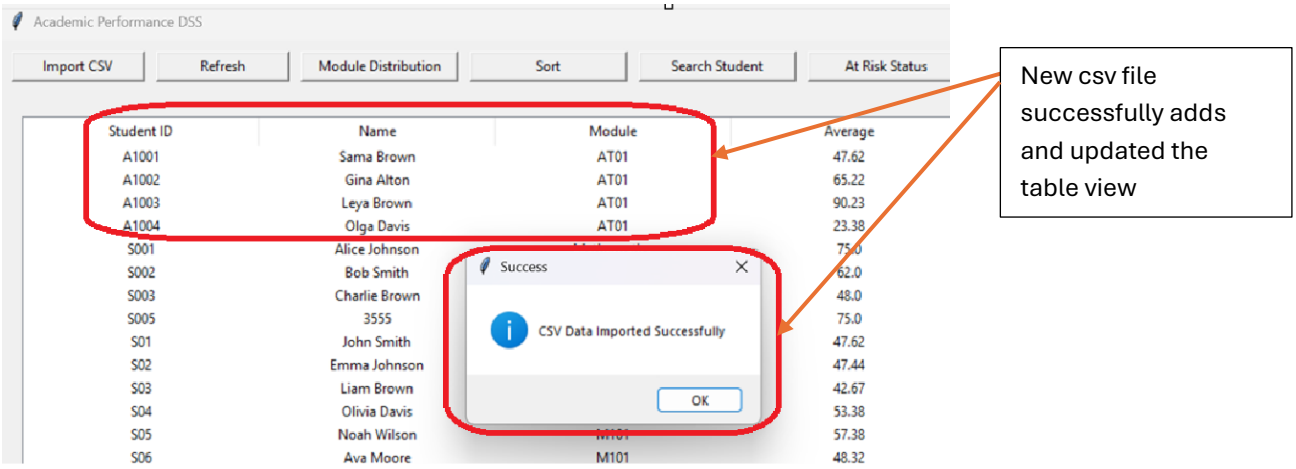


Figure 4

Figure 4 shows successful import of a valid CSV file, confirming that student records are stored and displayed correctly, meeting success criteria (SC3) identified in the analysis stage.

VALIDATION MESSAGE FOR MISSING CSV COLUMN

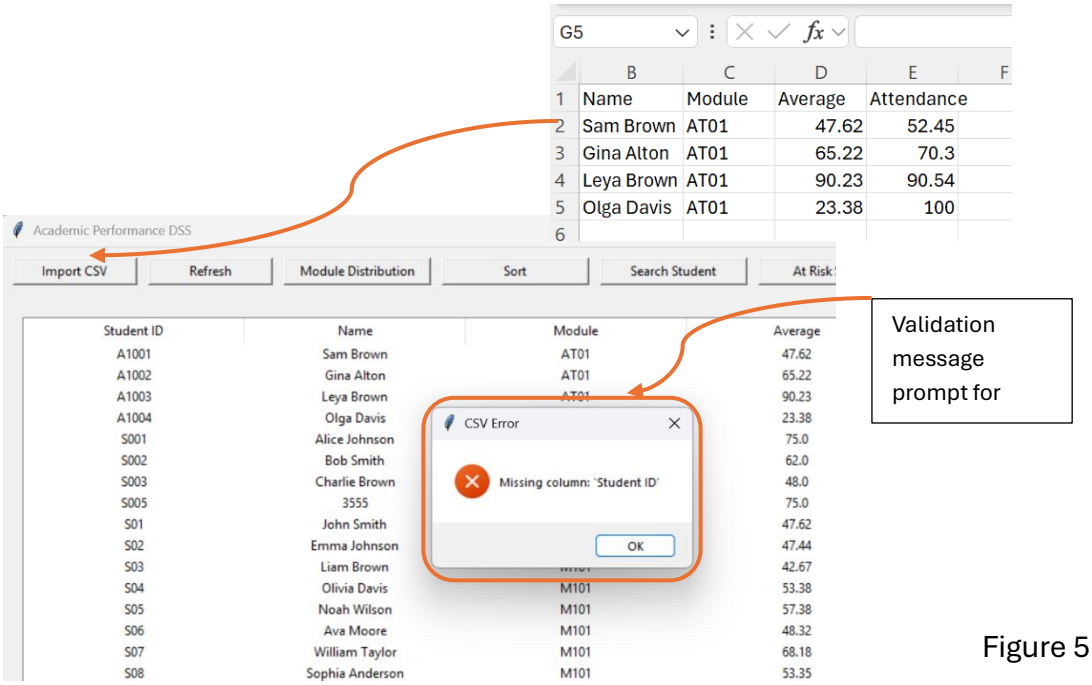


Figure 5

Figure 5 demonstrates the system detecting missing CSV columns and displaying a warning without crashing, confirming robust error handling, meeting success criteria (SC11) identified in the analysis stage.

VALIDATION MESSAGE FOR INVALID DATA TYPE

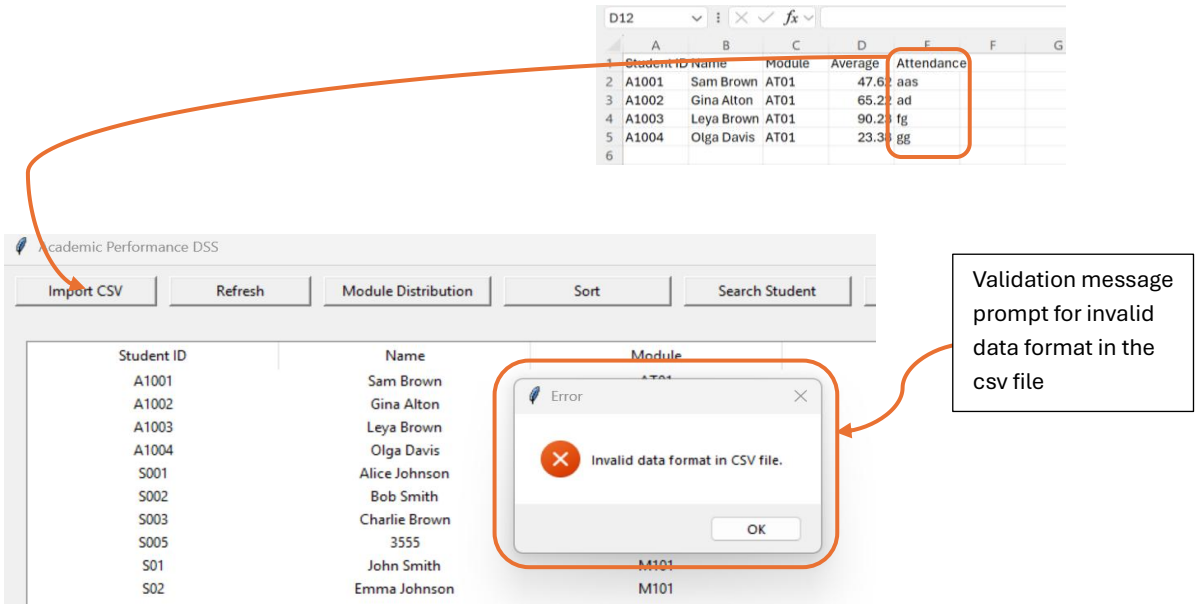


Figure 6

Figure 6 shows the system rejecting invalid numeric input in a CSV file, preventing incorrect data from being processed.

The system handled all invalid CSV scenarios safely and remained stable. Therefore, success criteria (SC3 and SC11) identified in the analysis stage are fully met.

SEARCH FUNCTIONALITY AND ACCESS CONTROL

Search testing confirmed that students can be located accurately using valid identifiers, while invalid input is handled safely.

VALID STUDENT ID SEARCH RESULT

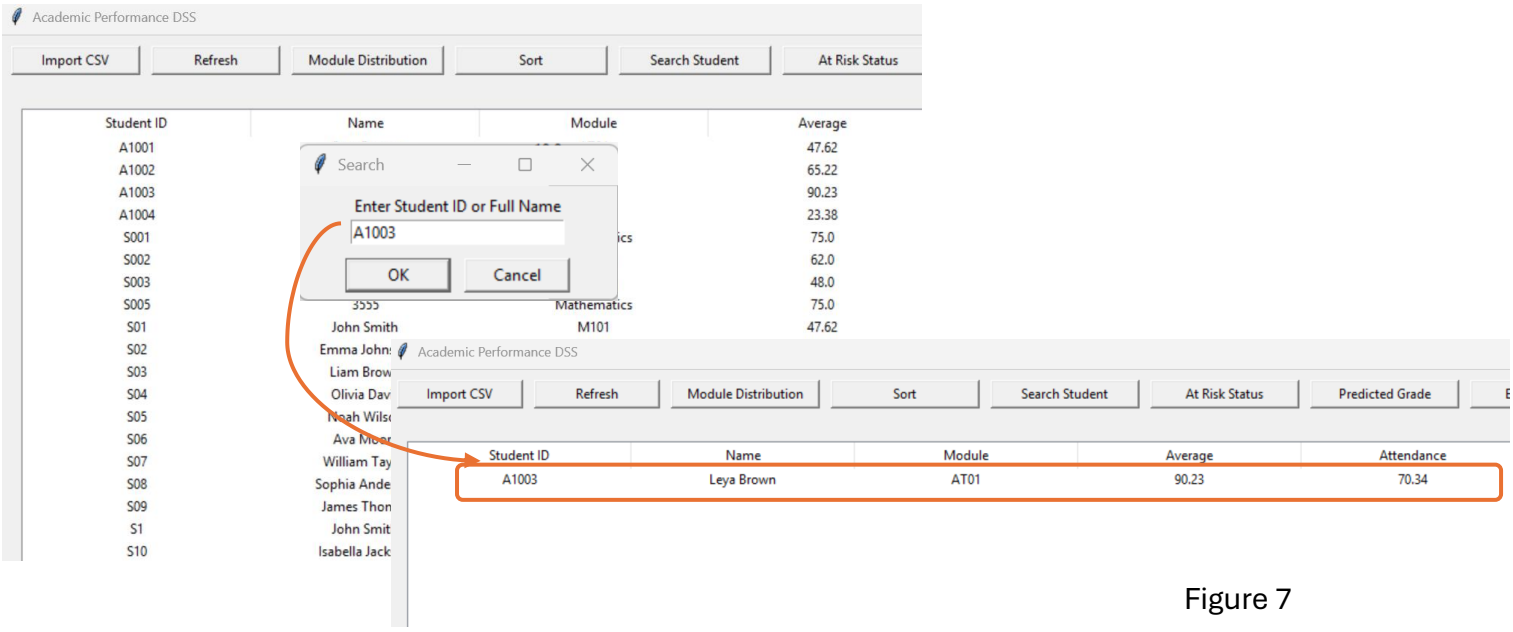


Figure 7

Figure 7 shows the table refreshing with matching results for a valid student ID, confirming accurate search functionality; the identified success criterion (SC4) in the analysis stage is fully met.

PARENT SEARCH RESULT SHOWN AS POPUP ONLY

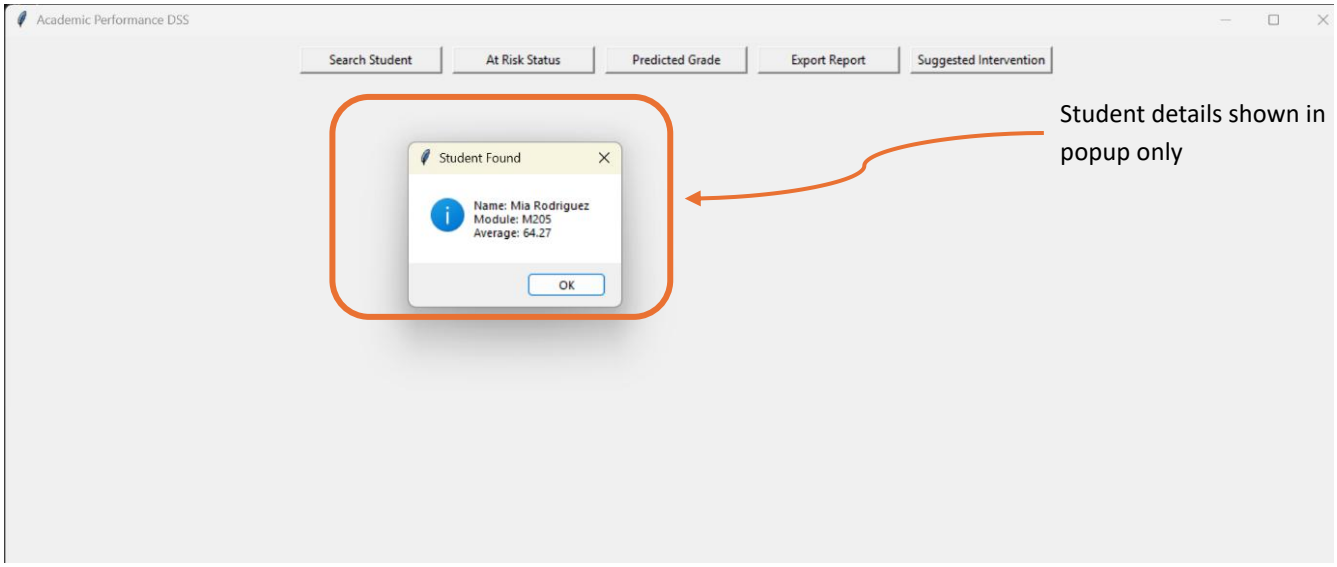


Figure 8

Figure 8 demonstrates that parent searches display student details in a popup only, confirming that access to full datasets is restricted by role, identified success criterion (SC2) in the analysis stage is fully met.

"NO STUDENT FOUND" / VALIDATION MESSAGE

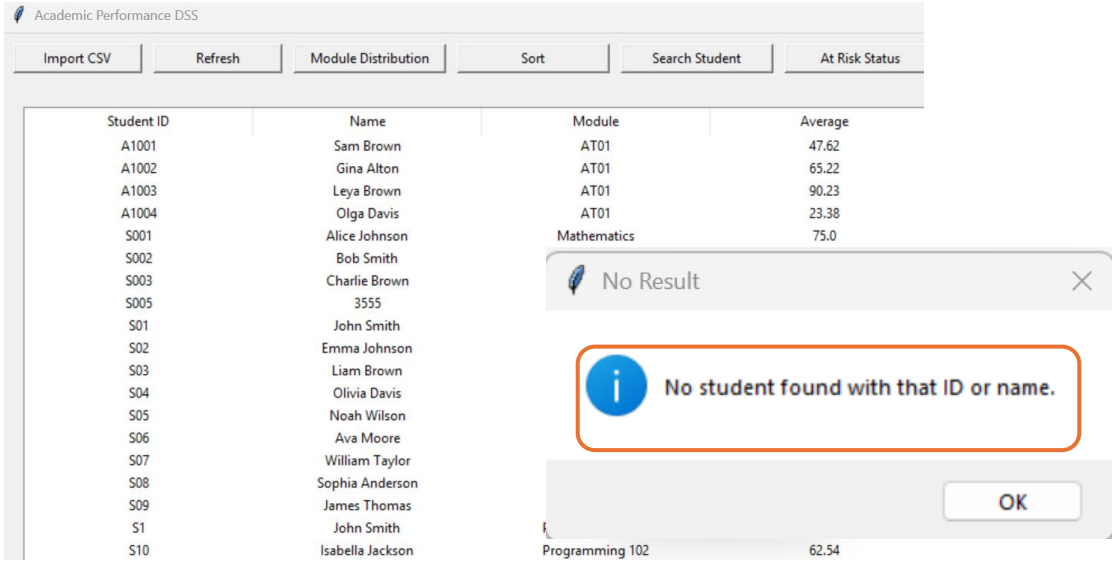


Figure 9

Figure 9 confirms that invalid or empty search input is detected and handled safely without processing, demonstrating robustness; the identified success criterion (SC11) in the analysis stage is fully met.

Search functionality was reliable, role-aware, and robust, therefore success criteria (SC4, SC2, and SC11) are fully met.

SORTING FUNCTIONALITY

Sorting was tested to ensure users can organise data effectively while maintaining access restrictions.

TEACHER VIEW SORTED ASCENDING/DESCENDING

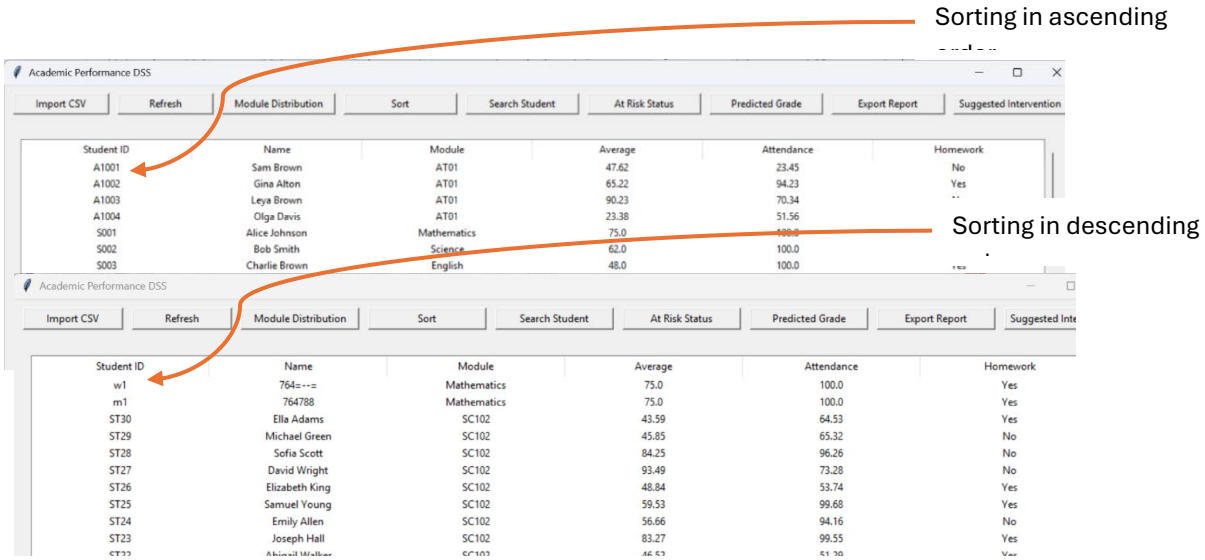


Figure 10

Figure 10 shows student records correctly sorted in ascending and descending order in the teacher view, confirming that the system meets the sorting requirement; the identified success criterion (SC10) in the analysis stage is fully met.

PARENT VIEW WITHOUT SORT BUTTON

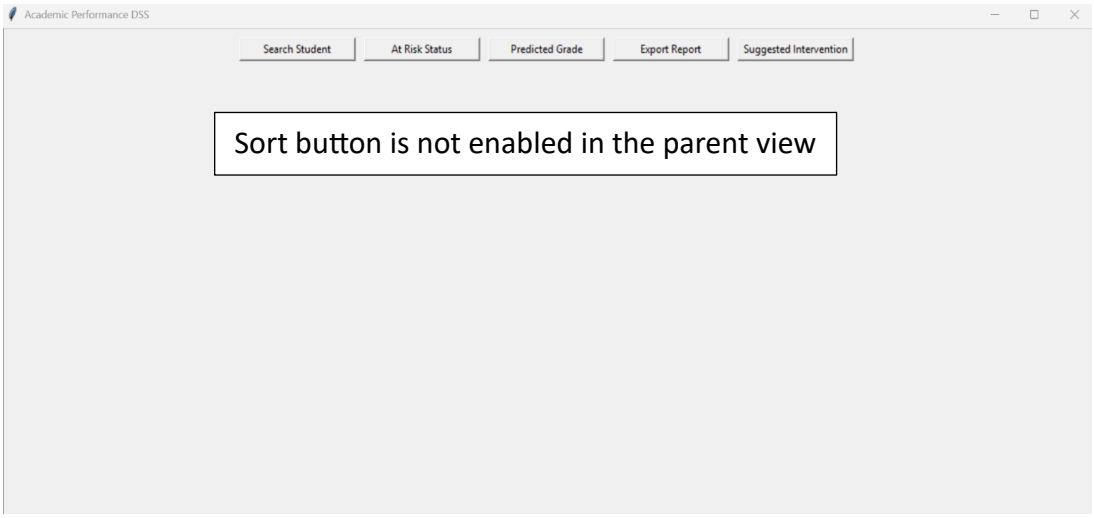


Figure11

Figure 11 shows the absence of the sort button in the parent view, confirming that restricted users cannot modify data organisation.

Overall, sorting behaved exactly as required from the user perspective; therefore, successful criterion (SC10) is fully met.

AT-RISK IDENTIFICATION AND BOUNDARY TESTING

STUDENT BELOW THRESHOLD FLAGGED AT RISK

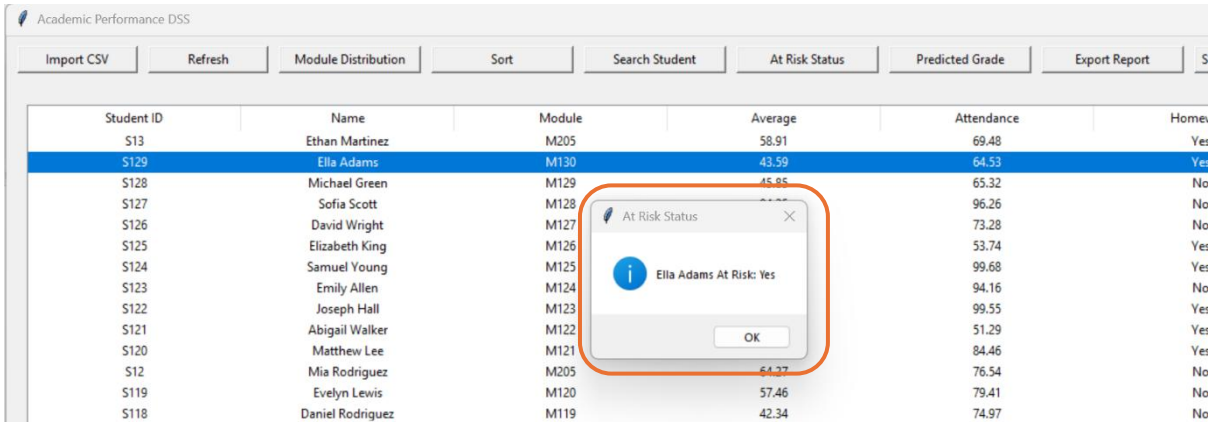


Figure 12

Figure 12 shows a student with an average below 50% correctly flagged as at risk, confirming accurate classification; the identified success criterion (SC5) in the analysis stage is fully met.

BOUNDARY VALUE AT 50%

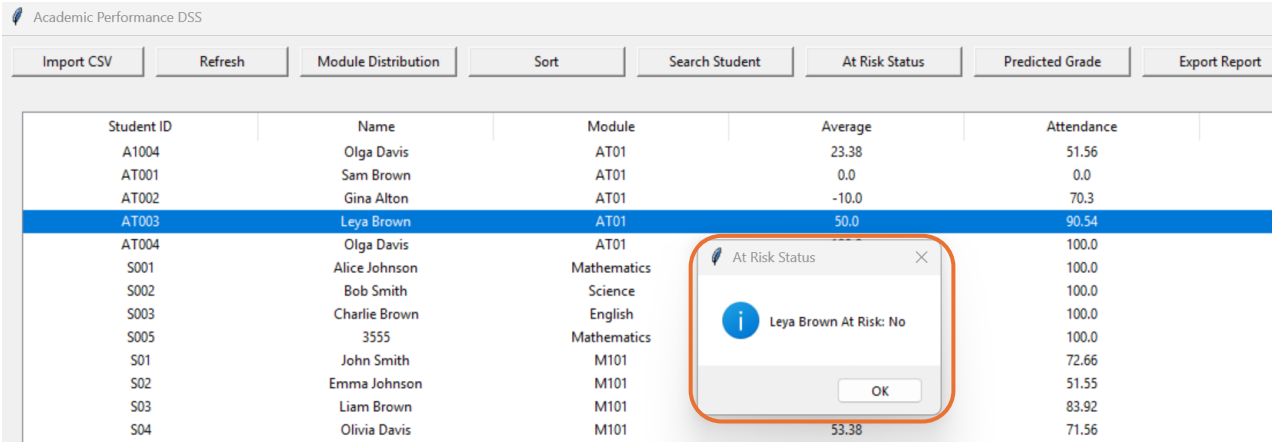


Figure 13

Figure 13 demonstrates correct handling of the boundary value, where a student scoring 50% is not flagged as at risk.

INVALID VALUE ADJUSTED

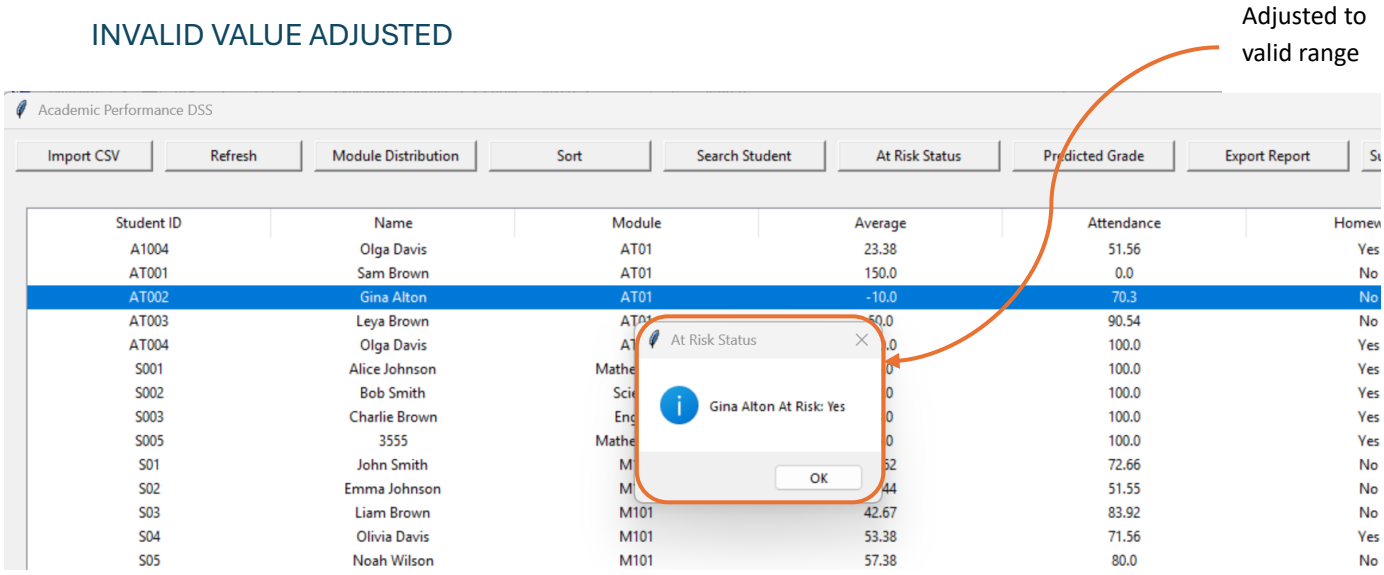


Figure 14

Figure 13 shows invalid input being adjusted to a valid range, preventing incorrect risk evaluation and demonstrating robustness; the identified success criterion (SC11) in the analysis stage is fully met.

Boundary and validation testing confirmed reliable classification and stability; therefore, SC5 and SC11 are fully met.

EXPORT REPORT FUNCTION

Post-development testing was carried out to verify that the Export Report function generates a complete and accurate PDF report for both teachers and parents while enforcing role-based access control.

For teachers, testing confirmed that a report can only be generated once a student is selected from the table. If no student is selected, the system displays a warning message and prevents the operation from continuing. This ensures that reports are not produced with missing or incorrect data. For parents, testing confirmed that the system requires a valid student ID or name before generating a report, and that parents can only generate reports for their own child.

SUCCESSFUL PDF EXPORT FOR A SELECTED STUDENT

Student Report - Mia Rodriguez

Student ID: S12
Name: Mia Rodriguez
Module: M205
Average: 64.27
Attendance: 76.54
Homework Complete: No
At Risk: No
Suggested Intervention: Encourage Practice
Predicted Grade: C

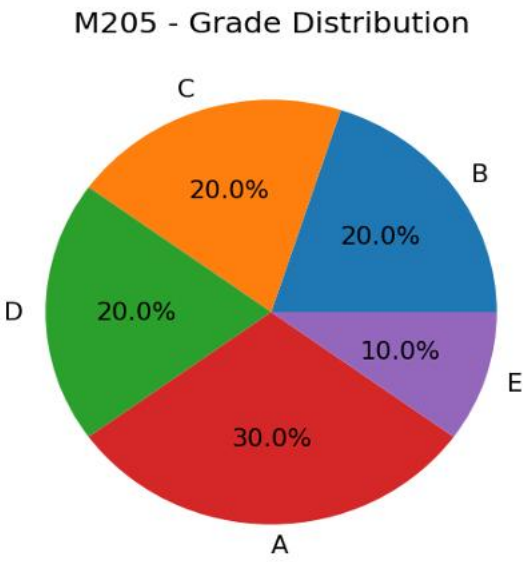


Figure 15

Figure 15 shows a successfully generated PDF report containing student details, at-risk status, predicted grade, suggested intervention, homework completion, attendance and a module grade distribution chart. This confirms that the Export Report function produces a complete and accurate report, meeting the success criterion (SC9) identified in the analysis stage.

During testing, invalid actions such as cancelling the save dialog or attempting to export without selecting or searching for a student were handled safely, with appropriate error messages displayed and no system crashes observed.

The Export Report function consistently generated correct PDF reports, enforced role-based restrictions, and handled invalid inputs robustly. Therefore, SC9 (report generation), SC2 (role-based access), and SC11 (robustness) are fully met.

SUGGESTED INTERVENTION FUNCTION

Post-development testing confirmed that the Suggested Intervention function correctly displays appropriate intervention recommendations based on a student's average score.

Testing showed that:

- High-performing students receive "None"
- Medium-performing students receive "Encourage Practice" or "Extra Tutoring"
- Low-performing students receive "Immediate Intervention"

For teachers, the intervention is displayed for the currently selected student in the table. For parents, the system requires a valid student ID or name before displaying the intervention for their child. Attempts to access the intervention feature without selecting or searching for a student resulted in validation error messages, preventing incorrect processing.

SUGGESTED INTERVENTION DISPLAYED FOR A STUDENT

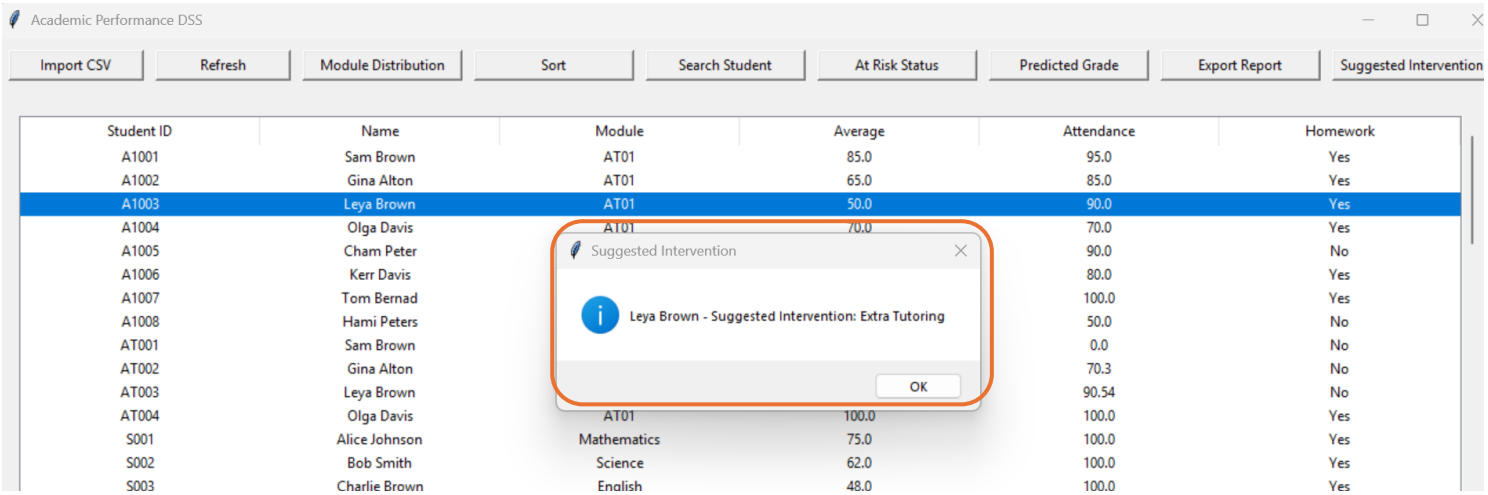


Figure 16

Figure 16 shows an intervention recommendation generated based on the student's performance. This confirms that the system applies the defined decision rules correctly and supports informed decision-making, meeting the success criterion (SC7) identified in the analysis stage.

The Suggested Intervention function behaved consistently with predefined rules, enforced role-based access, and safely handled invalid inputs. Therefore, SC7 (intervention generation), SC2, and SC11 are fully met.

MODULE GRADE DISTRIBUTION CHART

Post-development testing was conducted to verify that the Module Grade Distribution function accurately generates visual summaries of predicted grades.

Testing confirmed that when a valid module name is entered, the system correctly:

- Filters students belonging to the selected module
- Calculates predicted grades for each student
- Displays a pie chart showing the distribution of grades

If an invalid module name is entered or no students are found for the specified module, the system displays an informative message and exits safely without error.

MODULE GRADE DISTRIBUTION PIE CHART

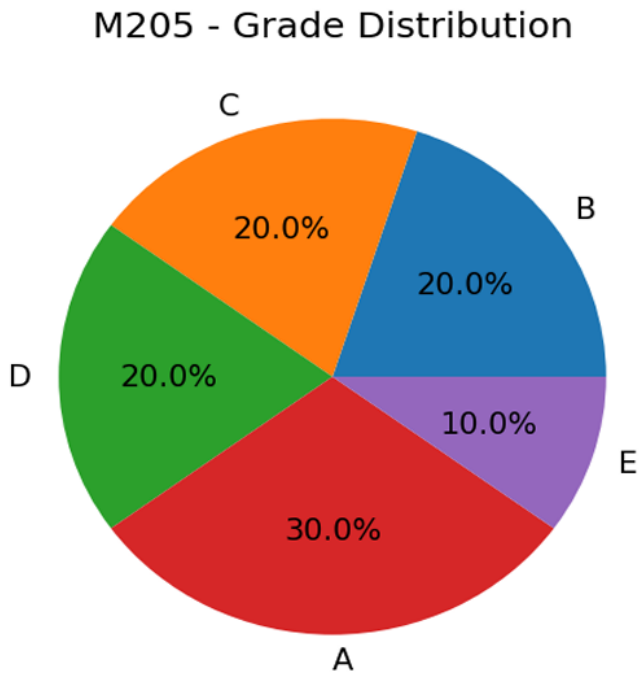


Figure 17

Figure 17 shows a correctly generated pie chart displaying predicted grade distribution for a selected module. This confirms accurate data aggregation and effective visualisation, meeting the success criterion (SC8) identified in the analysis stage.

The same chart is reused in the PDF report, confirming consistency between on-screen analysis and report generation.

The module distribution chart function produced accurate visualisations and handled invalid input safely. Therefore, SC8 (visualisation accuracy) and SC11 (robustness) are fully met.

USABILITY TESTING

Usability testing was carried out to assess whether both teachers and parents could use the system effectively, intuitively, and without the need for additional instructions or training. Testing focused on key usability principles including learnability, clarity of layout, feedback, error prevention, and role-based interface simplification. The outcomes of this testing are supported by annotated visual evidence.

DASHBOARD LAYOUT AND LEARNABILITY

The main dashboard was tested to determine whether users could immediately recognise available functions and complete common tasks such as searching for students, checking performance indicators, and generating reports.

CLEAN TEACHER DASHBOARD

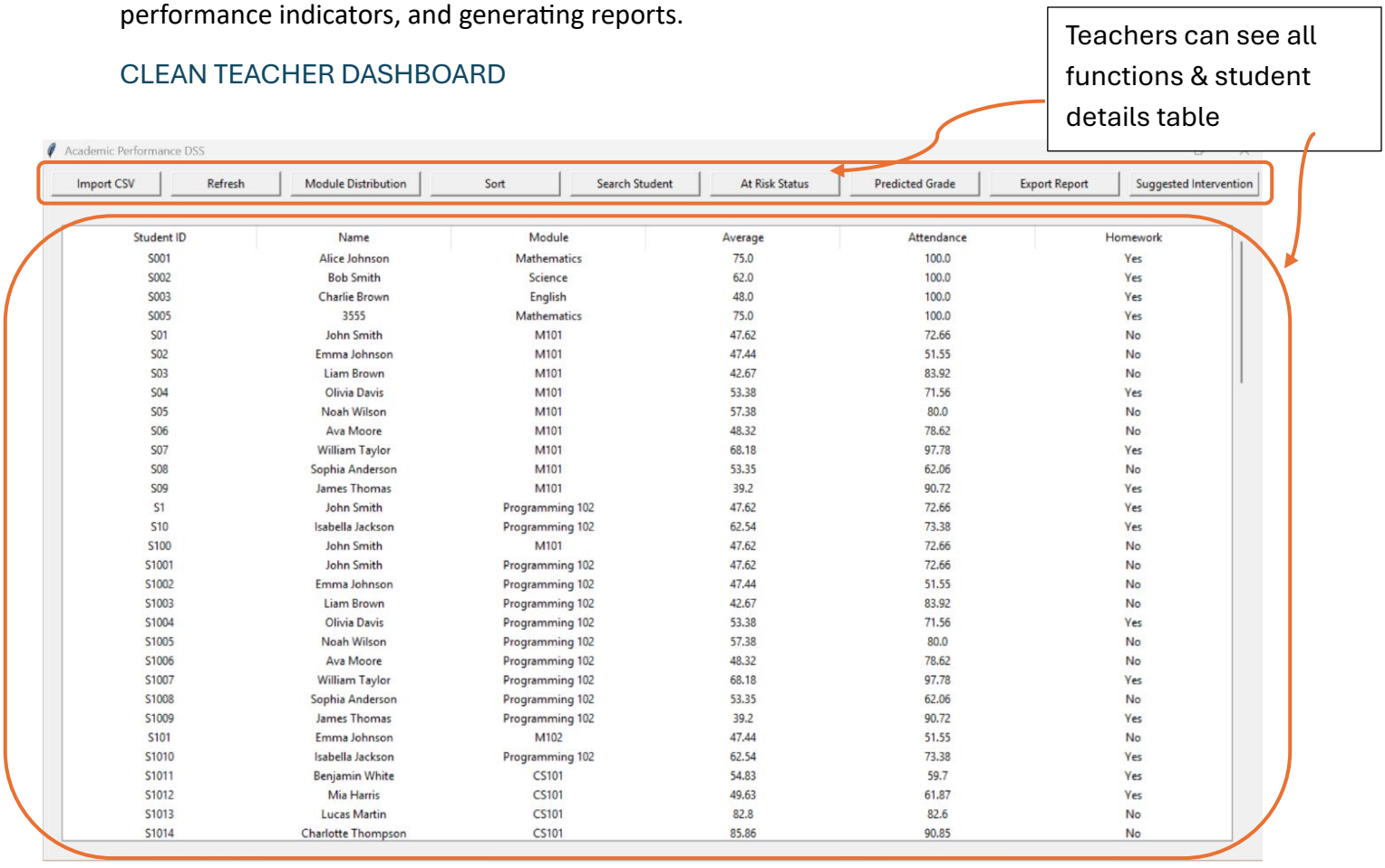


Figure 18

Figure 18 shows a logically structured teacher dashboard with clearly labelled buttons arranged in a consistent layout. Core actions are grouped together and displayed

prominently, allowing users to complete tasks without additional guidance. This demonstrates that the interface is intuitive and easy to learn, fully meeting the success criterion (SC12) identified in the analysis stage.

Usability testing showed that teachers were able to import data, sort records, analyse performance, and export reports without confusion. Parents, who are presented with a reduced interface, were able to locate their child's information and view performance indicators easily. The removal of unnecessary controls for parents reduced cognitive load and prevented accidental misuse of administrative features, further improving usability.

FEEDBACK, VALIDATION, AND ERROR PREVENTION

A key aspect of usability testing was verifying that the system provides clear and immediate feedback when users perform actions or attempt invalid operations. This ensures users understand what the system is doing and how to correct mistakes.

VALIDATION MESSAGE GUIDING USER

All dashboard actions include validations to maintain proper system operation:

- As per the figure 19, the Search Student feature provides below message if no matching student is found.

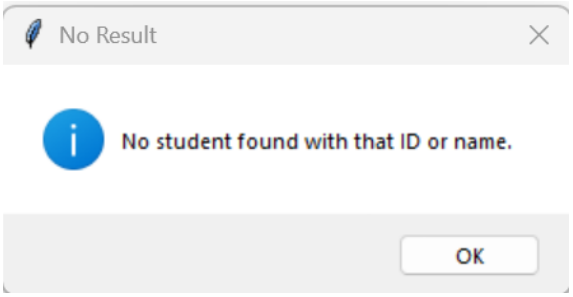


Figure 19

- As per the figure 20, table Actions require a student to be selected before displaying information such as At-Risk Status, Predicted Grade, report generation, or Suggested Interventions. Below validation message indicates the user to select a student first before displaying information.

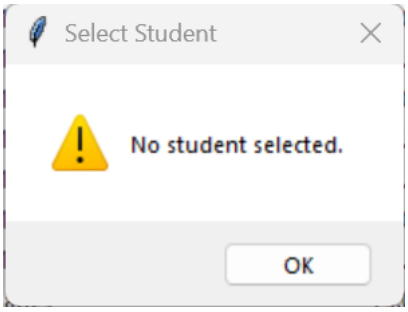


Figure 20

- As per the figure 21, the system validates search input to reject invalid character patterns. The validation error message confirms that incorrect data is detected early, preventing logical errors within the search algorithm and improving system robustness (SC4 and SC11 fully met).

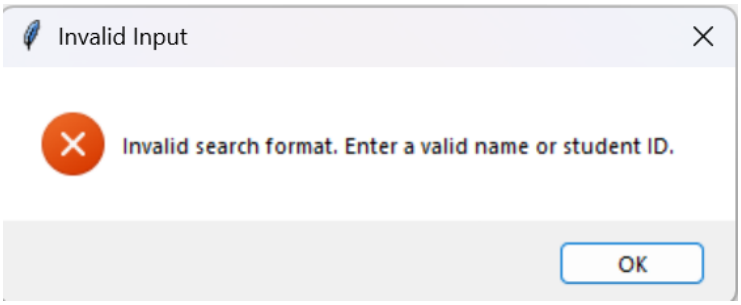


Figure 21

- As per the figure 22, the system correctly completes the search process when no matching records exist. Displaying a “No student found” message confirms that the algorithm handles empty result sets correctly without crashing or producing incorrect data (SC4 fully met).

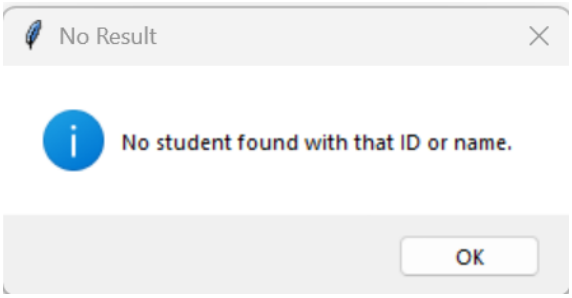


Figure 22

- As per the figure 23, the system correctly validates input before executing the search algorithm. The display of a validation error message confirms that empty inputs are detected and prevented from triggering unnecessary processing, demonstrating correct input validation logic (SC4 and SC11 fully met).

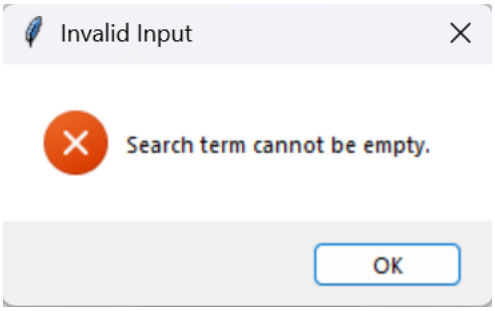


Figure 23

These validations prevent runtime errors, protect data integrity, and guide users in correct system usage.

ROLE-BASED INTERFACE SIMPLIFICATION

Usability testing also evaluated the effectiveness of role-based interface design. Teachers and parents were presented with different interfaces tailored to their needs and permissions.

Teachers were given access to all analytical and administrative tools, while parents were restricted to viewing only relevant information about their own child. This simplification reduced confusion, improved ease of use, and ensured that users were not overwhelmed by unnecessary features. Testing showed that both user types were able to complete tasks efficiently, confirming that the role-based design was effective from a usability perspective.

Overall usability testing confirms that the system is user-friendly, intuitive, and appropriate for both teachers and parents. Users were able to complete all key tasks without assistance, validation messages reduced errors, and feedback helped users understand system responses.

LOGIN SCREEN

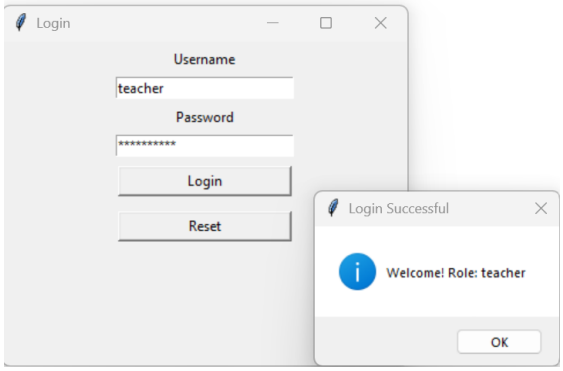


Figure 24

The figure 24 shows the login interface with clearly labelled fields and password masking enabled. This reduces user error and protects sensitive information, contributing to effective usability identified in success criterion (SC12) in analysis stage.

LOGIN FEEDBACK

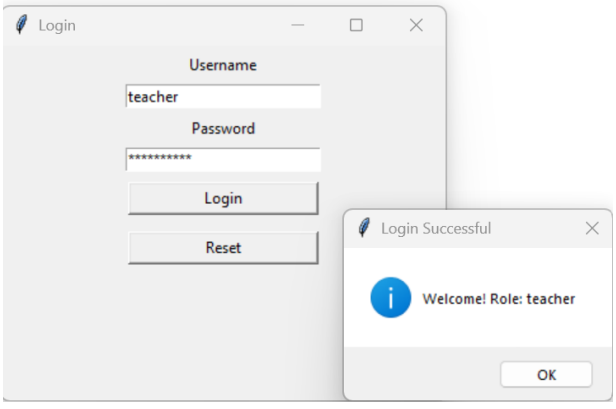


Figure 25

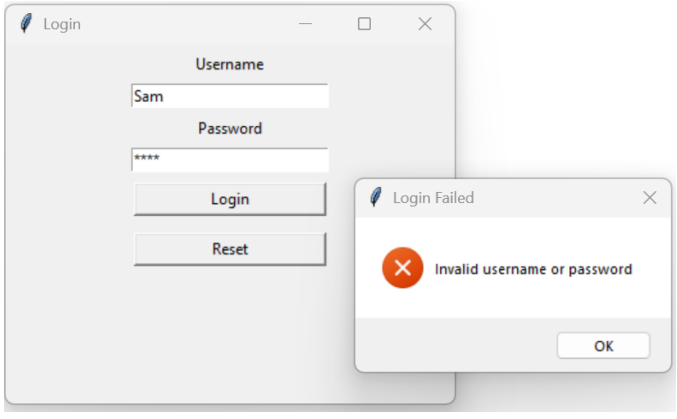


Figure 26

In figure 25 and 26 demonstrates immediate success/error feedback following a login attempt, allowing users to understand system responses without assistance. This improves confidence and confirms effective usability identified in success criterion (SC12) in analysis stage.

EVALUATION OF SUCCESS CRITERIA

The success criteria defined during the analysis stage were evaluated using evidence from post-development testing, robustness testing, and usability testing. The evaluation below summarises which criteria were fully met, partially met, or implicitly met, with justification based on observed system behaviour and annotated evidence.

FULLY MET SUCCESS CRITERIA

The majority of the success criteria were fully met, supported by consistent testing outcomes and annotated evaluation evidence.

The system successfully authenticated users and assigned correct roles, ensuring secure access to sensitive student data (SC1). Role-based access control was correctly enforced, restricting parents from administrative features such as sorting, CSV import, and access to full datasets, while allowing teachers full functionality (SC2).

CSV import and data storage operated reliably, with valid files imported correctly, data persisting after restart, and duplicate student IDs handled appropriately (SC3). The search feature functioned accurately for student names and IDs, including case-insensitive, partial, invalid, and empty inputs, all of which produced correct results or validation messages (SC4).

Decision-support features were consistently accurate. At-risk student identification correctly classified students above, below, and at the boundary threshold of 50% (SC5). Predicted grades were calculated reliably and consistently for identical inputs, with attendance and homework penalties and value clamping applied correctly (SC6). Suggested interventions followed predefined rules for all performance bands (SC7).

Analytical and reporting features were also fully met. Visualisations accurately represented grade distributions using pie charts derived from correct datasets (SC8). PDF report generation produced complete, well-formatted reports containing all required fields and charts for both teachers and parents (SC9).

The system handled invalid input, missing files, and incorrect formats without crashing, displaying appropriate error messages and maintaining stability (SC11).

One success criterion was initially partially met during development. Sorting student data (SC10) required refinement to correctly toggle between ascending and descending order. Although this issue was resolved in later development and the final implementation behaves as intended. Functionally, SC10 is fully met in the final system.

PARTIALLY MET SUCCESS CRITERIA

System performance efficiency (SC13) was judged primarily through observation rather than formal timing measurements. During testing, all key actions such as searching, sorting, and generating reports completed without noticeable delay. However, no explicit response-time measurements were recorded. Therefore, SC13 is best described as partially met or implicitly met, depending on interpretation. For the scope and dataset size of this system, performance was acceptable, but future evaluation could include formal timing tests.

USABILITY SUCCESS CRITERIA

Usability testing confirmed that users were able to complete key tasks without assistance, supported by a clear layout, informative feedback, and role-based interface simplification (SC12).

MAINTENANCE CONSIDERATIONS AND LIMITATIONS

The system has been designed with maintainability in mind through a modular and structured approach. Core functionalities such as authentication, searching, sorting, CSV import, reporting, and decision-support logic are implemented as separate functions and classes. This modular design makes the system easier to maintain, as individual components can be updated, tested, or replaced without affecting the entire program. The use of descriptive function and variable names further supports readability and long-term maintenance.

However, several limitations were identified during evaluation. The system uses SQLite as its database, which is suitable for a small to medium-sized dataset and simplifies deployment. While this is appropriate for the scope of the project, SQLite may become a limitation if the system were scaled to support very large datasets or simultaneous multi-user access. In such scenarios, performance and concurrency could be affected.

Another limitation is the rule-based approach used for at-risk identification, predicted grades, and intervention suggestions. Although this approach is transparent and consistent, it relies on predefined thresholds and does not account for more complex factors such as long-term trends or correlations between multiple performance indicators. This limits predictive accuracy when compared to more advanced analytical techniques.

Finally, the system operates as an offline desktop application, which restricts collaboration and remote access. While this improves simplicity and data privacy, it limits the ability for multiple users to access the system simultaneously or from different locations.

FURTHER DEVELOPMENT

Several realistic improvements could be made to enhance the system in future versions. One possible development is migrating from SQLite to a server-based database such as MySQL (mysql, 2026) or PostgreSQL (PostgreSQL, 2025). This would improve scalability, support concurrent users, and allow deployment in larger institutions.

Authentication could also be enhanced. Currently, user credentials are predefined, which is sufficient for a prototype. In a real-world implementation, authentication could be improved by introducing encrypted password storage (Vaadata , 2020). , individual parent accounts, and role-based user management stored securely in the database.

The decision-support features could be developed further by incorporating advanced predictive models. Machine learning techniques or statistical analysis could be used to consider historical performance trends, attendance patterns, and homework completion over time, improving the accuracy of predicted grades and at-risk identification.

The sorting functionality could also be expanded in future versions. Although the current system implements fixed multi-key sorting using predefined attributes, this could be enhanced to allow user-controlled, dynamic multi-key sorting. Teachers could be given the ability to select which attributes to sort by, such as average score, attendance, module, or predicted grade, and define the priority order of these keys. This would improve flexibility and make it

CONCLUSION

In conclusion, the evaluation shows that the Student Performance Decision Support System successfully achieves its intended goals. Through thorough post-development and usability testing, clear evidence was gathered to demonstrate that the system is functional, reliable, and easy to use. Most of the success criteria were fully met, with any limitations honestly identified and discussed.

The system effectively supports decision-making by identifying at-risk students, generating predicted grades and suggested interventions, providing visual insights, and automating report generation. Usability testing also showed that both teachers and parents were able to use the system confidently without assistance, thanks to clear feedback, input validation, and a well-designed role-based interface.

The sorting functionality could also be expanded in future versions. Although the current system implements fixed multi-key sorting using predefined attributes, this could be enhanced to allow user-controlled, dynamic multi-key sorting. Teachers could be given the ability to select which attributes to sort by, such as average score, attendance, module, or predicted grade, and define the priority order of these keys. This would improve flexibility and make it easier to analyse student performance from different perspectives.

While there are some limitations such as scalability, predictive accuracy, and deployment scope these are reasonable given the size of the project. Importantly, practical and achievable improvements have been identified for future development.

REFERENCES

PostgreSQL (2025). *PostgreSQL: About*. [online] Postgresql.org. Available at: https://www.postgresql.org/about/.

dev.mysql.com. (n.d.). *MySQL :: MySQL 8.4 Reference Manual*. [online] Available at: https://dev.mysql.com/doc/refman/8.4/en/.

Vaadata (2020). *How to securely store passwords in database*. [online] www.vaadata.com. Available at: https://www.vaadata.com/blog/how-to-securely-store-passwords-in-database/.

for, G. (2025). *Classroom Management Tools & Resources - Google for Education*. [online] Google for Education. Available at: https://edu.google.com/intl/ALL_uk/workspace-for-education/products/classroom/.

Moodle (2017). *Moodle*. [online] Moodle. Available at: https://moodle.com/.

www.tes.com. (n.d.). *School Seating Planner & Behaviour Management Software | Class Charts - Tes*. [online] Available at: https://www.tes.com/en-gb/for-schools/class-charts.

Mohan, S., 2015. *OCR A Level Computer Science*. Deddington: PG Online.

Ceredig Cattanach-Chell (2019). *Tackling A level projects in computer science: OCR A level computer science H446*. Tolpuddle: Pg Online Limited.

Sommerville, I., 2016. *Software Engineering*. 10th ed. Harlow: Pearson.

Chicha study (2026). *Decision Support System (DSS) Explained | Definition, Components & Examples*. [online] YouTube. Available at: https://www.youtube.com/watch?v=mHWudCh2YSs [Accessed 27 Feb. 2026].

Programming with Mosh, 2019. *Python Tutorial for Beginners – Full Course*. YouTube video. Available at: https://www.youtube.com/watch?v=_uQrJ0TkZlc (Accessed: 27 February 2026).

Schafer, C. (2017). *Python SQLite Tutorial: Complete Overview - Creating a Database, Table, and Running Queries*. *YouTube*. Available at: https://www.youtube.com/watch?v=pd-0G0MigUA.

Gillis, A. (2021). *What is a decision support system (DSS)?* [online] SearchCIO. Available at: https://www.techtarget.com/searchcio/definition/decision-support-system.

Python Software Foundation, 2024. *Python Documentation*. Available at: https://docs.python.org/3/ (Accessed: 27 February 2026).

lucid.co. (n.d.). *Lucid Software | A Visual Collaboration Suite*. [online] Available at: https://lucid.co/.

GeeksforGeeks (2017). *Python Tkinter*. [online] GeeksforGeeks. Available at: https://www.geeksforgeeks.org/python/python-gui-tkinter/.

freeCodeCamp.org (2020). SQLite Databases With Python - Full Course. YouTube. Available at: https://www.youtube.com/watch?v=byHcYRpMgI4

W3schools.com. (2025). W3Schools.com. [online] Available at: https://www.w3schools.com/python/python_dsa_bubblesort.asp

GeeksforGeeks (2016). Linear Search Python. [online] GeeksforGeeks. Available at: https://www.geeksforgeeks.org/python/python-program-for-linear-search/

ReportLab PDF Library User Guide. (n.d.). Available at: https://www.reportlab.com/docs/reportlab-userguide.pdf

matplotlib.org. (n.d.). Basic pie chart — Matplotlib 3.3.4 documentation. [online] Available at: https://matplotlib.org/stable/gallery/pie_and_polar_charts/pie_features.html.

Schafer, C. (2017). Python Tutorial: CSV Module - How to Read, Parse, and Write CSV Files. YouTube. Available at: https://www.youtube.com/watch?v=q5uM4VKywbA.

freeCodeCamp.org (2019). Tkinter Course - Create Graphic User Interfaces in Python Tutorial. YouTube. Available at: https://www.youtube.com/watch?v=YXPyB4XeYLA

APPENDIX

FULL CODE

```
import os
import sqlite3
import tkinter as tk
from tkinter import ttk, messagebox, filedialog, simpledialog
from reportlab.lib.pagesizes import A4
from reportlab.pdfgen import canvas
from collections import Counter
import matplotlib.pyplot as plt
import csv

DB_FILE = "student_dss.db"

#------------------------***USERS***-----------------------------------------------------------
USERS = {
    "teacher": {"password": "teacher123", "role": "teacher"},
    "parent": {"password": "parent123", "role": "parent"}
}

current_role = None
students = []
tree = None
```

```
#-------------------------***STUDENT CLASS***-----------------------------------------------------
class Student:
    def __init__(self, student_id, name, module, average, attendance=100, homework_complete=1):
        self.student_id = student_id
        self.name = name
        self.module = module
        self.average = average
        self.attendance = attendance
        self.homework_complete = homework_complete
    # ------------------*** AT-RISK DETENTION FUNCTION***---------------------------- ----------------
--
    def at_risk(self):
        return "Yes" if self.average < 50 else "No"
    # ------------------ ***PREDICTED GRADE FUNCTION***-------------------------------- ------
    def predicted_grade(self):
        score = self.average
        if self.attendance < 75:
            score -= 10
        elif self.attendance < 90:
            score -= 5
        if not self.homework_complete:
            score -= 5
        score = max(0, min(100, score))
        if score >= 70:
            return "A"
        if score >= 60:
            return "B"
        if score >= 50:
            return "C"
        if score >= 40:
            return "D"
        return "E"
    # ------------------***SUGGESTED INTERVENTION FUNCTION***---------------------------- -------------
---
    def suggested_intervention(self):
        if self.average >= 70:
            return "None"
        elif self.average >= 60:
            return "Encourage Practice"
        elif self.average >= 50:
            return "Extra Tutoring"
        else:
            return "Immediate Intervention"
```

```python
#------------***freeCodeCamp.org (2020). SQLite Databases With Python - Full Course. YouTube. Available
at: https://www.youtube.com/watch?v=byHcYRpMgI4.
# ----------------***creating the student table***-----------------------------------------------
def db_connect():
    return sqlite3.connect(DB_FILE)

def init_db():
    con = db_connect()
    cur = con.cursor()
    cur.execute("""
        CREATE TABLE IF NOT EXISTS students (
            student_id TEXT PRIMARY KEY,
            name TEXT NOT NULL,
            module TEXT NOT NULL,
            average REAL NOT NULL,
            attendance REAL DEFAULT 100,
            homework_complete INTEGER DEFAULT 1
        )
    """)
    con.commit()
    con.close()

def load_students():
    students.clear()
    con = db_connect()
    cur = con.cursor()
    cur.execute("""
        SELECT student_id,name,module,average,attendance,homework_complete
        FROM students
        ORDER BY student_id
    """)
    for sid, name, module, avg, att, hw in cur.fetchall():
        students.append(Student(sid, name, module, avg, att, hw))
    con.close()

#----------------------------***STUDENT TREE VIEW REFRESH FUNCTION-----------------------------------
----------
def refresh_table(data=None):
    if tree is None:
        return
    for r in tree.get_children():
        tree.delete(r)
    for s in data if data else students:
        tree.insert("", "end", values=(
            s.student_id,
            s.name,
            s.module,
            s.average,
            s.attendance,
            "Yes" if s.homework_complete else "No"
        ))

def refresh_table_button():
    refresh_table(students)
```

```python
#-----------------***W3schools.com. (2025). W3Schools.com. [online]
# Available at: https://www.w3schools.com/python/python_dsa_bubblesort.asp***--------------------

#-------------***SORTING STUDENT TABLE***-----------------------------------------------------------
def sort_students(student_list, ascending=True):

     #-------------------***INPUT VALIDATION***-------------------------------------
    if student_list is None or not isinstance(student_list, list):
        return []
    n = len(student_list)

    #-------------***BUBBLE SORT TO SORT STUDENT TABLE IN ASCENDING & DESCENDING ORDER***-------
    for i in range(n):
        for j in range(0, n - i - 1):
            a = student_list[j]
            b = student_list[j + 1]
            #----------------------****CREATE COMPARISON KEYS***--------------------------------
            key_a = (a.student_id, a.name, a.module, a.average)
            key_b = (b.student_id, b.name, b.module, b.average)
            #------------------------***SORTING IN ASCENDING & DESCENDING ORDERS***-------------
            if ascending:
                if key_a > key_b:
                    student_list[j], student_list[j+1] = student_list[j+1], student_list[j]
            else:
                if key_a < key_b:
                    student_list[j], student_list[j+1] = student_list[j+1], student_list[j]
    return student_list
```

```
#------GeeksforGeeks (2016). Linear Search Python. [online] GeeksforGeeks.
# Available at: https://www.geeksforgeeks.org/python/python-program-for-linear-search/

#--------------------***SEARCH STUDENTS FUNCTION***-----------------------------------------------
def search_students(students, search_term):
    if not search_term:
        return []
    search_term = search_term.lower()
    matches = []
    for s in students:
        if search_term in s.name.lower() or search_term in s.student_id.lower():
            matches.append(s)
    return matches

def search_ui():
    term = simpledialog.askstring("Search", "Enter Student ID or Full Name")

    if term is None:
        return  #------------***FOR CANCEL SEARCH***----------------

    valid, error_message = is_valid_search_term(term)

    if not valid:
        messagebox.showerror("Invalid Input", error_message)
        return

    results = search_students(students, term)

    if current_role == "teacher":
        if results:
            refresh_table(results)
        else:
            messagebox.showinfo("No Result", "No student found with that ID or name.")

    elif current_role == "parent":
        if results:
            s = results[0]
            messagebox.showinfo(
                "Student Found",
                f"Name: {s.name}\n"
                f"Module: {s.module}\n"
                f"Average: {s.average}\n"

            )
        else:
            messagebox.showinfo("No Result", "No student found with that ID or name.")
```

```
#----------------------***TO VALIDATE SEARCH TERMS***-----------------------------
def is_valid_search_term(term):
    term = term.strip()

    #---------------***FOR EMPTY INPUT***-------------------------------------
    if not term:
        return False, "Search term cannot be empty."

    #--------------***STUDENT ID VALIDATION***---------------------------------
    if term.replace(" ", "").isalnum():
        return True, None

    #----------------***STUDENT NAME VALIDATION***-------------------------------
    if all(c.isalpha() or c.isspace() for c in term):
        return True, None

    return False, "Invalid search format. Enter a valid name or student ID."
```

```
# --------------- ***SELECT STUDENT*** ------------------------------------------------------
def get_selected_student():
    """
    Returns the currently selected student from the table.
    Works only for roles that can see the table (teachers). when
    student object if selected, None otherwise.

    """
    try:
        if tree is None:
           return None
        #---------- ***TO NOTIFY THE USER THAT STUDENT IS NOT SELECTED***-------------------
        sel = tree.focus()
        if not sel:
            messagebox.showwarning("Select Student", "No student selected.")
            return None

        sid = tree.item(sel)["values"][0]
        for s in students:
            if s.student_id == sid:
                return s

        messagebox.showwarning("Select Student", "Selected student not found in records.")
        return None
    except Exception as e:
        messagebox.showerror("Error", f"Error selecting student: {e}")
        return None
```

```
# ---------------- ***SELECT STUDENT*** ------------------------------------------------------------
# ---------------- ***ROLE BASED AT RISK LEVEL STATUS***------------------------ ----------------
def show_at_risk():
    """
    Display whether a student is at risk.
    - Parents: Search for their child first
    - Teachers: Use selected row from table
    """
    try:
        if current_role == "parent":
            #search_ui()
              term = simpledialog.askstring("Search", "Enter Student ID or Full Name")
              if not term or term.strip() == "":
                 messagebox.showerror("Invalid Input", "Search term cannot be empty.")
                 return
              results = search_students(students, term)
              if results:
                s = results[0]
                messagebox.showinfo("At Risk Status", f"{s.name}:At Risk: {s.at_risk()}")
              else:
               messagebox.showinfo("No Result", "No student found with that ID or name.")


        s = get_selected_student()
        if s:
            messagebox.showinfo("At Risk Status", f"{s.name} At Risk: {s.at_risk()}")
    except Exception as e:
        messagebox.showerror("Error", f"Sorry, cannot display At Risk status: {e}")

# ---------------- ***ROLE BASED PREDICTED GRADE STATUS***------------------------ -------------
def show_predicted_grade():
    """
    Display predicted grade for a student.
    - Parents: Search for their child first
    - Teachers: Use selected row from table
    """
    try:
        if current_role == "parent":
            #search_ui()
            term = simpledialog.askstring("Search", "Enter Student ID or Full Name")
            if not term or term.strip() == "":
               messagebox.showerror("Invalid Input", "Search term cannot be empty.")
               return
            results = search_students(students, term)
            if results:
               s = results[0]
               messagebox.showinfo("Predicted Grade", f"{s.name} Predicted Grade:
{s.predicted_grade()}")
            else:
               messagebox.showinfo("No Result", "No student found with that ID or name.")

        s = get_selected_student()
        if s:
            messagebox.showinfo("Predicted Grade", f"{s.name} Predicted Grade: {s.predicted_grade()}")
    except Exception as e:
        messagebox.showerror("Error", f"Cannot display Predicted Grade: {e}")
```

```
# ---------------- ***ROLE SHOW INTERVENTION STATUS***----------------------- --------------
def show_intervention():
    """
    Display suggested intervention for a student.
    - Teachers: Use the selected row from the table
    - Parents: Search for their child first
    """
    try:
        if current_role == "parent":

            term = simpledialog.askstring("Search", "Enter Student ID or Full Name")
            if not term or term.strip() == "":
                messagebox.showerror("Invalid Input", "Search term cannot be empty.")
                return
            results = search_students(students, term)
            if results:
                s = results[0]
                messagebox.showinfo("Suggested Intervention",f"{s.name} - Suggested Intervention:
{s.suggested_intervention()}")
            else:
                messagebox.showinfo("No Result", "No student found with that ID or name.")

        #-------------*** Teacher selects from table***----------------------
        s = get_selected_student()
        if s:
            messagebox.showinfo(
                "Suggested Intervention",
                f"{s.name} - Suggested Intervention: {s.suggested_intervention()}"
            )


    except Exception as e:
        messagebox.showerror("Error", f"Cannot display suggested intervention: {e}")


#-------------***ReportLab PDF Library User Guide. (n.d.). Available at:
https://www.reportlab.com/docs/reportlab-userguide.pdf
# ---------------- ---------***EXPORT REPORT***----------------------------- ----------------
def export_student_report():
    #----------------------***PARENT SEARCH FOR THEIR CHILD***-------------------------------
    if current_role == "parent":
        term = simpledialog.askstring("Search", "Enter Student ID or Name")
        if not term or term.strip() == "":
            messagebox.showerror("Invalid Input", "Search term cannot be empty.")
            return
        results = search_students(students, term)
        if not results:
            messagebox.showinfo("No Result", "No student found with that ID or name.")
            return
        s = results[0]
    #----------------------***TEACHER SELECT STUDENT FROM TABLE***---------------------------
    else:
        s = get_selected_student()
        if not s:
            return

    module_students = [st for st in students if st.module == s.module]
    grades = Counter(st.predicted_grade() for st in module_students)

    #------***matplotlib.org. (n.d.). Basic pie chart — Matplotlib 3.3.4 documentation. [online]
Available at: https://matplotlib.org/stable/gallery/pie_and_polar_charts/pie_features.html.
    #----------------------***CREATE MODULE GRADE DISTRIBUTION PIE CHART***-----------------
    fig = plt.Figure(figsize=(4,4))
    ax = fig.add_subplot(111)
    ax.pie(list(grades.values()), labels=list(grades.keys()), autopct="%1.1f%%")
    ax.set_title(f"{s.module} - Grade Distribution")
    #----------------------***SAVE TEMPORARY CHART IMAGE***---------------------------------
    pie_path = "temp_pie.png"
    fig.savefig(pie_path, bbox_inches="tight", dpi=150)
    plt.close(fig)
    #----------------------***SELECT LOCATION TO SAVE PDF REPORT***-------------------------
    pdf_filename = filedialog.asksaveasfilename(defaultextension=".pdf")
    if not pdf_filename:
        os.remove(pie_path)
        return
```

```
#-------------***ReportLab PDF Library User Guide. (n.d.). Available at:
https://www.reportlab.com/docs/reportlab-userguide.pdf
    #----------------------***CREATE PDF REPORT***-----------------------------------------
    c = canvas.Canvas(pdf_filename, pagesize=A4)
    width, height = A4
    c.setFont("Helvetica-Bold", 16)
    c.drawString(50, height - 50, f"Student Report - {s.name}")
    c.setFont("Helvetica", 12)
    y = height - 80

    #----------------------***ADD STUDENT DETAILS TO REPORT***------------------------------
    fields = [
        ("Student ID", s.student_id),
        ("Name", s.name),
        ("Module", s.module),
        ("Average", s.average),
        ("Attendance", s.attendance),
        ("Homework Complete", "Yes" if s.homework_complete else "No"),
        ("At Risk", s.at_risk()),
        ("Suggested Intervention", s.suggested_intervention()),
        ("Predicted Grade", s.predicted_grade())
    ]
    for label, value in fields:
        c.drawString(50, y, f"{label}: {value}")
        y -= 18


    #----------------------***INSERT PIE CHART INTO PDF***----------------------------------
    y -= 10
    image_height = min(y - 40, 300)
    image_width = image_height
    x_pos = (width - image_width) / 2
    y_pos = y - image_height
    c.drawImage(pie_path, x_pos, y_pos, width=image_width, height=image_height)
    c.save()
    os.remove(pie_path)

    messagebox.showinfo("Success", "Report Exported Successfully")
```

```
#--------------------**Schafer, C. (2017). Python Tutorial: CSV Module - How to Read, Parse, and Write
CSV Files. YouTube. Available at: https://www.youtube.com/watch?v=q5uM4VKywbA.
#-------------------------***IMPORTING STUDENT RECORDS FROM CSV file***-------------------------------
-
def import_csv():

    #----------------------***OPEN FILE DIALOG TO SELECT CSV FILE***-------------------------
    try:
        path = filedialog.askopenfilename(filetypes=[("CSV","*.csv")])
        if not path:
            return

        #----------------------***ENSURE STUDENT TABLE EXISTS***-----------------------------
        init_db()

        con = db_connect()
        cur = con.cursor()

        #----------------------***READ CSV FILE AND INSERT DATA INTO DATABASE***--------------
        with open(path, "r", newline='', encoding='utf-8') as f:
            reader = csv.DictReader(f)
            for row in reader:
                cur.execute("""
                    INSERT OR REPLACE INTO students
                    (student_id, name, module, average, attendance, homework_complete)
                    VALUES (?, ?, ?, ?, ?, ?)
                """, (
                    row["Student ID"],
                    row["Name"],
                    row["Module"],
                    float(row["Average"]),
                    float(row.get("Attendance", 100)),
                    1 if row.get("Homework", "Yes").lower() in ["yes","1","true"] else 0
                ))

        #----------------------***SAVE CHANGES AND CLOSE DATABASE***-------------------------
        con.commit()
        con.close()

        load_students()
        refresh_table()

        messagebox.showinfo("Success", "CSV Data Imported Successfully")
```

```
#----------------------***HANDLE FILE OR DATA ERRORS***---------------------------------
    except FileNotFoundError:
        messagebox.showerror("Error", "CSV file not found.")

    except ValueError:
        messagebox.showerror("Error", "Invalid data format in CSV file.")

    except KeyError as e:
            messagebox.showerror("CSV Error", f"Missing column: {e}")
            return
  # ----------------***MODULE DISTRIBUTION FUNCTION***-------- ----------------
def module_distribution():
    module = simpledialog.askstring("Module", "Enter Module Name")
    if not module:
        return
    module_students = [s for s in students if s.module.lower() == module.lower()]
    if not module_students:
        messagebox.showinfo("No Data", f"No students found for module: {module}")
        return
    grades = [s.predicted_grade() for s in module_students]
    grade_count = Counter(grades)
    plt.pie(grade_count.values(), labels=grade_count.keys(), autopct="%1.1f%%")
    plt.title(module + " Grade Distribution")
    plt.show()

# ---------------- ***SORT STUDENTS UI FUNCTION*** --------------------------
sort_ascending = True

def sort_students_ui():
    global students, sort_ascending
    students = sort_students(students, ascending=sort_ascending)
    sort_ascending = not sort_ascending
    refresh_table(students)
```

```
#----------***freeCodeCamp.org (2019). Tkinter Course - Create Graphic User Interfaces in Python
Tutorial. YouTube. Available at: https://www.youtube.com/watch?v=YXPyB4XeYLA
#-------------------------***Main Window (Dashboard) UI***------------------------
def build_main_window(role):
    global tree
    tree = None

    root = tk.Tk()
    root.title("Academic Performance DSS")
    root.geometry("1400x600")

    btn_frame = tk.Frame(root)
    btn_frame.pack(pady=10)

    if role == "teacher":
        tk.Button(btn_frame, text="Import CSV", width=15, command=import_csv).pack(side=tk.LEFT,
padx=5)
        tk.Button(btn_frame, text="Refresh", width=15, command=refresh_table_button).pack(side=tk.LEFT,
padx=5)
        tk.Button(btn_frame, text="Module Distribution", width=18,
command=module_distribution).pack(side=tk.LEFT, padx=5)
        tk.Button(btn_frame, text="Sort", width=18, command=sort_students_ui).pack(side=tk.LEFT,
padx=5)

    tk.Button(btn_frame, text="Search Student", width=18, command=search_ui).pack(side=tk.LEFT, padx=5)
    tk.Button(btn_frame, text="At Risk Status", width=18, command=show_at_risk).pack(side=tk.LEFT,
padx=5)
    tk.Button(btn_frame, text="Predicted Grade", width=18,
command=show_predicted_grade).pack(side=tk.LEFT, padx=5)
    tk.Button(btn_frame, text="Export Report", width=18,
command=export_student_report).pack(side=tk.LEFT, padx=5)
    tk.Button(btn_frame, text="Suggested Intervention", width=18,
command=show_intervention).pack(side=tk.LEFT, padx=5)
```

```
#--------------***ROLE BASED (UI)***-------------------------------------
    if role == "teacher":
        table_frame = tk.Frame(root)
        table_frame.pack(fill=tk.BOTH, expand=True, padx=20, pady=20)

        columns = ("Student ID", "Name", "Module", "Average", "Attendance", "Homework")
        tree = ttk.Treeview(table_frame, columns=columns, show="headings")
        for col in columns:
            tree.heading(col, text=col)
            tree.column(col, width=160, anchor="center")

        scrollbar = ttk.Scrollbar(table_frame, orient="vertical", command=tree.yview)
        tree.configure(yscrollcommand=scrollbar.set)
        tree.pack(side=tk.LEFT, fill=tk.BOTH, expand=True)
        scrollbar.pack(side=tk.RIGHT, fill=tk.Y)

        refresh_table()

    root.mainloop()

# ---------------***LOGIN FUNCTION***--------------------- ----------------
def login():
    global current_role
    username = username_entry.get()
    password = password_entry.get()
    if username in USERS and USERS[username]["password"] == password:
        current_role = USERS[username]["role"]
        messagebox.showinfo("Login Successful", f"Welcome! Role: {current_role}")
        login_window.destroy()
        build_main_window(current_role)
    else:
        messagebox.showerror("Login Failed", "Invalid username or password")

def reset_fields():
    username_entry.delete(0, tk.END)
    password_entry.delete(0, tk.END)
    username_entry.focus()

#----------***freeCodeCamp.org (2019). Tkinter Course - Create Graphic User Interfaces in Python
Tutorial. YouTube. Available at: https://www.youtube.com/watch?v=YXPyB4XeYLA
# ---------------- ***START PROGRAM***---------------------- ----------------
init_db()
load_students()

login_window = tk.Tk()
login_window.title("Login")
login_window.geometry("350x280")

tk.Label(login_window, text="Username").pack(pady=5)
username_entry = tk.Entry(login_window, width=25)
username_entry.pack()

tk.Label(login_window, text="Password").pack(pady=5)
password_entry = tk.Entry(login_window, show="*", width=25)
password_entry.pack()

tk.Button(login_window, text="Login", width=20, command=login).pack(pady=8)
tk.Button(login_window, text="Reset", width=20, command=reset_fields).pack(pady=5)

login_window.mainloop()
```